

university of groningen

Performance Engineering: Lecture #01

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-08-31

Overview



1. Course Information
2. Computing Systems Organization
3. Performance Engineering
4. Performance and Performance Metrics
5. Final

Content



1. Course Information
2. Computing Systems Organization
3. Performance Engineering
4. Performance and Performance Metrics
5. Final

Course Practices



- **Questions are welcome at any time!**
  - Do not only consume information, reason about concepts
- **On-site presence is recommended**
  - No video broadcasting or recording is foreseen
- **On-site presence during project presentations is mandatory**
  - Exceptions need to be agreed ahead of the sessions
- **Assignments will be performed in groups**
  - *Remember*: During the exam you are at your own

Course Objective



- The purpose of this course is to learn **fundamental aspects and techniques** used for performance engineering
  - We will look at specific cases to create a better understanding
  - Technical details, e.g. about tools, will not be assessed during the exam

Course Topics



The course will cover the following topics

1. Performance and performance measurement
2. Compilers
3. Performance models
4. Optimizations at the processor core level
5. Optimizations at the memory subsystem level
6. Data structures
7. Optimization of parallel applications
8. Optimization for accelerators
9. I/O optimization

Available Experience



- **Question**: How many course participants have experience with the following technologies?
  - OpenMP
  - MPI
- **Note**: C or C++ programming experience is assumed

Intended Learning Outcomes (ILOs)



- **ILO #1**: Apply and assess different established optimization techniques
- **ILO #2**: Demonstrate and compare several performance modelling methods, and assess their usefulness for practical problems with focus on assessing attainable performance for a given application and computer architecture
- **ILO #3**: Design an empirical performance analysis process for any application, interpret its results, and recommend solutions for performance improvement
- **ILO #4**: Use different performance engineering techniques and tools including performance analysis tools and benchmarking techniques

Course Assessment



- For assessing your performance, you will have to accomplish the following tasks:
  - **Task #1**: Follow lab assignments
  - **Task #2**: Project
  - **Task #3**: The final exam
- Tasks #1 and #2 will be done in groups of 2 students

How are the ILOs Assessed?

	Task #1	Task #2	Task #3
ILO #1	✔	✔	✔
ILO #2	✔		✔
ILO #3		✔	
ILO #4	✔	✔	

Task #1: Lab Assignments



- One lab assignment per week
- They are graded P/F
- One resubmission is possible

Task #2: Project



- Performance analysis, modelling, and optimization for an application of choice
- Results should be documented in a report and presented during the last week
- Both report and presentation will be graded 1-10
  - Weight for final grade: 25% and 15%

Use of AI



- Use of Large Language Model (LLM) services (such as ChatGPT, DeepSeek etc.):
  - Is **allowed** for text polishing
  - Is **not allowed** for generation of full-text blocks
  - Is **allowed** for code generation, but you need to
    - Verify correctness
    - Proof, analyse, and explain performance behaviour
- You remain fully responsible for the submitted content
- Specific rules for using LLMs:
  - Add a reference with name and version of the LLM
  - Add a clarification if a paid version is used
  - Document all prompts
- Otherwise: **Reporting on experiences with AI is encouraged**

Task #3: Exam



- The final exam covers all the topics presented during the course
- It is graded 1-10
- Exam structure:
  - Multiple-choice and open questions
  - All 9 course topics will be covered
- Open questions will receive more points

Course Practices: Labs and Project



- Practical work to be implemented on device of your choice
  - Typically your own device
    - Installing software will be your responsibility
  - Access to an HPC cluster will be provided
    - Register for account in this Google Sheet
      IMPORTANT: Enter key using the OpenSSH key format
- **Documentation is key** to guarantee **reproducibility** of the results
  - Code
  - Hardware
  - Development environment (e.g., compiler)



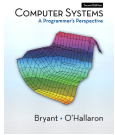
Course Text Books

- D. Bakhvalov, “Performance Analysis and Tuning on Modern CPUs,” 2nd edition, 2024. [Online]. Available: `https://github.com/dendibakh/perf-book/releases`

- R. E. Bryant and D. R. O’Hallaron, “Computer Systems: A Programmer’s Perspective,” 2nd edition, Pearson, 2011

- Georg Hager and Gerhard Wellein, “Introduction to high performance computing for scientists and engineers,” 1st edition, CRC, 2010. doi: 10.1201/EBK1439811924.







Content



1. Course Information
2. Computing Systems Organization
3. Performance Engineering
4. Performance and Performance Metrics
5. Final

Definition



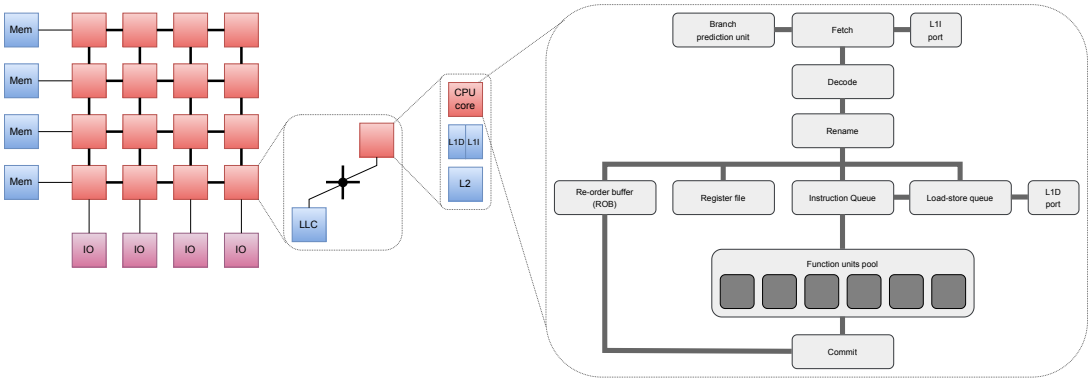
- **Computing system** = Integrated set of hardware and software components designed to transport, process, and store data
- Selected typical components
  - Central Processing Unit (CPU)
  - Storage hardware
  - Network hardware
  - Operating system
  - Compilers
  - ...

Complex Systems



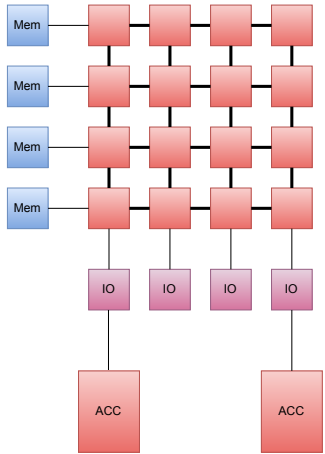
- **Complex system** = A system composed of multiple components or constituents that interact
- The interaction leads to properties of this system that cannot be easily inferred from the single components or constituents
- **Computing systems are complex systems**

Example: Many-Core CPU Architecture

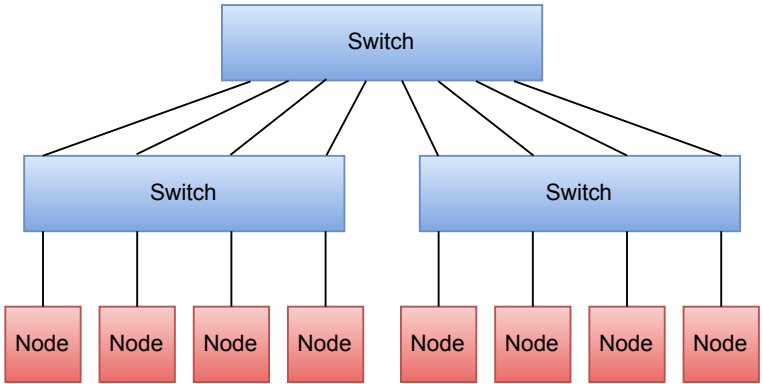


Accelerator

- Accelerators are mostly discrete and attached via an I/O interface to a CPU
- Examples:
  - GPU
  - TPU
  - Neuromorphic devices

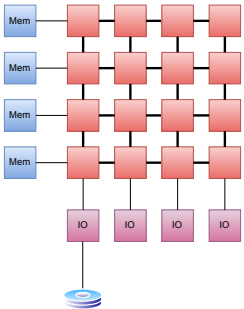


Multi-Node Systems

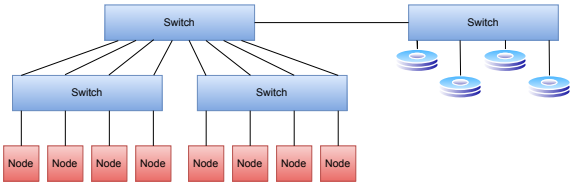


Storage

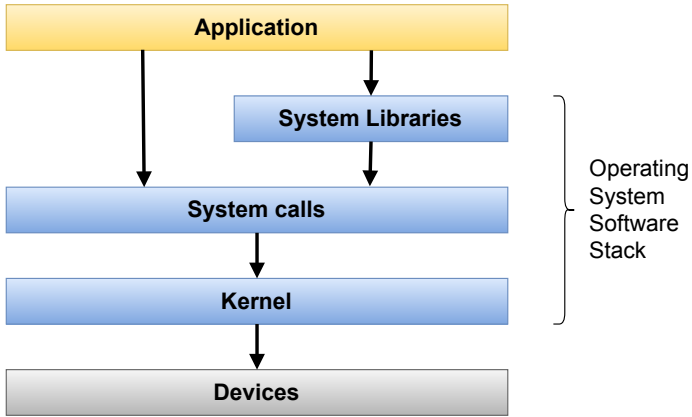
Node-local storage:



System-level storage:



System Software



Content



1. Course Information
2. Computing Systems Organization
3. **Performance Engineering**
4. Performance and Performance Metrics
5. Final

Definition



- **Performance Engineering** = Systematic endeavour to enhance the efficiency of software applications that satisfy all functional requirements
- Differing definitions of efficiency are permissible under this definition
  - **Performance** = Efficient use of the target system
  - **Energy efficiency** = Work per energy unit
- Performance engineering efforts should **not change the functional behaviour** of the software application

Importance of Performance Engineering



- Example speed-ups from performance engineering:
  A program that multiplies two 4096×4096 matrices

[Leiserson et al.; 2020]

Version	Implementation	Running time (s)	GFLOPS	Absolute speedup	Relative speedup	Fraction of peak (%)
1	Python	25,552.48	0.005	1	–	0.00
2	Java	2,372.68	0.058	11	10.8	0.01
3	C	542.67	0.253	47	4.4	0.03
4	Parallel loops	69.80	1.969	366	7.8	0.24
5	Parallel divide and conquer	3.80	36.180	6,727	18.4	4.33
6	plus vectorization	1.10	124.914	23,224	3.5	14.96
7	plus AVX intrinsics	0.41	337.812	62,806	2.7	40.45

Methodologies Overview

- Many different approaches to performance engineering have been identified in the literature
- Different approaches can be combined

☞ **There is no unique approach to performance engineering**

[Brendan Gregg; Prentice Hall; 2013]

Methodology	Type
Streetlight anti-method	observational analysis
Random change anti-method	experimental analysis
Blame-someone-else anti-method	hypothetical analysis
Ad hoc checklist method	observational and experimental analysis
Problem statement	information gathering
Scientific method	observational analysis
Diagnosis cycle	analysis life cycle
Tools method	observational analysis
USE method	observational analysis
Workload characterization	observational analysis, capacity planning
Drill-down analysis	observational analysis
Latency analysis	observational analysis
Method R	observational analysis
Event tracing	observational analysis
Baseline statistics	observational analysis
Performance monitoring	observational analysis, capacity planning
Queueing theory	statistical analysis, capacity planning
Static performance tuning	observational analysis, capacity planning
Cache tuning	observational analysis, tuning
Micro-benchmarking	experimental analysis
Capacity planning	capacity planning, tuning

Ad-Hoc Checklist Methodology



[Brendan Gregg; Prentice Hall; 2013]

- **Approach**: Process a **checklist** with each entry of the type:
  Run application X and if observation O, then take action A_1 or A_2 or ...
- Possible information contained in checklist entries
  - Type of performance tool to run
  - Instructions on how to interpret the results
  - Suggested actions
- **Advantages**: Easy to follow, can be fast, can be prescriptive
- **Disadvantages**: Limited to known performance issues, risk of missing **hidden parameters**

Problem Statement Methodology



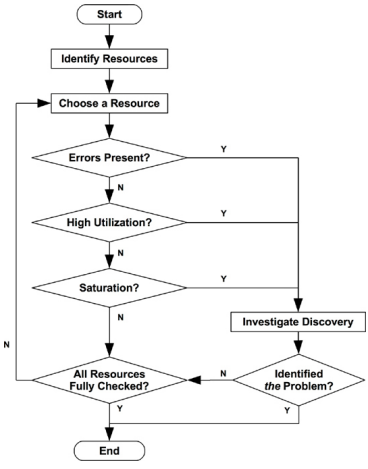
[Brendan Gregg; Prentice Hall; 2013]

- **Approach**: Check for **possible problems** through a **structured list of questions**:
  - What makes you think there is a performance problem?
  - Has this system ever performed well?
  - What has changed recently?
  - Can the performance degradation be expressed in terms of latency or run time?
- **Advantages**: Fast, can resolve some (more often occurring) issues
- **Disadvantages**: Limited scope

USE Methodology

- Investigate
  - **Utilization**: To what extent are relevant resources utilized?
  - **Saturation**: Do requests have to wait before being able to use a resource?
  - **Errors**: How many errors do occur?
- Resources may be both hardware and software resources

[Brendan Gregg; Prentice Hall; 2013]



Scientific Methodology



[Brendan Gregg; Prentice Hall; 2013]

1. **Question**: Performance problem statement
   - Example: Why is throughput of instructions low?
2. **Hypothesis**: Possible root cause for this problem
   - Example: There are chains of instructions that depend on each other (long critical paths)
3. **Prediction**: Measures that remedy this root cause
   - Example: Reorganization of the code resulting in shorter critical paths
4. **Test**: Perform runs after measures have been applied
5. **Analysis**: Assess the impact of the measures
   - Possibly need to go back to step #2

Model-based Methodology



1. **Measurement**: Measure performance and apply performance analysis techniques to identify performance critical regions
2. **Modelling**: Develop performance models to determine attainable performance
3. **Verification**: Verify whether model is sufficiently accurate
   - May have to go back to step #2
4. **Analysis**: Analyse deviations between observed and predicted performance
5. **Optimize**: Develop and implement strategy to address performance issues

Optimization versus Improvement



- **Optimization** = Process of modifying an application and/or the considered system with the goal of enlarging application performance **towards a defined optimum**

- **Improvement** = Process of modifying an application and/or the considered system with the goal of enlarging application performance **with respect to a reference performance**

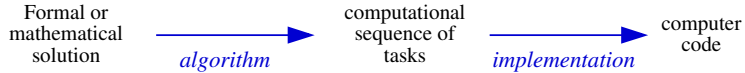
Content



1. Course Information
2. Computing Systems Organization
3. Performance Engineering
4. Performance and Performance Metrics
5. Final

Problem-Solving Abstraction

- Numerical problem-solving abstraction



- Execution of computer code = computation
- What is computation?
  - Process of manipulating data according to simple rules
  - Let data be represented by $w \in \{0,1\}^n$

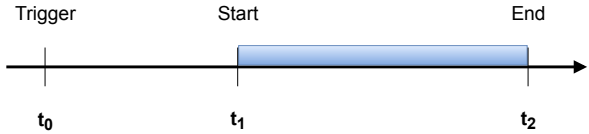
$$w^{(0)} \to w^{(1)} \to \cdots \to w^{(i)} \to \cdots$$

Events



- An event refers to a point in time where something happens
  - An event has no duration
- Examples:
  - Instruction is scheduled
  - Change of a signal value (e.g. indicating arrival of data)
  - Arrival of a message
  - Change of a state
  - A counter reaches/exceeds a given value

Latency



- **Latency** = Time from trigger of an operation until its completion
  - Here: $\Delta t = t_2 - t_0$
  - Example: Time from issue of a load instruction until arrival of the data at the destination
  - Also known as **turnaround time** or **time-to-solution**
- **Start-up latency** = Time from trigger of an operation until effects become visible
  - Here: $\Delta \tau = t_1 - t_0$
  - Example: Time from issue of a message receive command until arrival of first data
  - Also known as **response time**

Wall-Clock Time



- **Wall-clock time** = Time from a start event until an end event
  - Also known as **elapsed real time**
  - Examples: Start and completion of an application or task
  - Special case of latency

Resource Utilization Time



- **Resource utilization time** = Accumulated time between two events during which a resource is in use
- The reference clock is not incremented while the resource is not in use
- **Example**: Time during which a process or thread uses a CPU core

Performance, Bandwidth, and Throughput



- **Performance** = Amount of work W per time unit

$$b(W) = \frac{W}{\Delta t(W)}$$

  - While this definition does not specify work, it needs to be well-defined, e.g. execution of a specific **workload** (also called problem size)

- **Bandwidth** = Amount of data passing a **particular interface** per time unit
  - It is important to define this interface

- **Throughput** = Amount of work items processed per time unit
  - Example: Number of floating-point operations per time unit

$$B_{\mathrm{fp}} = \frac{N_{\mathrm{fp}}}{\Delta t}$$

Algorithmic Versus Machine Performance



- Performance of an application may be increased by
  - Improving the **algorithmic performance**
    - This is not considered in this lecture
  - Improving the **machine performance**
    - Focus of this lecture for given hardware architecture and system software stack

Content



1. Course Information
2. Computing Systems Organization
3. Performance Engineering
4. Performance and Performance Metrics
5. Final

Summary Lecture #01



- Computing systems
  - Complex systems comprising different hardware and software components
- Performance and performance metrics
  - Definition of various terms: Latency, start-up latency, performance, bandwidth, throughput
  - Wall-clock versus resource utilization time
- Performance engineering
  - Definition of performance engineering and different approaches to performance engineering

Questions?



[Wiki Media]



university of groningen

Performance Engineering: Lecture #02

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-02

Overview



1. Performance Measurement
2. Compiler Chain
3. Assembler
4. Final

Content



1. Performance Measurement

2. Compiler Chain

3. Assembler

4. Final

Performance Measurement Methods



- Methods
  - **Profiling**: Characterize the behaviour of an application in terms of aggregate performance metrics
    - Example: Average time spent in a subroutine
  - **Sampling**: Creation of profiles by means of periodic or random probing of performance observables independently of the current state of application execution
  - **Event-tracing**: Generate a list of event records comprising time stamp, location, type information, event-specific information
- Typically, additional instructions need to be inserted into the application (**instrumentation**)

Typical Time Scales



- Assume a processor with $f = 2.5\,\text{GHz}$ clock frequency
- Typical figures:

Core clock cycle	$0.4\,\text{ns}$
L1 cache access latency	$3 - 4\,\text{clock cycles}$
L2 cache access latency	About $10\,\text{clock cycles}$
L3 cache access latency	$\geq 35\,\text{clock cycles}$
Memory access latency	$O(200 - 300)\,\text{clock cycles}$

☞ Need time resolution $\ll 1\,\mu\text{s}$

Time Measurements



- Different time-related metrics
  - **Elapsed real time** or **wall-clock time**
    - Time from start to end as it could be observed using an external timer
  - **Resource utilization time**
    - Time during which a task occupies a resource, e.g. a CPU core
    - Timer is on hold when resource is used for other purposes
  - **Cycle counter**
    - The counter increment rate depends on the clock frequency
    - There may be different clock domains with variable clock
      - Exception: Reference clock

Hardware Performance Counters



- Modern processors provide hardware counters organised in a **Performance Monitoring Unit (PMU)**
- **Hardware performance counters** = Small set of registers that count events
  - The counter is incremented whenever a particular event occurs
  - The PMU is reconfigurable
- **Caveat**: Events are not always well-defined

Hardware Performance Counters: Examples

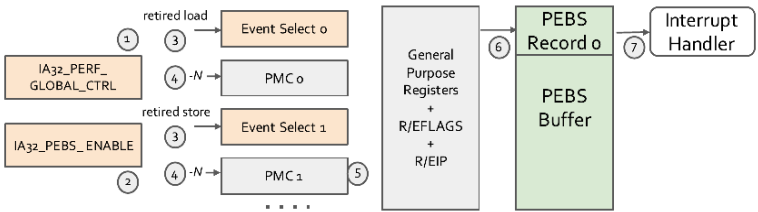


- Clock tick
- Instructions issued
- Integer instructions
- Load/store instructions
- ...
- Conditional branch instructions
- ...
- Level 1 data cache misses
- Level 1 instruction cache misses
- ...
- Translation lookaside buffer misses
- Hardware interrupts

Advanced Hardware Performance Counters

- Also known as: **precise event sampling**
- Examples:
  - Intel Processor Event Based Sampling (PEBS)
  - AMD Instruction-Based Sampling (IBS)
  - Arm Statistical Profiling Extension (SPE)

[Sasongko et al.; 2023]



Measurement Issues



- Issues related to performance measurement
  - Code instrumentation may impact performance
  - Accessibility of hardware counters
    - Performance counters by default require privileges
    - Support from the kernel may be required
  - Unclear or different definitions of hardware events
  - Management of large event traces

Deterministic versus Statistical Observables



- **Deterministic observables**:
  - Total number of committed instructions
  - Number of branch instructions
  - ...
- **Statistical observables**:
  - Execution time
  - Number of cache misses
  - Number of mispredicted conditional branch instructions
  - ...

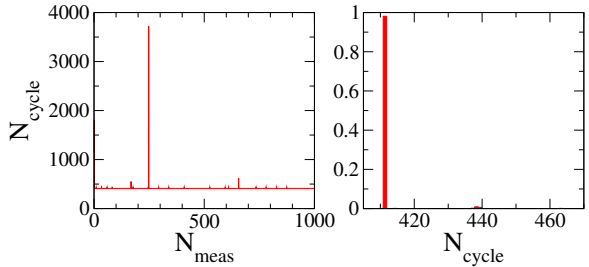
Variability



- Variability of the same system
  - Non-deterministic hardware (e.g. caches, memory management)
    - Example: Initially **caches are cold**
  - Non-uniform data paths (e.g. NUMA nodes)
  - Interference with other processes or (kernel-)threads (“OS jitter”)
  - Dynamic change of clock speed
- Variability between different but identical systems
  - Temperature variations
  - Manufacturing variability
  - “fail-slow” hardware

Variability: Real Example

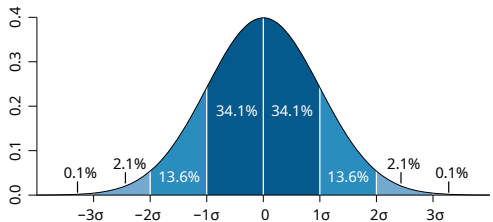
- Execute a sequence of M loads and stores of double precision floats N times
  (here $N = 10$, $M = 16$)
- Tested processor: TI OMAP4 with ARM Cortex-A9 cores



☞ Statistical distributions are likely **asymmetric** (e.g. no normal distributions)

Statistical Analysis

- Latency (or resource utilization times) are **statistical quantities**
  - Multiple measurements will result be different
- A statistical analysis can involve the following steps:
  - Estimation of a central value
  - Estimation of a distribution width
  - Estimation of the distribution (histogram)



Arithmetic Mean



- **Arithmetic mean** assuming N measurements $\Delta t^{(k)}$ have been performed:

$$\overline{\Delta t} = \frac{1}{N} \sum_{k=1}^{N} \Delta t^{(k)}$$

- **Challenge**
  - The average value is not a good estimator of a central value in case of asymmetric distributions
  - A single large of outlier can strongly affect the estimate (unless N is very large)
- **Note**: The distribution of latency measurements is typically not symmetric and large outliers may occur

Minimal Value

- **Minimal value** assuming N measurements $\Delta t^{(k)}$ have been performed:

$$\Delta t^{(\min)} = \min\left(\Delta t^{(k)}\right)$$

- The minimal value is robust with respect to outliers
- **Note**: Often there is a minimal latency but no maximum latency

Percentile (1/2)

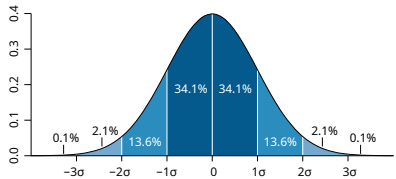
- **Percentile** = A number that denotes the position of a data point within a numeric dataset by indicating the percentage of the dataset with a lesser value

- Determination of a percentile Pk with $0 < k < 100$
  - Assume N measurements to be ordered such that

$$\Delta t^{(1)} \leq \Delta t^{(2)} \leq \cdots \leq \Delta t^{(N)}$$

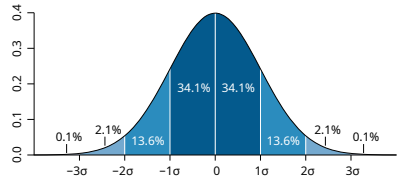
  - For sufficiently large N

$$Pk = \Delta t^{(\lceil k \cdot N/100 \rceil)}$$

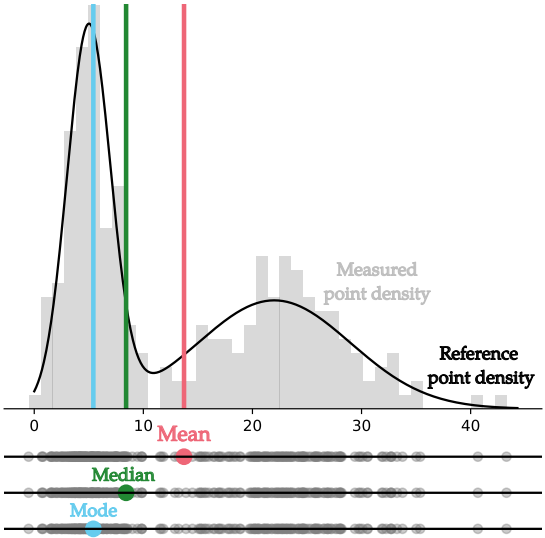


Percentile (2/2)

- Estimate of central value: $P50$
  - Also known as: **Median**, **second quartile**
- Variability estimate obtained by computing
  - First quartile $P25$
  - Third quartile $P75$
- Benefits of using the median:
  - Relatively easy to compute
  - Robust wrt outliers
  - Suitable for asymmetric probability distributions
  - Distribution width can be estimated using the P25 and P50 percentiles



Comparison



[Wikimedia; 2026]

Using Fits



- **Idea**: Perform measurements for different number of benchmark iterations
- **Assumption**:

$$\Delta t(N_{\text{iter}}) = N_{\text{iter}} \cdot \Delta t(1)$$

- **Approach**:
  - Measure $\Delta t(N_{\text{iter}})$ for different values of N_{iter}
  - Estimate benchmark execution time $\Delta \tilde{t}$ from fit to

$$\Delta t(N_{\text{iter}}) = c + N_{\text{iter}} \cdot \Delta \tilde{t}$$

Content



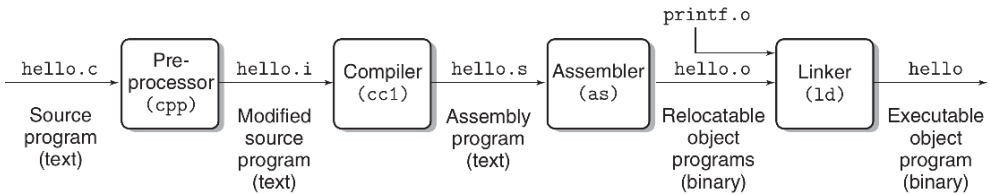
1. Performance Measurement

2. Compiler Chain

3. Assembler

4. Final

Compiler Chain: Overview



- Figure shows C compiler chain
- Other (imperative programming) languages similar

(C) Compiler Chain: Details



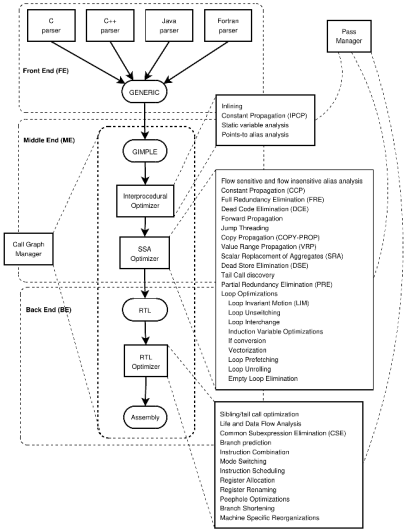
- **Pre-processing phase**: Source-to-source transformation
  - Can be exploited to turn run-time to compile-time computations
- **Compilation phase**: Translate high-level program to sequence of assembly instructions
  - Each assembly language statement corresponds to a machine instruction
  - Transformation from (largely) machine-agnostic to machine-specific code
- **Assembly phase**: Create relocatable object programs
  - Translates assembly into machine instructions
  - Typically, no further optimizations at this level
- **Linker phase**: Combine multiple relocatable object programs to an executable program

Main Compiler Tasks



- Code generation
- Instruction memory management (code blocks, address binding of instructions)
- Data memory management (stack variables)

GCC Organization (1/2)



[Diego Novillo; 2006]

GCC Organization (2/2)



- **Front-end**: Interpret different high-level languages and map to a generic representation
  - **Intermediate Representation** (IR) called GENERIC
- **Middle-end**:
  - Transformation to the GIMPLE IR
  - **Interprocedural optimizer**
  - **Static Single Assignment (SSA) optimizer**
- **Back-end**:
  - Transformation to the Register Transfer Language (RTL) IR
  - **RTL optimizer**
  - Assembly code generator

Compiler Optimization Steps



- **Legality-check**: Identify opportunities for code transformations and check correctness
  - A performance engineer can help by
    - Ensuring all necessary information being available
    - Making it easier for compilers to identify opportunities
- **Profitability-check**: Assess the benefits of the code transformation
  - Step based on heuristics that may be wrong
  - Sometimes a performance engineer can help by providing additional information
- **Transformation**: Implement the code transformation correctly

Selected Compiler Optimization Techniques (1/2)



- **Interprocedural optimizer**
  - **Function Inlining**
  - **Constant Propagation**
  - ...
- **Static Single Assignment (SSA) optimizer**
  - **Dead Code Elimination**
  - Loop Optimizations
    - Loop Interchange
    - **Loop Unrolling**
    - **Vectorization**
  - ...

Selected Compiler Optimization Techniques (2/2)



- **RTL optimizer**
  - **Common Subexpression Elimination**
  - Branch prediction
  - Instruction Combination
  - Register Allocation
  - Register Renaming
  - Peephole Optimizations
  - **Machine-Specific Reorganizations**
  - ...

Why is this Knowledge Relevant?



- Have an understanding of what compilers can and cannot do
- Understand what a compiler could have done
- Be able to check whether compilers decided to apply certain optimization techniques

GCC Optimization-Related Command-Line Options



- `O<#>`
  - `O0`: Disable optimizations
  - `O1 ... O3`: Increase optimization levels
- Different optimizations can also be enabled individually
- Some optimizations can be influenced using parameters
  - Usually not needed
- For others see, e.g.,
  `https://gcc.gnu.org/onlinedocs/gcc-14.2.0/gcc/Optimize-Options.html`
- Note: Optimization can also be controlled using compiler-specific pragma statements

Optimization Caveats

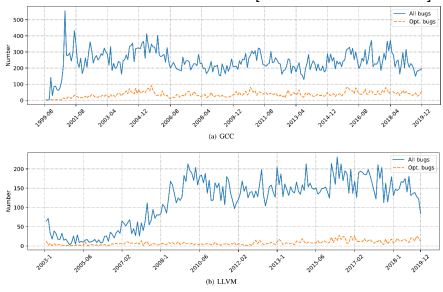


- Increased risk of bugs
- Change of rounding errors
- Code transformations could have a negative performance impact

Optimization Bugs

- Empirical research indicates that the optimizers are the most buggy part of GCC and LLVM
- This assessment is based on analysis of bug tickets
- Optimizations with the largest number of GCC bug tickets:
  - Value range propagation
  - Value numbering
  - Instruction combination
  - Partial redundancy elimination
- Note: The analysis does not take compiler release stage into account

[Zhide Zhou et al.; 2021]



Other Compilers



- LLVM
  - Most popular open-source compiler next to GCC
  - While GCC is GPL licensed, LLVM uses the Apache Licence
  - Recommended to test as an alternative to GCC
- Various vendor solutions
  - Often placed under a commercial licence (possible “community editions”)
  - Many are meanwhile based on LLVM
  - Goal: Provide machine-specific optimizations early
  - Examples:
    - Intel oneAPI compilers
    - AMD Optimizing C/C++ and Fortran Compilers (AOCC)
    - Arm Development Studio, Arm Compiler for Linux

Content



1. Performance Measurement

2. Compiler Chain

3. Assembler

4. Final

Assembler Generation



- Example GNU C-compiler: `gcc -S -O0 myprog.c`
  - Command will generate output file `myprog.s`
  - Usually better to start with zero-level optimization (`-O0`)
- Use cases:
  - Inspection of the instructions generated by the compiler
  - Starting point for own assembler implementations

Assembler Generation Example: x86



C program:

```c
void stupid_loop() {
  int i;
  for (i=0; i<100; i++);
}
```

Compilation:

```
gcc-14 -S -O0 myprog.c
```

Assembler:

```
        .file   "myprog.c"
        .text
        .globl  stupid_loop
        .type   stupid_loop, @function
stupid_loop:
.LFB0:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        movl    $0, -4(%rbp)
        jmp     .L2
.L3:
        addl    $1, -4(%rbp)
.L2:
        cmpl    $99, -4(%rbp)
        jle     .L3
        nop
        nop
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
```

x86 ISA



- ISA based on Intel 8086 CPU (launched 1978)
- Architecture dominates PC mass market and is used in most supercomputers
  - Top500: $93.4\,\%$ of the top500 systems use x86 CPUs [Top500, 06/2025]
- Manufacturers: Intel, AMD
- Main features
  - CISC-type ISA
  - One operand can be a memory location
- Extensions (selection):
  - x86-64
  - SSE2, SSE3, SSE4.x
  - AVX, AVX2, AVX-512

Arm ISA



- ARM = Advanced RISC Machine
  - Current main market: mobile and embedded electronics
  - Emerging server processor market
- ISA features
  - RISC-type ISA
  - Load/store architecture
- Extensions (selection):
  - SVE, SVE2
  - SME

Assembler Generation Example: Armv8



C program:

```
1 void stupid_loop() {
2   int i;
3   for (i=0; i<100; i++);
4 }
```

Compilation:

```
gcc-14 -S -O0 myprog.c
```

Assembler:

```
1           .arch armv8-a
2           .file   "myprog.c"
3           .text
4           .align  2
5           .global stupid_loop
6           .type   stupid_loop, %function
7   stupid_loop:
8   .LFB0:
9           .cfi_startproc
10          sub     sp, sp, #16
11          .cfi_def_cfa_offset 16
12          str     wzr, [sp, 12]
13          b       .L2
14  .L3:
15          ldr     w0, [sp, 12]
16          add     w0, w0, 1
17          str     w0, [sp, 12]
18  .L2:
19          ldr     w0, [sp, 12]
20          cmp     w0, 99
21          ble     .L3
22          nop
23          nop
24          add     sp, sp, 16
25          .cfi_def_cfa_offset 0
26          ret
27          .cfi_endproc
```

Hints



- Start by compiling with -O0 to avoid difficult to understand Optimizations
- Use inline assembler statements to insert markers

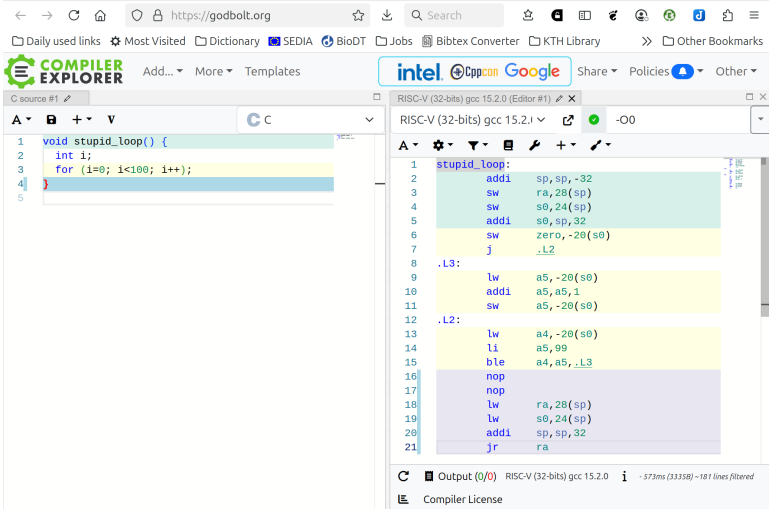
C program:

```
1 double f(double x, int n) {
2     asm("mymark1:");
3     for (int i = 0; i < n; i++)
4         x *= x;
5     asm("mymark2:");
6     return x;
7 }
```

Assembler:

```
16 #APP
17 # 2 "ex.c" 1
18         mymark1:
19 # 0 "" 2
20 #NO_APP
21         movl    $0, -4(%rbp)
22         jmp     .L2
23 .L3:
24         movsd   -24(%rbp), %xmm0
25         mulsd   %xmm0, %xmm0
26         movsd   %xmm0, -24(%rbp)
27         addl    $1, -4(%rbp)
28 .L2:
29         movl    -4(%rbp), %eax
30         cmpl    -28(%rbp), %eax
31         jl      .L3
32 #APP
33 # 5 "ex.c" 1
34         mymark2:
35 # 0 "" 2
36 #NO_APP
```

Godbolt


https://godbolt.org
Search
Daily used links
Most Visited
Dictionary
SEDIA
BioDT
Jobs
Bibtex Converter
KTH Library
Other Bookmarks
COMPILER EXPLORER
Add...
More
Templates
intel
Cppcon
Google
Share
Policies
Other
C source #1
C
RISC-V (32-bits) gcc 15.2.0 (Editor #1)
RISC-V (32-bits) gcc 15.2.0
-O0
void stupid_loop() {
int i;
for (i=0; i<100; i++);
}
stupid_loop:
addi sp,sp,-32
sw ra,28(sp)
sw s0,24(sp)
addi s0,sp,32
sw zero,-20(s0)
j .L2
.L3:
lw a5,-20(s0)
addi a5,a5,1
sw a5,-20(s0)
.L2:
lw a4,-20(s0)
li a5,99
ble a4,a5,.L3
nop
nop
lw ra,28(sp)
lw s0,24(sp)
addi sp,sp,32
jr ra
Output (0/0)
RISC-V (32-bits) gcc 15.2.0
- 573ms (33358) ~181 lines filtered
Compiler License

Content



1. Performance Measurement

2. Compiler Chain

3. Assembler

4. Final

Summary Lecture #02



- Performance measurement
  - Different performance measurement methods
  - Means for measuring performance and performance relevant observables
- Optimizations leveraging the compiler
- Analysis of assembler to understand code optimization or guide further code optimizations

Questions?



[Wiki Media]

These slides contain content and figures from

- R. E. Bryant and D. R. O'Hallaron, "Computer Systems: A Programmer's Perspective," 2nd edition, Pearson, 2011



university of groningen

Performance Engineering: Lecture #03

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-07

Overview



1. Case Study: `inc2`
2. Case Study: `vreduce`
3. Other Compiler Optimizations
4. Compiler Reports
5. Final

Content



1. **Case Study:** `inc2`
2. Case Study: `vreduce`
3. Other Compiler Optimizations
4. Compiler Reports
5. Final

Case Study: `inc2`



- Compilers must be careful to apply only safe optimizations to a code
- Purpose of this case study:
  - Study consequence of compiler decisions based on lacking knowledge
  - Understand the **memory aliasing** problem
  - Explore a remedy
- Specification for `inc2`:
  - Function does not return anything, input and output are provided as C pointers
  - The function adds twice the input integer to the output integer

inc2: Implementations (1/3)



- Two implementations:

```
1  void inc2a(int *xp, int *yp)
2  {
3      *xp += *yp;
4      *xp += *yp;
5  }
6
7  void inc2b(int *xp, int *yp)
8  {
9      *xp += 2 * (*yp);
10 }
```

- **Question**: Will both codes return the same answer?

inc2: Implementations (2/3)



- Two implementations:

```
1  void inc2a(int *xp, int *yp)
2  {
3      *xp += *yp;
4      *xp += *yp;
5  }
6
7  void inc2b(int *xp, int *yp)
8  {
9      *xp += 2 * (*yp);
10 }
```

- **Answer**: NO
- Value stored in `*xp` depends on whether the pointers are the same or different

	different	same
`inc2a`	$x \leftarrow x + 2y$	$x \leftarrow 4x$
`inc2b`	$x \leftarrow x + 2y$	$x \leftarrow 3x$

inc2: Implementations (3/3)



Code for inc2a():

```
1  inc2a:
2  .LFB0:
3          .cfi_startproc
4          endbr64
5          movl    (%rsi), %eax
6          addl    (%rdi), %eax
7          movl    %eax, (%rdi)
8          addl    (%rsi), %eax
9          movl    %eax, (%rdi)
10         ret
11         .cfi_endproc
```

Code for inc2b():

```
1  inc2b:
2  .LFB1:
3          .cfi_startproc
4          endbr64
5          movl    (%rsi), %eax
6          addl    %eax, %eax
7          addl    %eax, (%rdi)
8          ret
9          .cfi_endproc
```

Memory Aliasing



- **Situation**: The same memory location can be accessed through more than one symbolic name (like a variable or a pointer) in a program
- **Problem**: Various optimizations are impossible if memory aliasing cannot be excluded
  - See the `inc2` example
- **Solution**: Avoid memory aliasing by providing sufficient information to the compiler
  - Use the `restrict` keyword in C

inc2: Optimization



- The memory aliasing issue can be solved by using `restrict`-qualified pointers
- By using this qualifier, one promises to the compiler that the object, to which the pointer points, will only be accessed through this pointer

Modified C-Code:

```
1 void inc2c(int *restrict xp,
2             int *restrict yp)
3 {
4     *xp += *yp;
5     *xp += *yp;
6 }
```

Assembler:

```
1 inc2o:
2 .LFB0:
3         .cfi_startproc
4         endbr64
5         movl    (%rsi), %eax
6         addl    %eax, %eax
7         addl    %eax, (%rdi)
8         ret
9         .cfi_endproc
```

Content



1. Case Study: `inc2`
2. **Case Study: `vreduce`**
3. Other Compiler Optimizations
4. Compiler Reports
5. Final

Case Study: `vreduce`



- **Purpose** of this case study: Explore different loop-related optimization strategies
- **Specification** for `vreduce`:
  - Use a data type for arrays of numbers with a length fixed at run-time
  - Provide a function that computes the sum of all array elements using a pointer to an array on input and a pointer referencing the output
  - Different simple data types should be supported

vreduce: Vector Data Type (1/3)



- The following information is stored in a file `myvector.h`
- Data structure:

```c
#include <stdlib.h>

#define DTYPE double

typedef struct {
    long int  len;
    DTYPE     *data;
} vec_rec, *vec_ptr;
```

`vreduce`: Vector Data Type (2/3)



- Data structure **creator**:

```
11 vec_ptr new_vec(long int len)
12 {
13     /* Allocate header structure */
14     vec_ptr result = (vec_ptr) malloc(sizeof(vec_rec));
15     if (!result)
16         return NULL;                  /* Allocation failed */
17     result->len = len;
18
19     /* Allocate array */
20     if (len > 0) {
21         DTYPE *data = (DTYPE *) calloc(len, sizeof(DTYPE));
22         if (!data) {
23             free((void *) result);
24             return NULL;              /* Allocation failed */
25         }
26         result->data = data;
27     }
28     else
29         result->data = NULL;
30
31     return result;
32 }
```

`vreduce`: Vector Data Type (3/3)



- Array **element accessor**:

```
35 int get_vec_element(vec_ptr v, long int index, DTYPE *dest)
36 {
37     if (index < 0 || index >= v->len)
38         return 0;
39     *dest = v->data[index];
40     return 1;
41 }
```

- Array **length accessor**:

```
44 long int get_vec_length(vec_ptr v)
45 {
46     return v->len;
47 }
```

- Array **data accessor**:

```
50 DTYPE* get_vec_data(vec_ptr v)
51 {
52     return v->data;
53 }
```

`vreduce`: 1st Implementation (1/2)



- Our first implementation:

```
1 #include "myvector.h"
2
3 void vreduce1(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6
7      *dest = 0;
8      for (i = 0; i < get_vec_length(v); i++) {
9          DTYPE val;
10         get_vec_element(v, i, &val);
11         *dest += val;
12     }
13 }
```

- **Performance challenge**: **function calls**

`vreduce`: 1st Implementation (2/2)



- Compilation using -O0:

```
1  #include "myvector.h"
2
3  void vreduce1(vec_ptr v, DTYPE *dest)
4   {
5       long int i;
6
7       *dest = 0;
8       for (i = 0; i < get_vec_length(v); i++) {
9           DTYPE val;
10          get_vec_element(v, i, &val);
11          *dest += val;
12      }
13  }
```

- **Observation(s)**:
  - Two function calls
  - Number of instructions: 19

```
159         jmp     .L16
160 .L17:
161         leaq    -24(%rbp), %rdx
162         movq    -16(%rbp), %rcx
163         movq    -40(%rbp), %rax
164         movq    %rcx, %rsi
165         movq    %rax, %rdi
166         call    get_vec_element
167         movq    -48(%rbp), %rax
168         movsd   (%rax), %xmm1
169         movsd   -24(%rbp), %xmm0
170         addsd   %xmm1, %xmm0
171         movq    -48(%rbp), %rax
172         movsd   %xmm0, (%rax)
173         addq    $1, -16(%rbp)
174 .L16:
175         movq    -40(%rbp), %rax
176         movq    %rax, %rdi
177         call    get_vec_length
178         cmpq    %rax, -16(%rbp)
179         jl      .L17
```

`vreduce`: 2nd Implementation (1/2)



- **First remedy**: Store array length in local variable:

```
1  #include "myvector.h"
2
3  void vreduce2(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6      long int n = get_vec_length(v);
7
8      *dest = 0;
9      for (i = 0; i < n; i++) {
10         DTYPE val;
11         get_vec_element(v, i, &val);
12         *dest += val;
13     }
14 }
```

`vreduce`: 2nd Implementation (2/2)



- Compilation using `-O0`:
- **Observation(s)**:
  - Only one function call
  - Number of instructions: 17

```
163         jmp     .L16
164 .L17:
165         leaq    -32(%rbp), %rdx
166         movq    -24(%rbp), %rcx
167         movq    -40(%rbp), %rax
168         movq    %rcx, %rsi
169         movq    %rax, %rdi
170         call    get_vec_element
171         movq    -48(%rbp), %rax
172         movsd   (%rax), %xmm1
173         movsd   -32(%rbp), %xmm0
174         addsd   %xmm1, %xmm0
175         movq    -48(%rbp), %rax
176         movsd   %xmm0, (%rax)
177         addq    $1, -24(%rbp)
178 .L16:
179         movq    -24(%rbp), %rax
180         cmpq    -16(%rbp), %rax
181         jl      .L17
```

`vreduce`: 3rd Implementation (1/2)



- **Second remedy**: Access data directly:

```
1  #include "myvector.h"
2
3  void vreduce3(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6      long int length = get_vec_length(v);
7      DTYPE    *data  = get_vec_data(v);
8
9      *dest = 0;
10     for (i = 0; i < length; i++) {
11         *dest += data[i];
12     }
13 }
```

`vreduce`: 3rd Implementation (2/2)



- Compilation using `-O0`:
- **Observation(s)**:
  - No function calls left
  - Number of instructions: 15

```
164             jmp     .L16
165     .L17:
166             movq    -48(%rbp), %rax
167             movsd   (%rax), %xmm1
168             movq    -24(%rbp), %rax
169             leaq    0(,%rax,8), %rdx
170             movq    -8(%rbp), %rax
171             addq    %rdx, %rax
172             movsd   (%rax), %xmm0
173             addsd   %xmm1, %xmm0
174             movq    -48(%rbp), %rax
175             movsd   %xmm0, (%rax)
176             addq    $1, -24(%rbp)
177     .L16:
178             movq    -24(%rbp), %rax
179             cmpq    -16(%rbp), %rax
180             jl      .L17
```

`vreduce`: 4th Implementation (1/2)



- **Next optimization**: Use local variable for accumulation:

```
1  #include "myvector.h"
2
3  void vreduce4(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6      long int length = get_vec_length(v);
7      DTYPE    *data  = get_vec_data(v);
8      DTYPE    accum  = 0;
9
10     for (i = 0; i < length; i++) {
11         accum += data[i];
12     }
13     *dest = accum;
14 }
```

`vreduce`: 4th Implementation (2/2)



- Compilation using `-O0`:
- **Observation(s)**:
  - No function calls left
  - Number of instructions: 13

```
163         jmp     .L16
164 .L17:
165         movq    -32(%rbp), %rax
166         leaq    0(,%rax,8), %rdx
167         movq    -8(%rbp), %rax
168         addq    %rdx, %rax
169         movsd   (%rax), %xmm0
170         movsd   -24(%rbp), %xmm1
171         addsd   %xmm1, %xmm0
172         movsd   %xmm0, -24(%rbp)
173         addq    $1, -32(%rbp)
174 .L16:
175         movq    -32(%rbp), %rax
176         cmpq    -16(%rbp), %rax
177         jl      .L17
```

vreduce: Back to 1st Implementation



- Compilation using -O3:

```
1  #include "myvector.h"
2
3  void vreduce1(vec_ptr v, DTYPE *dest)
4   {
5       long int i;
6
7       *dest = 0;
8       for (i = 0; i < get_vec_length(v); i++) {
9           DTYPE val;
10          get_vec_element(v, i, &val);
11          *dest += val;
12      }
13  }
```

- **Observation(s)**:
  - No function calls left
  - Further reduction of the number of instructions

```
115 vreduce1:
116 .LFB20:
117         .cfi_startproc
118         endbr64
119         movq    (%rdi), %rdx
120         movq    $0x000000000, (%rsi)
121         testq   %rdx, %rdx
122         jle     .L20
123         movq    8(%rdi), %rax
124         pxor    %xmm0, %xmm0
125         leaq    (%rax,%rdx,8), %rdx
126         .p2align 4,,10
127         .p2align 3
128 .L22:
129         addsd   (%rax), %xmm0
130         addq    $8, %rax
131         movsd   %xmm0, (%rsi)
132         cmpq    %rax, %rdx
133         jne     .L22
134 .L20:
135         ret
136         .cfi_endproc
```

Function Inlining



- **Transformation**: A function call is replaced by the body of the function itself
- **Prerequisite**: Function body code must be available
- **Hints**:
  - The `inline` qualifier is typically not required
  - The `inline` qualifier does not enforce inlining
  - Inlining can be enforced using compiler-specific function declaration attributes:
    - GCC: `__attribute__((always_inline))`

Content



1. Case Study: `inc2`
2. Case Study: `vreduce`
3. **Other Compiler Optimizations**
4. Compiler Reports
5. Final

Constant Folding and Constant Propagation



- **Constant folding**:
  - **Transformation**: Replace an arithmetic expression at compile time, e.g. `i = 5 * 12` → `i = 60`
  - Caveat: Evaluation of arithmetic expressions on systems used for compilation and execution may differ
- **Constant propagation**:
  - **Transformation**: Substitute the values of known constants in expressions at compile time

```
1 int f() {
2     int i = 3 * 15;
3     int j = 3 * 7 - 42 / 2;
4     return (i * j);
5 }
```

→

```
1 int f() {
2     return 0;
3 }
```

Common Subexpression Elimination



- **Transformation**:
  - Identify instances of identical expressions
  - Evaluate expression only once and store result in a temporary variable
  - Reuse this temporary variable
- Example:

```c
int f(int a, int b, int c) {
    int x, y;

    x = a * b + c;
    y = a * b + 2 * c;

    /* more code */

    return (x + y);
}
```

→

```c
int f(int a, int b, int c) {
    int x, y, t;

    t = a * b;
    x = t + c;
    y = t + 2 * c;

    /* more code */

    return (x + y);
}
```

Dead Code Elimination



- **Transformation**: Remove code using dead variables
- **Dead variables**: A variable is said to be dead if it is never used after its last definition
- **Example** compiled using `gcc-14 -O3`:

```
1  void f() {
2      int i;
3      for (i = 0; i < 128; i++);
4  }
5
6  int main() {
7      f();
8      return 0;
9  }
```

```
6  f:
7  .LFB0:
8          .cfi_startproc
9          endbr64
10         ret
11         .cfi_endproc
12 .LFE0:
13         .size   f, .-f
14         .section        .text.startup,"ax",@progbits
15         .p2align 4
16         .globl  main
17         .type   main, @function
18 main:
19 .LFB1:
20         .cfi_startproc
21         endbr64
22         xorl    %eax, %eax
23         ret
24         .cfi_endproc
```

Optimizations Impacting Floating-Point Operations



- Compilers support optimizations on instructions for floating-point operations that affect results
  - Results may not any more comply to IEEE standards
  - Typically, this concerns special cases like subnormal numbers or floating-point values `NaN`, `Inf`
- Example GCC: `-ffast-math`
  - Includes `-fno-math-errno` that suppresses setting of `errno`, which may allow avoiding function calls

☞ **Trade precision for speed**

Machine Specific Optimizations



- Compilers may use apply different optimizations or may apply optimizations differently when the target architecture is known
  - Example: Use of specific instructions
- Relevant GCC options:
  - `-mtune=<cpu-type>`: Tune code for given CPU architecture while keeping it portable
  - `-march=<cpu-type>`: Allow compiler to generate code that may not run on processors apart from the one indicated
    - Code generated using `-march=skylake-avx512` might only run on Intel Skylake processors
    - Code generated using `-march=armv8-a+sve` might only run on Arm-based processors supporting both Arm-v8 and SVE
  - Set `cpu-type` to `native` to let the compiler autodetect the current platform

Content



1. Case Study: `inc2`
2. Case Study: `vreduce`
3. Other Compiler Optimizations
4. **Compiler Reports**
5. Final

Compiler Reports: Introduction



- Compilers can generate optimization reports
- **Caveat**: Some of the reports are more suitable for compiler developers
- Inspecting compiler optimization reports is often easier than analysis of the generated assembler

GCC Optimization Reports



- Relevant GCC options:
  - `-fopt-info`: Write default optimization information
  - `-fopt-info-<selector>`: Write selected information
  - `-fopt-info-<selector>=<file>`: Write selected information to given file
    - Selector = `missed`, `loop`, `inline`, `vec`, `missed`, `optall`, ...

GCC Optimization Reports: Example (1/2)



Example code:

```
11 vec_ptr new_vec(long int len)
12 {
13     /* Allocate header structure */
14     vec_ptr result = (vec_ptr) malloc(sizeof(vec_rec));
15     if (!result)
16         return NULL;
17     result->len = len;
18
19     /* Allocate array */
20     if (len > 0) {
21         DTYPE *data = (DTYPE*) calloc(len,sizeof(DTYPE));
22         if (!data) {
23             free((void *) result);
24             return NULL;
25         }
26         result->data = data;
27     }
28     else
29         result->data = NULL;
30
31     return result;
32 }
```

```
1  #include "myvector.h"
2
3  void vreduce1(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6
7      *dest = 0;
8      for (i = 0; i < get_vec_length(v); i++) {
9          DTYPE val;
10         get_vec_element(v, i, &val);
11         *dest += val;
12     }
13 }
```

GCC Optimization Reports: Example (2/2)



Optimization report: `gcc-14 -c -O1 -fopt-info-missed vreduce1.c`

```
1 vreduce1.c:8:22: missed:   not inlinable: vreduce1/20 -> get_vec_length/18, function not inline candidate
2 vreduce1.c:10:10: missed:   not inlinable: vreduce1/20 -> get_vec_element/17, function not inline candidate
3 myvector.h:23:13: missed:   not inlinable: new_vec/16 -> free/23, function body not available
4 myvector.h:21:33: missed:   not inlinable: new_vec/16 -> calloc/22, function body not available
5 myvector.h:14:32: missed:   not inlinable: new_vec/16 -> malloc/21, function body not available
```

LLVM clang Optimization Reports



- Relevant clang options:
  - `-Rpass=<arg>`: Report transformations performed
  - `-Rpass-missed=<arg>`: Report missed transformations
  - `-Rpass-analysis=<arg>`: Report transformation analysis
    - arg = POSIX regular expression

clang Optimization Reports: Example (1/2)



Example code:

```
11 vec_ptr new_vec(long int len)
12 {
13     /* Allocate header structure */
14     vec_ptr result = (vec_ptr) malloc(sizeof(vec_rec));
15     if (!result)
16         return NULL;
17     result->len = len;
18
19     /* Allocate array */
20     if (len > 0) {
21         DTYPE *data = (DTYPE*) calloc(len,sizeof(DTYPE));
22         if (!data) {
23             free((void *) result);
24             return NULL;
25         }
26         result->data = data;
27     }
28     else
29         result->data = NULL;
30
31     return result;
32 }
```

```
1  #include "myvector.h"
2
3  void vreduce1(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6
7      *dest = 0;
8      for (i = 0; i < get_vec_length(v); i++) {
9          DTYPE val;
10         get_vec_element(v, i, &val);
11         *dest += val;
12     }
13 }
```

clang Optimization Reports: Example (2/2)

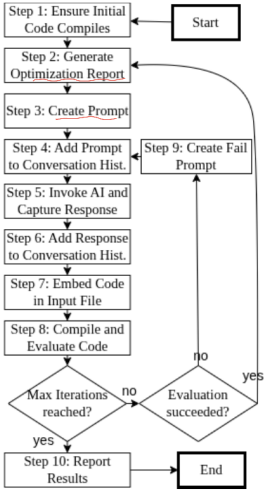


Optimization report: `clang-19 -c -O1 -Rpass-missed=.* vreduce1.c`

```
vreduce1.c:11:16: remark: failed to move load with loop-invariant address because the loop may invalidate its value [-Rpass-missed=licm]
   11 |           *dest += val;
      |                ^
vreduce1.c:11:16: remark: failed to hoist load with loop-invariant address because load is conditionally executed [-Rpass-missed=licm]
vreduce1.c:11:16: remark: failed to move load with loop-invariant address because the loop may invalidate its value [-Rpass-missed=licm]
vreduce1.c:8:6: remark: loop not vectorized [-Rpass-missed=loop-vectorize]
    8 |       for (i = 0; i < get_vec_length(v); i++) {
      |       ^
In file included from vreduce1.c:1:
./myvector.h:11:1: remark: 7 virtual registers copies 2.708984e+00 total copies cost generated in function [-Rpass-missed=regalloc]
   11 | vec_ptr new_vec(long int len)
      | ^
```

Compiler Reports Processing Using LLMs

- Recent research result: **CompilerGPT** [Pirkelbauer, Liao; 2025]
- **Idea**: Use compiler oztimisation reports as input for **optimized code generation by an LLM**



Content



1. Case Study: `inc2`
2. Case Study: `vreduce`
3. Other Compiler Optimizations
4. Compiler Reports
5. **Final**

Summary Lecture #03



- Manual optimization techniques
  - Avoid memory aliasing problem
  - Eliminate function calls in loops
- Compiler optimization techniques
  - Constant folding and propagation
  - Common subexpression elimination
  - Dead code elimination
  - Floating-point operation optimizations
  - Machine-specific optimizations
- Compiler optimization reports

Questions?



[Wiki Media]



university of groningen

Performance Engineering: Lecture #04

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-10

Overview



1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. BSP Model
6. Final

Content



1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. BSP Model
6. Final

Benchmarks: Introduction



- **Benchmark** = Program which is executed to measure the performance of a system or a system component
- **Benchmarks categorization**
  - **Work-load type**
    - Real application work-load
    - Micro-benchmarks: Programs which stimulate a particular feature of the system
  - **Code type**
    - Kernels: Relevant pieces of real application programs
    - Synthetic benchmarks: Fake programs which try to match the profile and behaviour of real programs
- **Motivations** for using benchmarks
  - Comparison of different architectures
  - **Create knowledge for performance engineering**
    - What are the hardware characteristics of the target system?
    - What is the highest attainable performance?

Arithmetic Operations Throughput



- **High-Performance Linpack Benchmark (HPL)**
  - Solve a dense $N \times N$ system of linear equations

$$A \cdot x = b$$

  - Basis for Top500 ranking of supercomputers [Top500]
  - Top500 **benchmarking rules**
    - Problem size is not fixed
    - Algorithm is prescribed (LU factorisation with partial pivoting, double precision)
    - Code is not fixed, reference implementation provided: `https://netlib.org/benchmark/hpl/`
- Several benchmarks that aim at maximising filling of the floating-point pipelines
  - Mainly suitable for exploring hardware characteristics
  - Example: uarch-bench, UBENCH-x86

Memory Performance



- **STREAM**
  - De facto standard benchmark for the measurement of **CPU memory bandwidth**
  - Includes several benchmarks: `copy`, `scale`, `add`, `triad`
  - `https://github.com/jeffhammond/STREAM`
- **BabelStream** [Deakin et al.; 2018]
  - Support of various programming models (including GPU programming models)
  - Augmented STREAM by a `dot` benchmark
- **tinymembench**
  - Benchmark for memory bandwidth and **latency**

Network Performance



- **Intel MPI Benchmarks**
  - Benchmarks various communication patterns implemented in MPI
  - `https://github.com/intel/mpi-benchmarks.git`
- **NetGauge**
  - Network parameter measurement tool that supports benchmarking of different network protocols and communication patterns
  - `https://htor.inf.ethz.ch/research/netgauge/`
- **perftest**
  - Collection of tests for micro-benchmarking network technologies supporting RDMA
  - `https://github.com/linux-rdma/perftest`

Application-Based Benchmarks



- **NAS Parallel Benchmark Suite (NPB)**
  - Set of programs designed to help evaluate the performance of parallel computers
  - Implemented kernels and pseudo-applications:
    - IS: Integer Sort, random memory access
    - EP: Embarrassingly Parallel
    - CG: Conjugate Gradient, irregular memory access and communication
    - MG: Multi-Grid on a sequence of meshes, long- and short-distance communication, memory intensive
    - FT: discrete 3D fast Fourier Transform, all-to-all communication
    - BT: Block Tri-diagonal solver
    - SP: Scalar Penta-diagonal solver
    - LU: Lower-Upper Gauss-Seidel solver
  - Source:
    - Original version (Fortran): `https://www.nas.nasa.gov/software/npb.html`
    - Reimplementation in C: `https://github.com/benchmark-subsetting/NPB3.0-omp-C`

Content



1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. BSP Model
6. Final

Introduction to Computational Dwarfs



- **Defining dwarfs** = An approach to capture patterns of computation and communication common to a class of important applications
  - Approach introduced 2006 [Asanovic et al.; 2006]
  - Various dwarfs already identified in earlier work
- **Goal**: Identify commonalities between numerical applications to exploit common knowledge and experience about
  - Performance characteristics (e.g. arithmetic intensity)
  - Best-practices for implementation on a given architecture

Computational Dwarfs: Overview



Dwarfs for HPC

1. Dense linear algebra
2. Sparse linear algebra
3. Spectral methods
4. N-body methods
5. Structured grids
6. Unstructured grids
7. MapReduce (Monte Carlo)

Other Dwarfs

8. Combinational logic
9. Graph traversal
10. Dynamic programming
11. Back-track and branch + bound
12. Graphical models
13. Finite state machines

Computational Dwarf: Dense Linear Algebra



- Key data objects are **densely populated matrices or vectors**
- Typical **memory access pattern**:
  Unit-stride or fixed-stride regular sequential access
- Example: $y_i \leftarrow \sum_j A_{ij} \cdot x_j$

$$\begin{pmatrix} 1 & 4 & 5 & 6 \\ 0 & 3 & 2 & 6 \\ 1 & 2 & 3 & 1 \\ 5 & 4 & 3 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 48 \\ 36 \\ 18 \\ 30 \end{pmatrix}$$

- Popular library with dense linear algebra kernels: **BLAS (Basic Linear Algebra Subprograms)**
  - Different implementations available:
    - Open source: NETLIB BLAS, OpenBLAS, BLIS
    - Closed source: MKL (Intel), ESSL (IBM), Arm Performance Library (Arm)
  - Classification of operations
    - BLAS-1: scalar, vector and vector-vector
    - BLAS-2: matrix-vector
    - BLAS-3: matrix-matrix

Computational Dwarf: Sparse Linear Algebra



- Key data objects are **matrices or vectors with many zeros**
- To avoid storing matrices with many zeros, **compressed formats** are used
  - Popular format: **Compressed Sparse Row (CSR)**
    - Number of non-zero entries in matrix is denoted as N_{nz}
    - Vector A_i ($i = 0, \ldots, N_{\mathrm{nz}} - 1$) stores the values of the matrix (collected row-by-row)
    - Vector C_i ($i = 0, \ldots, N_{\mathrm{nz}} - 1$) stores column index of each element of A
    - Vector R_i ($i = 0, \ldots, N_{\mathrm{nz}}$) stores the number of non-zero elements up to and not including row i with $R_0 = 0$
  - Example
    - $A_i = \{4, 7, 6, 9\}$
    - $C_i = \{3, 0, 1, 0\}$
    - $R_i = \{0, 1, 3, 3, 4\}$

$$\begin{pmatrix} 0 & 0 & 0 & 4 \\ 7 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 9 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 16 \\ 19 \\ 0 \\ 36 \end{pmatrix}$$

Computational Dwarf: Structured Grids (1/3)



- Applications perform periodic updates of **regular, multi-dimensional grids**
- Memory access features
  - Regular strided memory access
  - High spatial data locality, i.e. consecutive access to data that is close in memory
- Application areas
  - Heat transfer
  - Lattice Quantum Chromodynamics
  - Computational fluid dynamics using the Lattice Boltzmann Method

Computational Dwarf: Structured Grids (2/3)



Example: 2-dimensional Poisson equation

- Formulation in the continuum

$$-\frac{\partial^2 v(x,y)}{\partial x^2} - \frac{\partial^2 v(x,y)}{\partial y^2} = f(x,y)$$

- Discretisation of 2nd-order derivative

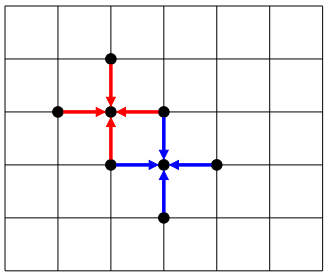
$$-\frac{\partial^2 v(x,y)}{\partial x^2} \leftarrow \frac{2v_{i,j} - v_{i-1,j} - v_{i+1,j}}{h^2}$$

- Discrete Poisson equaiton in 2 dimensions:

$$Tv = h^2 f \quad \text{where} \quad T = \begin{pmatrix} 4 & -1 & 0 & \cdots \\ -1 & 4 & -1 & \cdots \\ \vdots & \vdots & \ddots & \end{pmatrix}$$

Computational Dwarf: Structured Grids (3/3)

Graphical representation of Tv:



Observations:

- Matrix T acts as a stencil operator
- Any element of vector v is reused 4 times ☞ data locality

Computational Dwarf: Spectral Methods

- **Discrete Fourier Transform (DFT)**

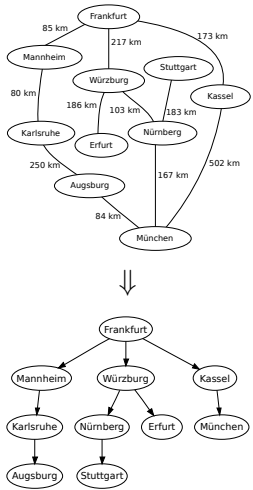
$$y_k = \sum_{n=0}^{N-1} x_n \, e^{-\frac{2\pi i}{N} n k}$$

- DFT using the radix-2 Cooley-Tukey algorithm

$$y_k = \sum_{m=0}^{N/2-1} x_{2m} \, e^{-\frac{2\pi i}{N} (2m) k} + \sum_{m=0}^{N/2-1} x_{2m+1} \, e^{-\frac{2\pi i}{N} (2m+1) k}$$

- Approach can be generalised to $N = N_1 \cdot N_2$:
  - Perform N_1 DFTs of size N_2 (original data layout)
  - Multiply by complex roots of unity + transform of $N_1 \times N_2$ matrix
  - Perform N_2 DFTs of size N_1 (new data layout)

Computational Dwarfs: Graph Traversal



[Wikimedia Commons]

- Applications that **traverse a number of objects** and **examine characteristics of those objects** such as would be used for search
- **Example problem**: Create search tree for city connections
- **Popular algorithm**: **Breadth-First Search (BFS)**
  - BFS explores all of the neighbour nodes at the present depth prior to moving on to the nodes at the next depth level
- **Typical characteristics**
  - Indirect table lookups
  - Little computation

Content



1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. BSP Model
6. Final

Model



- **Model** = Material or virtual artefacts that map to an origin [Thalheim; 2014]
  - An **origin** can be a material world or an artificial artefact
  - Model and origin only share relevant features (truncation property)
  - Models are designed for a specific purpose (purpose property)
  - Models intended to generate specific values and benefit (added value property)
- **Motivation for using models**
  - Performance modelling
  - Functional modelling and specification
  - Validation and verification
  - ...
- **Types of models**
  - Mathematical models
  - Event simulation
  - Signal level simulations (in soft- or hardware)
  - ...

Performance, Bandwidth, and Throughput



- **Performance** = Amount of work W per time unit

$$b(W) = \frac{W}{\Delta t(W)}$$

  - While this definition does not specify work, it needs to be well-defined, e.g. execution of a specific **workload** (also called problem size)

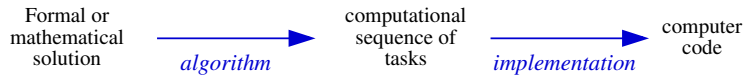
- **Bandwidth** = Amount of data passing a **particular interface** per time unit
  - It is important to define this interface

- **Throughput** = Amount of work items processed per time unit
  - Example: Number of floating-point operations per time unit

$$B_{\mathrm{fp}} = \frac{N_{\mathrm{fp}}}{\Delta t}$$

Performance Model

- **Performance model** = Estimate of latency Δt as a function of different parameters
- **Parameters**
  - Problem to be solved
  - Chosen algorithm
  - Implementation
  - Computer architecture
  - Problem size



Motivation for Performance Modelling

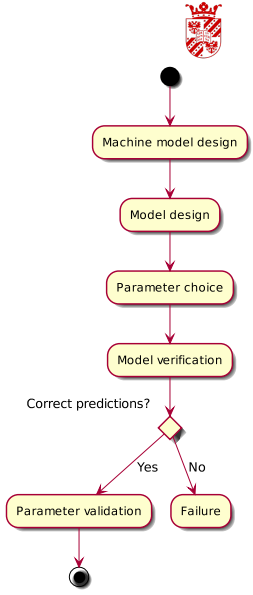


[Hoefler et al., 2011]

- **Modelling for performance optimization**
  - Identify optimization opportunities
  - Choose the right algorithm
- **Modelling during software development**
  - E.g., software design following the waterfall model including the steps: analysis, design, implementation, testing, maintenance
- **Modelling during system design and proposal**
  - Create basis for trade-off decisions
- **Modelling during deployment and testing**
  - Compare expected and obtained performance
- **Modelling during operation**
  - Compare expected and operational performance (e.g., to identify performance degradation)

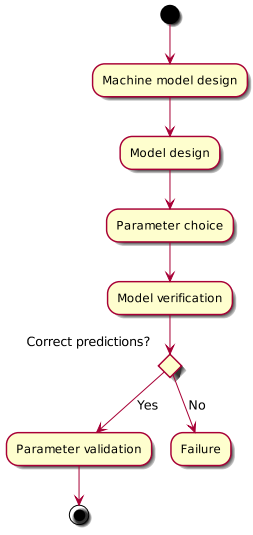
Modelling Workflow (1/2)

- **Machine model design**
  - Any performance model is based on a simplified model of the hardware (and/or software) architecture
- **Performance Model design**
  - Analytic model
  - Semi-analytical model
  - Simulation model
- **Model parameter choice/determination**, e.g.
  - Determine parameters using data sheets
  - Fit parameters to performance data
  - Apply training/learning methods



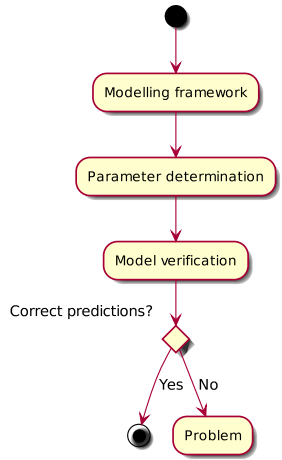
Modelling Workflow (2/2)

- **Model verification**
  - Use model to make predictions
  - Compare predictions to measurements
- Optional: **Parameter validation**
  - Do parameters have expected size?



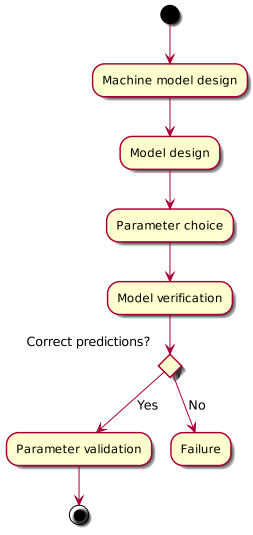
Black Box Modelling

- **Black box modelling** approaches
  - Statistical methods to parametrize performance data
  - Machine learning techniques
- Black box modelling methods may be independent from architecture and application
  - Rely on pattern matching and curve fitting
- **Advantage(s)**
  - Simple to apply
- **Disadvantage(s)**:
  - Does not create understanding
    - Failure of the validation step merely showcase limitation of the approach



White Box Modelling

- **White box modelling** approaches
  - Choice/design of both, machine model and performance model based on knowledge and/or assumptions
  - Example: Knowledge about the memory hierarchy
- In case of **failures**
  - Select new or adjust the machine model
  - Select new or adjust the performance model
- **Disadvantage(s)**:
  - Much more difficult
- **Advantage(s)**:
  - Creation of new insight
    - Model assumptions are challenged



(White Box) Performance Model Classification



- **Analytic performance models**
  - Performance is expressed with purely analytic expressions
  - Parameters are fixed from data-sheet information
- **Semi-analytical performance models**
  - Performance is expressed with purely analytic expressions
  - Parameters are determined through fits to performance data
- **Simulation models**
  - Simulates behaviour of machine model for given input stimuli
  - May include cycle-accurate simulation of digital logic

Modelling Different Quantities



- **Which quantities to model?**
  - **Latency** Δt
    - Time-to-solution in case of latency of full application
  - **Energy-to-solution** ΔE
  - **Resource utilization**
    - Example: Number of cache misses
  - **Workload requirements**
    - Example: Number of floating-point operations
- **Classification of quantities involved in modelling**
  - **Deterministic quantities**
    - Number of floating-point operations
  - **Statistical quantities**
    - Latency Δt

Workload



- **Workload** = A definition of a particular work requests imposed on the underlying computing resources
- The definition may involve
  - Software application
  - Command-line parameters
  - Input data
  - ...
- Workload documentation is important as **performance is typically a function of the work**

Content



1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. **Machine Models**
5. BSP Model
6. Final

Proxy Architectures (1/2)



[Ang et al. (Sandia/LBNL), 2014]

- **Proxy architecture** = Parameterized version of a machine model
  - Parameter set limited to describe hardware capability and capacity of relevant components
  - **Example parameters**
    - Number of cores
    - Memory bandwidth, floating-point throughput
    - Capacity of caches
- **Why parametrized model?**
  - Model different versions of the same types of components, e.g. CPU
  - Explore architectural design space

Proxy Architectures (2/2)



[Ang et al. (Sandia/LBNL), 2014]

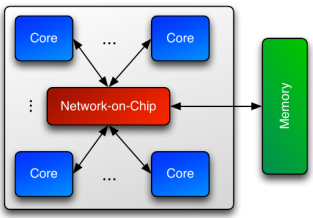
- **Different scopes**
  - Processor level
  - Node level
  - System level
- **Different level of details**
  - As little details as possible, as many details as necessary

Proxy-Architectures for CPUs (1/2)

[Ang et al. (Sandia/LBNL), 2014]

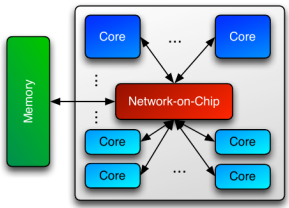
Homogeneous multi-core processor:

- Multiple cores of same type
- Processor-internal network
- External memory



Heterogeneous multi-core processor:

- Multiple cores of different type
- Processor-internal network
- External memory

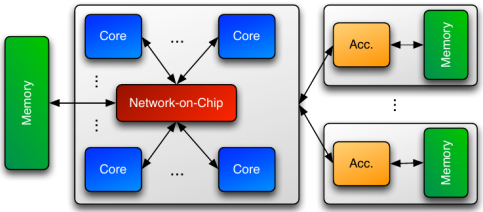


Proxy-Architectures for CPUs (2/2)

[Ang et al. (Sandia/LBNL), 2014]

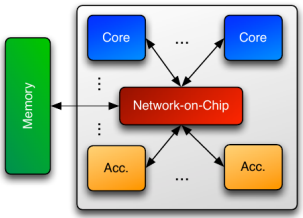
Multi-core processor plus discrete accelerator:

- Example: CPU + GPU

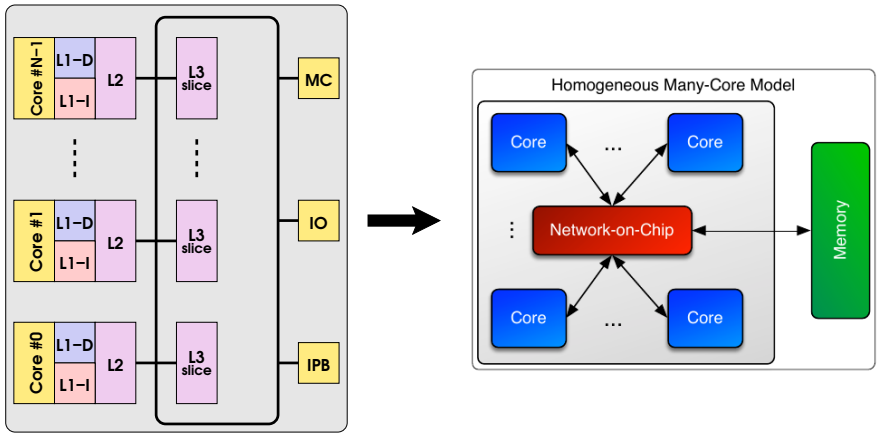


Multi-core processor plus integrated accelerators:

- Example: AMD APU



Proxy-Architecture for Intel Xeon E5 (Haswell)



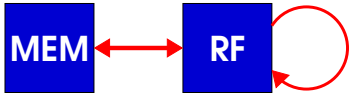
Abstract Machine Model (1/3)



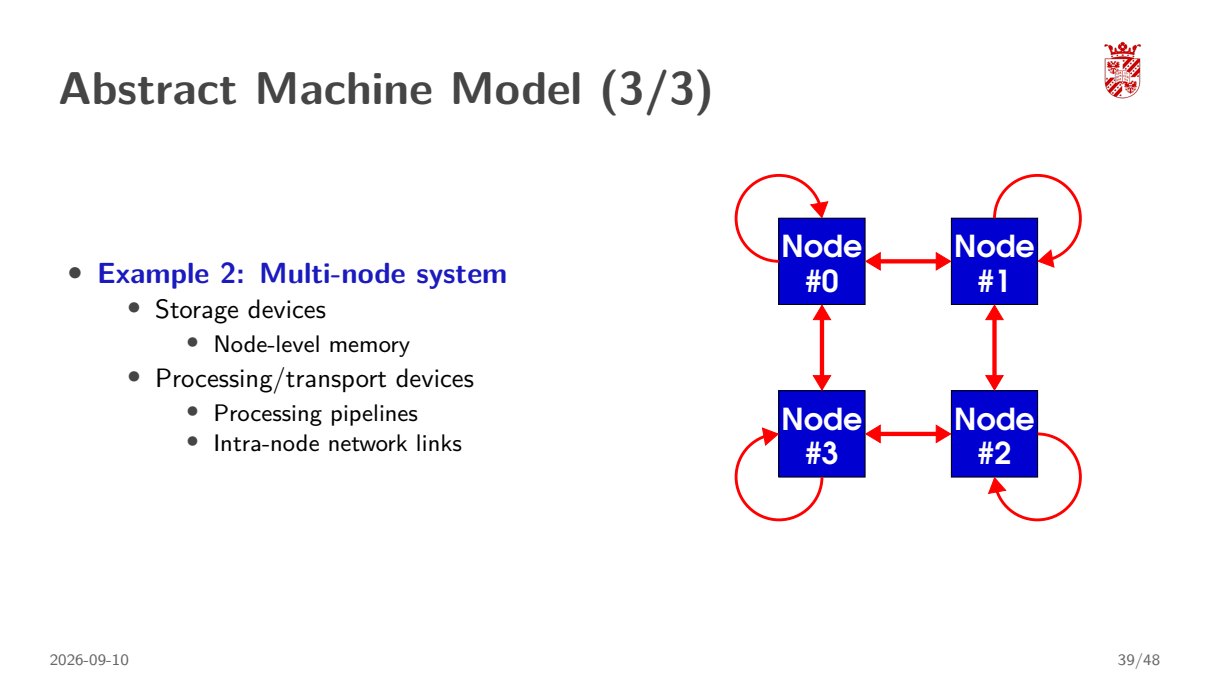
- **Machine** = set of interconnected devices
  - **Storage devices**
  - **Processing/transport devices**
- **Storage devices**
  - Examples
    - External memory
    - Register file
  - Parameter: capacity
- **Processing/transport devices**
  - Examples
    - Arithmetic pipeline
    - Memory bus
  - Possible parameters
    - Bandwidth/throughput
    - Start-up latency

Abstract Machine Model (2/3)

- **Graphical representation**
  - Vertices = storage devices
  - Edges = Processing/transport devices
- **Example 1: Processor architecture**
  - Storage devices
    - External memory (MEM)
    - Register file (RF)
  - Processing/transport devices
    - Memory bus
    - Processing pipeline



Abstract Machine Model (3/3)



- **Example 2: Multi-node system**
  - Storage devices
    - Node-level memory
  - Processing/transport devices
    - Processing pipelines
    - Intra-node network links

Content

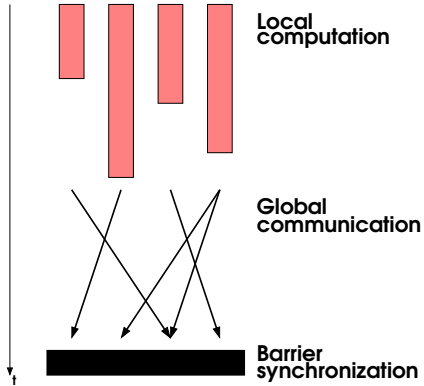


1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. **BSP Model**
6. Final

Bulk-Synchronous Parallel (BSP) Model (1/2)

[Valiant, 1990]

- **Machine model**:
  - Parallel machine comprising set of processors
  - Each processor has local memory
  - Interconnect network allows to communicate packets of fixed size
- **Performance modelling approach**:
  - Computation is divided in **supersteps**
    - Operations on local data
    - Send/receive packets
    - Barrier synchronisation



Bulk-Synchronous Parallel (BSP) Model (2/2)



- **Performance model parameters**:
  - P ... Number of processors
  - g ... Gap = time required to communicate one packet
  - L ... "Latency" = minimum duration of a superstep
  - S ... Number of supersteps
- **Latency** of superstep i and total latency

$$\Delta t_i = w_i + g\,h_i + L, \quad \Delta t = \sum_i \Delta t_i = W + g\,H + L\,S$$

  where $W = \sum_i w_i$ and $H = \sum_i h_i$
- **Superstep work depth** w_i is the largest amount of work performed in superstep i
- h_i **is largest number of packets sent or received** by any processor in superstep i

Cannon Algorithm (1/2)



- **Computational task**

$$C_{i,j} \leftarrow \sum_{k=0}^{N-1} A_{i,k}\, B_{k,j} \quad (i,j = 0,\ldots,N-1)$$

- **Parallelisation strategy**
  - Organise P processing units in a 2-dimensional mesh: $P = p^2$
  - Perform 2-dimensional decomposition of matrices A, B, C
    - Each processing unit holds a block of size $(N/p) \times (N/p)$
  - All elements of C stay fixed on a processing unit
    - All computations are performed on the processing unit that will hold the result
  - Blocks of matrices A and B need to be communicated
    - Canon algorithm consists of a description how to shift the blocks

Cannon Algorithm (2/2)

Shifting of blocks according to Cannon:

- Initial alignment step
- Blocks of A are shifted left
- Blocks of B are shifted upwards

Example $N = 3$, $P = 3^2$:

Initial state	Initial alignment	Shift #1	Shift #2
$\begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} \\ A_{1,0} & A_{1,1} & A_{1,2} \\ A_{2,0} & A_{2,1} & A_{2,2} \end{pmatrix}$	$\begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} \\ A_{1,1} & A_{1,2} & A_{1,0} \\ A_{2,2} & A_{2,0} & A_{2,1} \end{pmatrix}$	$\begin{pmatrix} A_{0,1} & A_{0,2} & A_{0,0} \\ A_{1,2} & A_{1,0} & A_{1,1} \\ A_{2,0} & A_{2,1} & A_{2,2} \end{pmatrix}$	$\begin{pmatrix} A_{0,2} & A_{0,0} & A_{0,1} \\ A_{1,0} & A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} & A_{2,0} \end{pmatrix}$
$\begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} \\ B_{1,0} & B_{1,1} & B_{1,2} \\ B_{2,0} & B_{2,1} & B_{2,2} \end{pmatrix}$	$\begin{pmatrix} B_{0,0} & B_{1,1} & B_{2,2} \\ B_{1,0} & B_{2,1} & B_{0,2} \\ B_{2,0} & B_{0,1} & B_{1,2} \end{pmatrix}$	$\begin{pmatrix} B_{1,0} & B_{2,1} & B_{0,2} \\ B_{2,0} & B_{0,1} & B_{1,2} \\ B_{0,0} & B_{1,1} & B_{2,2} \end{pmatrix}$	$\begin{pmatrix} B_{2,0} & B_{0,1} & B_{1,2} \\ B_{1,0} & B_{1,1} & B_{2,2} \\ B_{0,0} & B_{2,1} & B_{0,2} \end{pmatrix}$

BSP Model for Cannon Algorithm

- Number of superstep $S = \sqrt{P}$
- Local computation at each superstep

$$w_i \propto \left(\frac{N}{\sqrt{P}}\right)^3$$

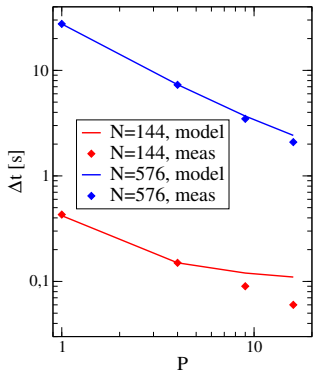
- Number of packets sent at each superstep

$$h_i = 2\left(\frac{N}{\sqrt{P}}\right)^2$$

- Overall latency:

$$\Delta t = \gamma \frac{N^3}{P} + g\,2\,\frac{N^2}{\sqrt{P}} + L\sqrt{P}$$

[Goudrea et al., 1999]



Content



1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. BSP Model
6. **Final**

Summary Lecture #04



- Benchmarking
  - Motivation for benchmarking
  - Benchmarks categorization
  - Introduced various benchmarks
- Computational dwarfs
  - Dense and sparse linear algebra, structured grids, spectral methods, graph traversal
- Introduction to modelling
- Machine models versus performance models
- Modelling techniques
  - BSP model

Questions?



[Wiki Media]



university of groningen

Performance Engineering: Lecture #05

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-14

Overview



1. Semi-Analytical Performance Models
2. Information Exchange
3. Roofline Model
4. ECM Model
5. Final

Content



1. Semi-Analytical Performance Models
2. Information Exchange
3. Roofline Model
4. ECM Model
5. Final

Semi-Analytical Performance Modelling



Procedure: [Hoefler, 2011]

- **Analytic steps**
  - Identify application kernels
  - Identify input parameters that influence runtime
    ☞ Parametrization of work-load W
  - On basis of prior knowledge formulate scaling formulae describing dependence of latency $\Delta t(W)$ as function of work-load W
- **Empirical steps**
  - Measure $\Delta t(W)$ for different W
  - Fit scaling formulae to measurements
- **Model verification**
  - Compare predicted and measured values
- **Parameter validation**
  - Check fitted parameters for plausibility

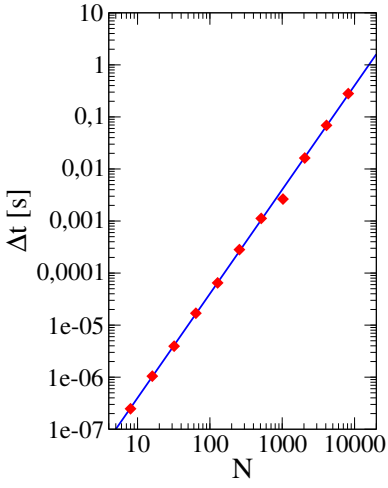
Example (1/2)

- **Kernel**: Matrix-vector multiplication

$$y_i \leftarrow \sum_{j=0}^{N-1} A_{ij} x_j \quad (i = 0, \ldots, N-1)$$

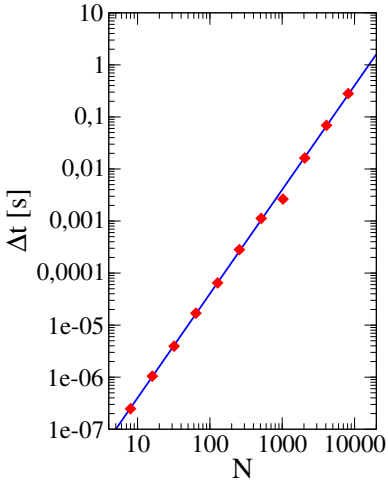
- **Ansatz**

$$\Delta t(N) = a_0 + a_2 N^2$$



Example (2/2)

- **Test setup**:
  - Double-precision floating-point numbers
  - Intel i7-2620M processor, single thread
  - No compiler optimization
- **Results**:
  - $a_0 \simeq 0\,\text{ns}$, $a_2 \simeq 4\,\text{ns}$
- **Plausibility analysis**
  - Clock $f = 2.7\,\text{GHz} \Rightarrow 1/f = 0.4\,\text{ns}$
  - Need about 10 cycles per (i, j)



Content



1. Semi-Analytical Performance Models
2. Information Exchange
3. Roofline Model
4. ECM Model
5. Final

Information Exchange Function



- Each task on the computer eventually implies that information is transferred from a storage device x to a storage device y
  - Machine model = abstract machine model
- **Information Exchange Function**:

$$I^k_{x,y}(W) \quad = \quad \text{data transferred between computer sub-systems for specific computational task } k$$

x ... source storage device (e.g. memory)
y ... destination storage device (e.g. register file)
W ... workload

Bandwidth or Throughput vs. Latency (1/2)



Reminder:

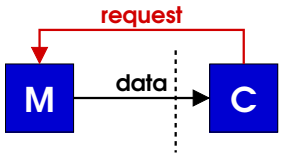
- Definitions bandwidth versus throughput:
  - **Bandwidth** = Amount of items passing a particular interface per time unit
  - **Throughput** = Amount of work done in a given time
- Definition **latency (or response time)** $\Delta t =$
  Time between events indicating start of an task and the event signalling its completion

Bandwidth or Throughput vs. Latency (2/2)



Example:

- Memory read operation

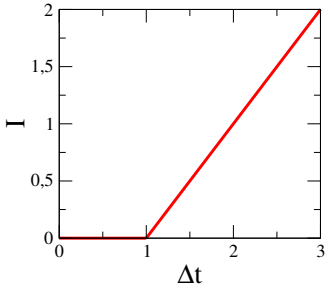


- **Bandwidth**: Amount of data passing memory interface per time unit
- **Latency**:
  - **Start event**: Memory read operation started
  - **End event**: All data arrived in processor
- **Note**: This is an abstract model and details remain open
  - Example: Load instruction versus cache line read command

Latency-Bandwidth Model (1/2)

- Simple model describing **latency** Δt as a function of the information exchange I
- Model parameters:
  - **Start-up latency** λ = Time between event trigger and start of arrival of response
  - **Bandwidth** β: Parametrizes time to exchange information I
- Model:

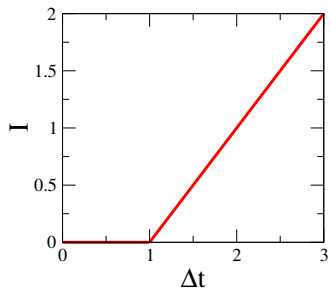
$$\Delta t(I) = \lambda + \frac{I}{\beta}$$

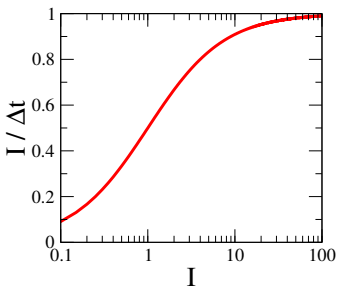


Latency-Bandwidth Model (2/2)

- **Effective bandwidth** $\tilde{b} = I/\Delta t(I)$
- Example: Read I Bytes from memory or network

$$\Delta t(I) = \lambda + \frac{I}{\beta} \quad \Rightarrow \quad \tilde{b}(I) = \left(\frac{\lambda}{I} + \frac{1}{\beta}\right)^{-1}$$





Information Exchange Model: Latency Predictions

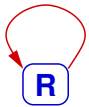


Ansatz to predict latency:

$$\Delta t^k_{x,y} \simeq \lambda_{x,y} + \frac{I^k_{x,y}}{\beta_{x,y}}$$

Example:

- x, y = R
- $\beta_{R,R}$ = throughput arithmetic unit
- $I^k_{x,y}$ = number of input (or output) operands



arithmetic unit

register file

Beware of limitations of this ansatz:

- Transfer mechanism may depend on task size N
- Bandwidth changes due to resource congestion is ignored
- ...

Simple Example (1/2)

- Example operation: $y_i \leftarrow \alpha \cdot x_i$ $(i = 1, \ldots, N)$
  x_i, y_i ... vectors of single precision floating-point numbers
  α ... single-precision floating-point number
- Information exchange for each i:

load x_i	4 Bytes
multiply α and x_i	1 Flop
store y_i	4 Bytes

- Assume the following hardware parameters:

Memory bandwidth β_{mem}	1 Byte/clock cycle
Floating-point unit throughput β_{fp}	1 Flop/clock cycle

- Latency predictions:

Δt_{mem}	$N \cdot 8$ clock cycles
Δt_{fp}	$N \cdot 1$ clock cycles

Simple Example (2/2)



- **Latency** for full operation:
  - No overlap of memory load/store and arithmetic operations:

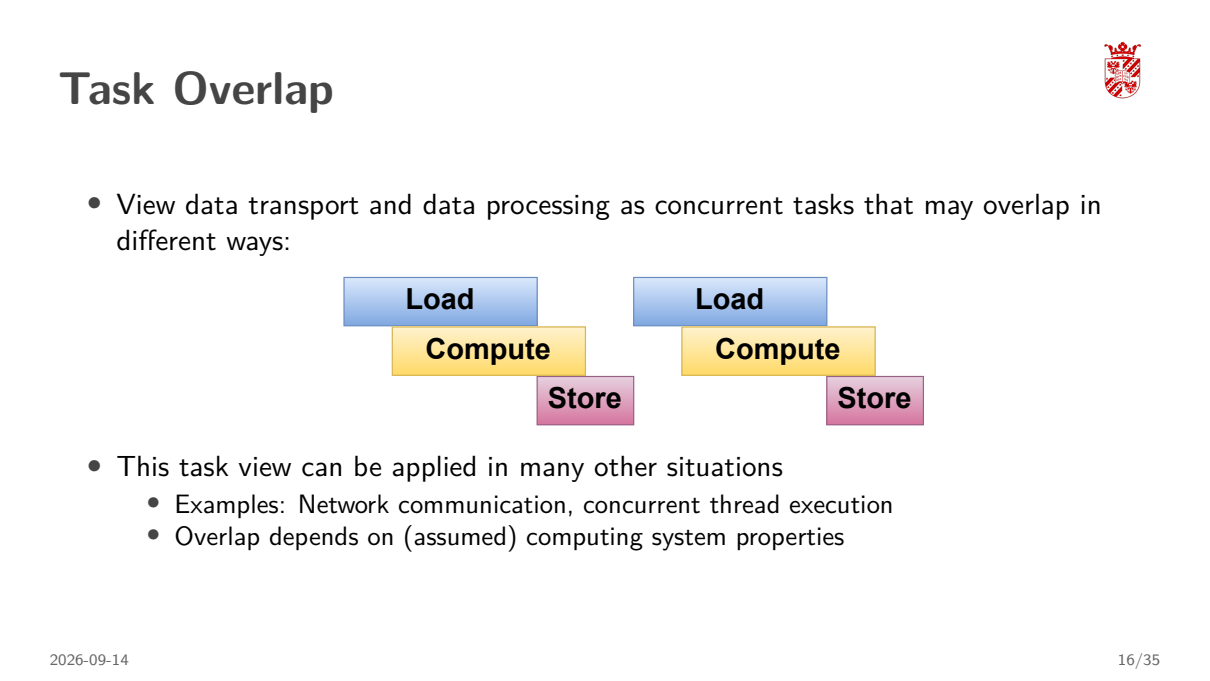
$$\Delta t(N) = \Delta t_{\text{mem}} + \Delta t_{\text{fp}} = N \cdot 9$$

  - Perfect overlap of memory load/store and arithmetic operations:

$$\Delta t(N) = \max\left(\Delta t_{\text{mem}}, \Delta t_{\text{fp}}\right) = N \cdot 8$$

☞ **Memory (bandwidth) limited** problem
  - Opposed to **compute (throughput) limited** problems

Task Overlap



- View data transport and data processing as concurrent tasks that may overlap in different ways:

- This task view can be applied in many other situations
  - Examples: Network communication, concurrent thread execution
  - Overlap depends on (assumed) computing system properties

Memory Bandwidth



- Definitions
  - Measured bandwidth b_{mem}
    - Amount of transferred data I_{mem} over measured latency Δt
  - Latency-bandwidth model parameter β_{mem}
  - Effective bandwidth $\tilde{b}_{\mathrm{mem}}$
  - **Nominal hardware bandwidth** B_{mem}
    - Data rate on hardware link/bus
- Relations
  - Upper limit for measured bandwidth: $b_{\mathrm{mem}} \leq B_{\mathrm{mem}}$
  - Upper limit for effective bandwidth: $\tilde{b}_{\mathrm{mem}} \leq \beta_{\mathrm{mem}}$
  - Note: $\beta_{\mathrm{mem}} \leq B_{\mathrm{mem}}$ is not guaranteed
    - But expected if bandwidth-latency model is a good model

Semi-Analytical and Information Exchange Modelling

- **Ansatz**: Use Information Exchange to create semi-analytical ansatz, e.g.

$$\Delta t = a_0 + a_1 I(W)$$

  - a_1 has dimension of inverse throughput/bandwidth
- **Advantages**
  - No need to determine start-up latency λ and asymptotic bandwidth β
  - Allow for easy comparison with hardware performance parameters

Example



- Double-precision matrix-vector multiplication

$$y_i \leftarrow \sum_{j=0}^{N-1} A_{ij}\, x_j \quad (i = 0, \ldots, N-1)$$

- Information exchange

$$\begin{aligned} I_{\mathrm{mem,rf}} &= (N^2 + 2\,N)\,8\,\mathrm{B} \\ I_{\mathrm{rf,rf}} &= (2\,N - 1)\,N\,\mathrm{Flop} \end{aligned}$$

- Re-use previously presented results for Intel i7-2620M and comparison to nominal throughput/bandwidth B

Storage devices	$1/a_1$	B
mem,rf	$2\,\mathrm{GB/s}$	$21.3\,\mathrm{GB/s}$
rf,rf	$0.5\,\mathrm{GFlop/s}$	$43.2\,\mathrm{GFlop/s}$

Content



1. Semi-Analytical Performance Models
2. Information Exchange
3. Roofline Model
4. ECM Model
5. Final

Arithmetic Intensity



- **Arithmetic Intensity** $=$ [H. Harris, 2005]

$$\mathrm{AI} = \frac{\text{Number of floating-point operations}}{\text{Amount of transferred data}} = \frac{I_{\mathrm{fp}}}{I_{\mathrm{mem}}}$$

- Example: $y_i \leftarrow \alpha \cdot x_i$

$$\mathrm{AI} = \frac{1\,\mathrm{Flop}}{8\,\mathrm{B}} \qquad \text{single-precision}$$

$$\mathrm{AI} = \frac{1\,\mathrm{Flop}}{16\,\mathrm{B}} \qquad \text{double-precision}$$

Roofline Model (1/2)



[S. Williams et al.; 2009]

- Floating-point performance

$$b_{\mathrm{fp}} = \frac{I_{\mathrm{fp}}}{\Delta t} \quad \Rightarrow \quad \Delta t = \frac{I_{\mathrm{fp}}}{b_{\mathrm{fp}}}$$

- Floating-point and memory performance limits:

$$b_{\mathrm{fp}} \leq B_{\mathrm{fp}}, \quad b_{\mathrm{mem}} \leq B_{\mathrm{mem}}$$

- Upper limit for latency assuming perfect overlap of memory and arithmetic operations:

$$\Delta t \geq \max\left(\Delta t_{\mathrm{fp}}, \Delta t_{\mathrm{mem}}\right)$$

Roofline Model (2/2)



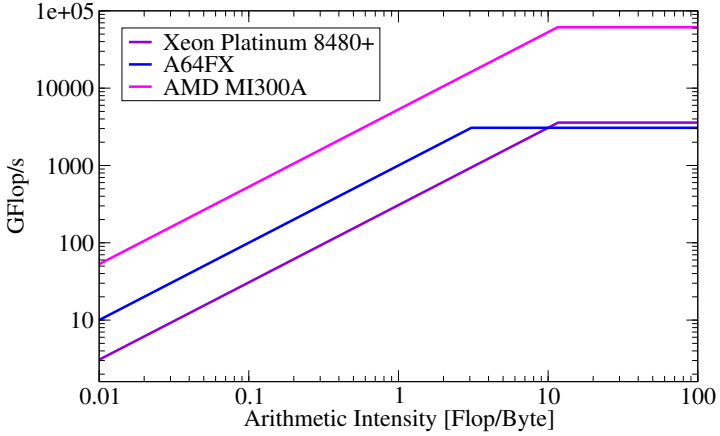
- Assume latency-bandwidth model, but neglect start-up latencies

$$\Delta t \geq \max\left(\lambda_{\mathrm{fp}} + \frac{I_{\mathrm{fp}}}{B_{\mathrm{fp}}}, \lambda_{\mathrm{mem}} + \frac{I_{\mathrm{mem}}}{B_{\mathrm{mem}}}\right) \quad \Rightarrow \quad \frac{1}{\Delta t} \leq \min\left(\frac{B_{\mathrm{fp}}}{I_{\mathrm{fp}}}, \frac{B_{\mathrm{mem}}}{I_{\mathrm{mem}}}\right)$$

- Upper limit (=roof) for floating-point performance ("attainable performance")

$$b_{\mathrm{fp}} \leq \min\left(B_{\mathrm{fp}}, \frac{I_{\mathrm{fp}}}{I_{\mathrm{mem}}} B_{\mathrm{mem}}\right) = \min\left(B_{\mathrm{fp}}, \mathrm{AI} \cdot B_{\mathrm{mem}}\right)$$

Roofline Examples



Cache-Aware Roofline Models



- The naive Roofline model only considers bandwidth towards the external memory
- **Cache-aware roofline models**: Account for accesses to the different data cache levels
- **One possible approach**: Consider B_{mem} as a function of I_{mem} [Ilic et al.; 2014]
  - For sufficiently small I_{mem} set

$$B_{\mathrm{mem}}(I_{\mathrm{mem}} \ll C_{\mathrm{L}_1}) = B_{\mathrm{L}_1}$$

- **Alternative approach**: Define information exchange for the different levels
  - This results in different values for AI

Content



1. Semi-Analytical Performance Models
2. Information Exchange
3. Roofline Model
4. ECM Model
5. Final

ECM Model (1/3)

[J. Treibig and G. Hager, 2010]

- **ECM** = Execution-Cache-Model
- **Goal**: Analytic model to predict performance of bandwidth-limited algorithms in all memory hierarchy levels
- **Ansatz** for overall latency

$$\Delta t = \max(\Delta t_{\mathrm{OL}}, \Delta t_{\mathrm{nOL}} + \Delta t_{\mathrm{mem}})$$

Δt_{mem}	...	Cycles spent on load/store up to L1
Δt_{OL}	...	Cycles spent on overlapping instructions
Δt_{nOL}	...	Cycles spent on non-overlapping instructions

ECM Model (2/3)



- **Ansatz** for data transport latency
  - Data resides in L1 cache

$$\Delta t_{\mathrm{mem}} = 0$$

  - Data resides in external memory

$$\Delta t_{\mathrm{mem}} = \Delta t_{\mathrm{L1,L2}} + \cdots + \Delta t_{\mathrm{LLC,mem}}$$

ECM Model (3/3)



- **Conventions**
  - All memory transfers in **units of cache lines (CL)**
  - All times to be understood as inverse bandwidth, i.e. $\Delta t_{\mathrm{mem}} = 1\,\mathrm{CL}/B_{\mathrm{mem}}$
  - Time in units of core clock cycles
  - External memory bandwidth is determined phenomenologically
- Short-cut notation for model parameters (assuming L3 = LLC)

$$\{\Delta t_{\mathrm{OL}} || \Delta t_{\mathrm{nOL}} | \Delta t_{\mathrm{L1,L2}} | \Delta t_{\mathrm{L2,L3}} | \Delta t_{\mathrm{L3,mem}}\}$$

- Short-cut for model predictions

$$\{\Delta t_{\mathrm{L1}} \rceil \Delta t_{\mathrm{L2}} \rceil \Delta t_{\mathrm{L3}} \rceil \Delta t_{\mathrm{mem}}\}$$

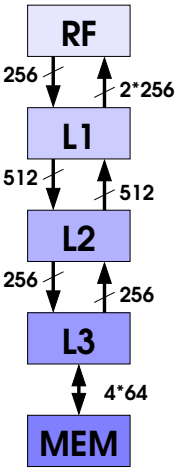
  where Δt_{L1} is the prediction if data resides in L1

Intel Xeon E5-v3 (Haswell) Memory Hierarchy



- 2 CL can be loaded to RF in 2 cycles
- Bandwidth to memory is shared by all cores
- Example memory bandwidth
  - Core clock speed $f = 2.3\,\text{GHz}$
  - $b_{\text{mem}} = 32.4\,\text{GB/second}$ (measured)
  - Cache line load latency

$$\begin{aligned}\Delta t_{\text{mem}} &= (64\,\text{B/CL}) \\ &\quad \cdot (32.4\,\text{GB/second})^{-1} \\ &\quad \cdot (2.3\,\text{GHz}) \\ &\simeq 4.54\,\text{cycle/CL}\end{aligned}$$



ECM Model Example (1/2)



[J. Hofmann et al.; 2015]

- Computational task

$$c \leftarrow \sum_i a_i \cdot b_i$$

- ECM model parameter determination
  - Processor is capable of performing 2 256-bit SIMD FMA per cycle
    - Need 1 cycle to process a full cache line
    - $\Delta t_{\mathrm{OL}} = 1$
  - Load of 2 CL from L1→RF and L2→L1 requires 2 cycle
    - $\Delta t_{\mathrm{L1,L2}} = 2$
  - Assume that loads are non-overlapping
    - $\Delta t_{\mathrm{nOL}} = 2$
  - Bandwidth from L3→L2 only 256 bit/cycle
    - $\Delta t_{\mathrm{L2,L3}} = 4$
  - Load of 2 CL from MEM requires $2 \cdot 4.54$ cycle
  - Short notation

$$\{1 \,||\, 2 \,|\, 2 \,|\, 4 \,|\, 9.1\} \text{ cycle}$$

ECM Model Example (2/2)

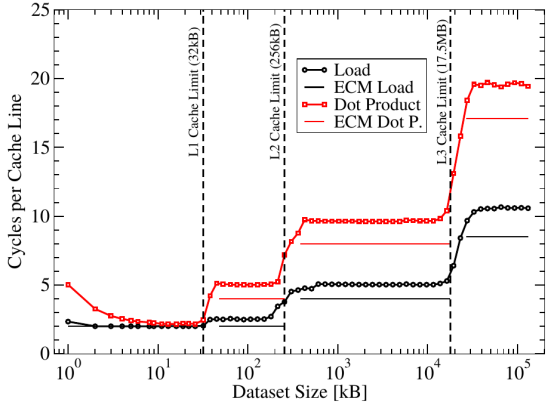
- ECM model prediction

$$\{2 \rceil 4 \rceil 8 \rceil 17.1\} \text{ cycle}$$

- Measured timings on Xeon E5-2695 v3 (Haswell, 14 cores)

$$\{2.1 \rceil 4.7 \rceil 9.6 \rceil 19.4\} \text{ cycle}$$

[J. Hofmann et al.; 2015]



Content



1. Semi-Analytical Performance Models
2. Information Exchange
3. Roofline Model
4. ECM Model
5. Final

Summary Lecture #05



- Modelling techniques
  - Semi-Analytical performance models
  - Information Exchange and Latency-Bandwidth model
  - Roofline model and cache-aware Roofline model
  - Execution-Cache-Model (ECM)

Questions?



[Wiki Media]



university of groningen

Performance Engineering: Lecture #06

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-17

Overview



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. Final

Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. Final

Performance Metric

- **Parallel speedup** using P processes

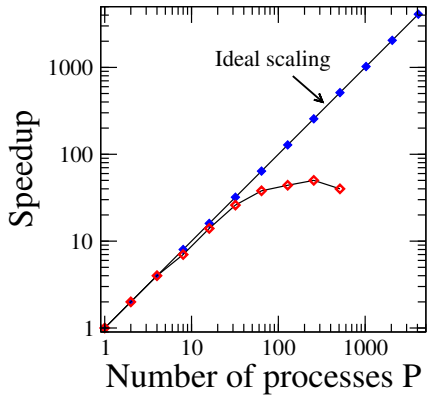
$$S(P) = \frac{\Delta t_{\text{serial}}}{\Delta t_{\text{parallel}}(P)}$$

- **Parallel efficiency**

$$\epsilon = \frac{S(P)}{P}$$

Example: $S = 26$ using $P = 32$ processing elements
$\Rightarrow \epsilon = 81\%$

- **Super-linear speedup**: $S > P$



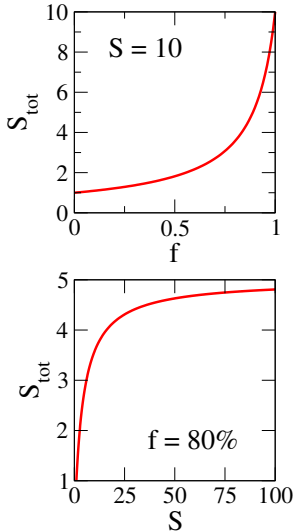
Amdahl's Law

- Consider the following case:
  - Fraction f of application can be accelerated resulting in speed-up by factor S
  - The remaining fraction $1 - f$ remains serial
- Execution time after acceleration:

$$\Delta t' = (1 - f)\Delta t + \frac{f\Delta t}{S}$$

- **Total speed-up**

$$S_{\text{tot}} = \frac{\Delta t}{\Delta t'} = \frac{1}{(1 - f) + f/S}$$



Gustafson's Law



- Amdahl's law: Only considers scaling with fixed workload ☞ **strong scaling**
- Different ansatz by Gustafson:
  - Increase workload proportional to number of processing elements P ☞ **weak scaling**
  - Assume non-parallelized code part to be independent of workload
- Execution times
  - Serial execution
    - Execution time for original problem: Δt
    - Execution time for scaled problem size: $(1-f)\Delta t + fP\Delta t$
  - Parallel execution
    - Execution time for scaled problem size: $(1-f)\Delta t + f\Delta t$
- Speed-up:

$$S = \frac{(1-f)\Delta t + fP\Delta t}{(1-f)\Delta t + f\Delta t} = (1-f) + fP$$

Parallel Efficiency: Amdahl vs. Gustafson



- Parallel efficiency according to Amdahl:

$$\epsilon_{\text{Amdahl}}(P) = \frac{1}{P} \cdot \frac{1}{(1-f) + f/P} = \frac{1}{P(1-f) + f}$$

- Parallel efficiency according to Gustafson:

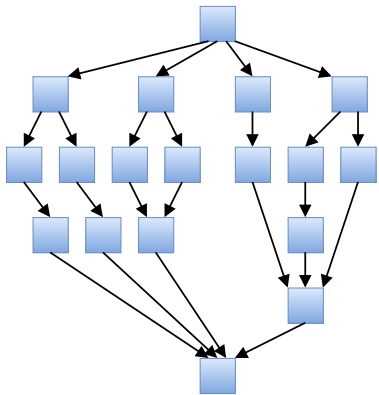
$$\epsilon_{\text{Gustafson}}(P) = \frac{(1-f) + fP}{P} = \frac{1-f}{P} + f$$

- Limit of large number of processing elements P:

$$\begin{aligned} \lim_{P\to\infty} \epsilon_{\text{Amdahl}}(P) &= 0 \\ \lim_{P\to\infty} \epsilon_{\text{Gustafson}}(P) &= f \end{aligned}$$

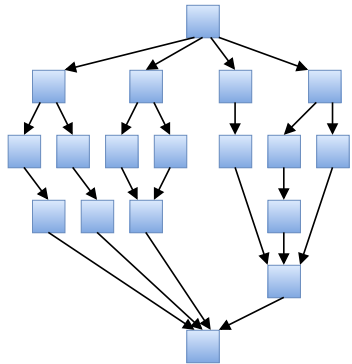
Work-Span Model

- **Consider**: A **task graph**, which is assumed to be a **Directed Acyclic Graph (DAG)**
- **Assumptions** (inspired by the PRAM model)
  - The latency of each task is one time unit
  - Task scheduling is greedy
- **Parallel latency** $\Delta t(P)$ for P processing units
- **Shown example**:
  - Maximum concurrency: 7
  - $\Delta t(P) = 6$ for $P \geq 7$



Work and Span

- $\Delta t(1)$ is called **work of an algorithm**
  - Time needed to executed all task in a serial manner
  - Here: $\Delta t(1) = 18$
- $\Delta t(\infty)$ is called **span of an algorithm**
  - Time needed on an ideal machine with an infinite number of processors
  - Here: $\Delta t(\infty) = 6$
  - Alternative terms: **step complexity**, **depth**



Speed-Up Limit

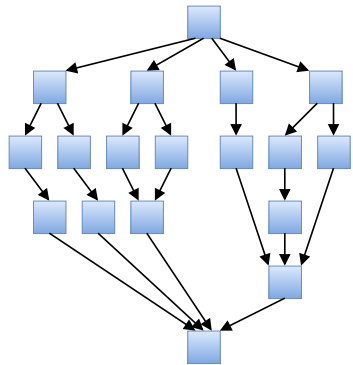
- Super-linear speed-up is not possible in the work-span model

$$S(P) = \frac{\Delta t(1)}{\Delta t(P)} \leq \frac{\Delta t(1)}{\Delta t(1)/P} = P$$

- On an ideal machine with greedy scheduling, adding processors never slows down an algorithm

$$S(P) = \frac{\Delta t(1)}{\Delta t(P)} \leq \frac{\Delta t(1)}{\Delta t(\infty)}$$

  - In words: speed-up $\leq$ work / span



Brent's Lemma



- **Brent's Lemma** provides a lower bound for speed-up [Richart P. Brent; 1964]

$$\Delta t(P) \leq \frac{\Delta t(1) - \Delta t(\infty)}{P} + \Delta t(\infty)$$

- **Interpretation** of Brent's Lemma
  - **Imperfectly parallelizable work**, which always takes $\Delta t(\infty)$
  - **Perfectly parallelizable work**, which for $P = 1$ takes the remaining time $\Delta t(1) - \Delta t(\infty)$
- **Note**: $\Delta t(\infty)$ depends on the concurrency
  - Not to be confused with the serial execution time Δt_{serial} in Amdahl's Law
  - Increasing concurrency will reduce $\Delta t(\infty)$
  - In practice, excess parallelism (**parallel slack**) may result in more efficient task allocation

Comparison: Amdahl versus Brent



- Amdahl's argument results in a lower bound for $\Delta t(P)$
  - Bound applies in case of ideal parallelization
  - Incentive to think about more scaling ("HPC view")
- Brent's Lemma puts an upper bound for $\Delta t(P)$
  - It says what happens if task dependencies are taken into account

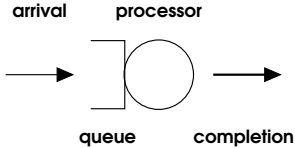
Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. Final

Network Queuing Models

- **Machine model**: Network of queues



- **Empirical ansatz**: System is observed for time period T
- **Observables**:

B	Time during which resource was busy
A	Number of observed request arrivals
C	Number of observed request completions
W	Average waiting time

Derived Quantities

- Utilization
$$U = \frac{B}{T}$$
- Throughput = completion rate
$$X = \frac{C}{T}$$
- Average service requirement
$$S = \frac{B}{C}$$
- Average number of requests in the system
$$N = \frac{W}{T}$$
- Average system residence time
$$R = \frac{W}{N}$$

Utilization Law

$$\frac{B}{T} = \frac{C}{T} \cdot \frac{B}{C} \quad \Rightarrow \quad U = XS$$

Utilization Law = The utilization of a resource is equal to the product of the throughput of that resource and the average service requirement of that resource.

Example:

- Disk services 40 requests per second
- Servicing a request requires 0.02 seconds
- Disk utilization is 80 %

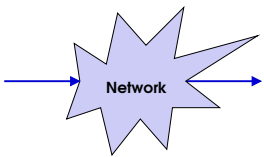
Little's Law

$$\frac{W}{T} = \frac{C}{T} \cdot \frac{W}{C} \quad \Rightarrow \quad N = XR.$$

Little's Law = Average number of requests in a system is equal to the product of the throughput of that system and the average time spent in that system by a request

Example:

N = Average number of packets in network
X = Packet injection rate
R = Average amount of time packet spends in network



Queuing Delay (1/2)



- Estimated average time of staying in the queue (**residence time**):

$$R = N \cdot S + S$$

- Apply Little's Law and solve for R:

$$R = (X \cdot R) \cdot S + S \quad \Rightarrow R = \frac{S}{1 - X \cdot S}$$

- Consider system utilization:

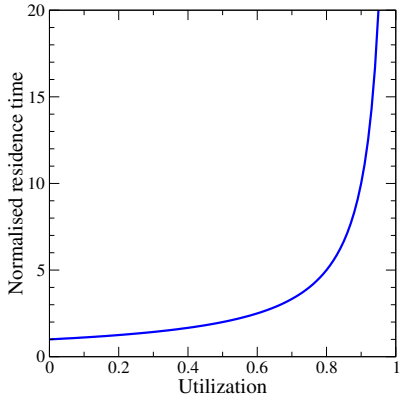
$$U = \frac{B}{T} = \frac{B}{C} \cdot \frac{C}{T} = S \cdot X$$

Queuing Delay (1/2)

- Determine normalised residence time as a function of utilization:

$$\frac{R}{S} = \frac{1}{1-U}$$

☞ **High utilization leads to diverging queuing delays**



Kendall's Notation

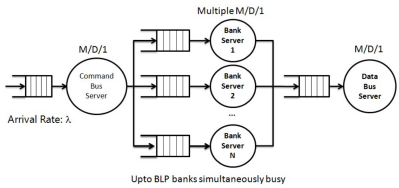


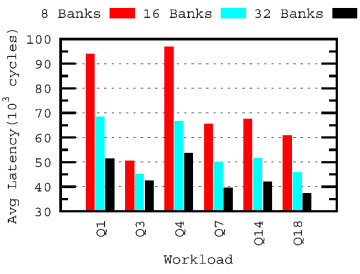
- Queues are characterised by (at least) the following parameters
  - A: Arrival process
    - M: Markovian (Poisson distribution)
    - D: Degenerate distribution
    - ...
  - S: Service time distribution
    - M: Markovian (exponential distribution)
    - D: Degenerate distribution
    - ...
  - c = Number of servers
- Kendall's notation: A/S/c
  - Example: M/D/1 queue
    - Arrivals occur at rate λ according to a Poisson process
    - Service times are deterministic (service rate $\mu = 1/D$)

Queuing Theory Example: DRAM Modelling

- Model ansatz: DRAM channel as a 3-stage network of queues
  - Stage 1: command bus
    - Memory requests arrive following an Poisson distributed (arrival rate $\lambda = A/T$)
  - Stage 2: memory banks
    - Set of bank servers to model the actual servicing of any request by the memory
  - Stage 3: data bus
    - Data bus server which captures the actual data transfer on the channel

[Gulur et al., 2014]





Content



1. Parallel Performance
2. Queueing Theory
3. **Simulation Models**
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. Final

Simulation Models



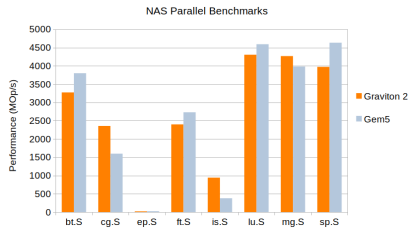
- Simulation models require a simulator for creating observables
- **Examples**:
  - **Cache simulators**: Simulation of cache behaviour
  - Full CPU **discrete event simulators**, e.g. gem5
  - **Hardware Description Language (HDL) simulators**: Signal-level simulations

Event Simulator: GEM5

[Binkert et al.]

- Modular simulator for different CPU models and attached memory
- Support for different CPU models using different ISA: Alpha, ARM, SPARC, x86
- Support of different scopes
  - System-call Emulation (SE)
  - Full-System (FS):
    model complete system including OS
- Different memory models

[Brank, Pleiter; 2023]



Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. **Relevant Hardware Architecture Concepts**
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. Final

Parallel Computer Architectures



- **Microarchitecture level**
  - Pipelines
  - Instruction-level parallelism
  - Data-level parallelism: SIMD/vector instructions
  - Simultaneous multithreading (SMT)
- **Chip level**
  - Multi-core architectures
  - Memory-level parallelism
- **Node level**
  - Multi-socket
- **System level**
  - Multi-node

Pipeline: Time Diagram

	P_0	P_1	P_2	P_3	P_4
t_0	C_0				
t_0+1	C_1	C_0			
t_0+2	C_2	C_1	C_0		
t_0+3	C_3	C_2	C_1	C_0	
t_0+4	C_4	C_3	C_2	C_1	C_0
t_0+5	C_5	C_4	C_3	C_2	C_1
t_0+6		C_5	C_4	C_3	C_2
t_0+7			C_5	C_4	C_3
t_0+8				C_5	C_4
t_0+9					C_5

- **pipeline filling**:
  $t = t_0 ... (t_0 + 3)$
- **pipeline draining**:
  $t = (t_0 + 6) ... (t_0 + 9)$
- Time to perform n operations in k **pipeline stages** assuming a nominal throughput $B = 1$:

$$\Delta t_p(n, k) = n + (k - 1)$$

Here: $\Delta t_p(6, 5) = 10$

Pipeline Costs



- In our example: throughput $B = 1$
- **Throughput**:

$$b(n,k) = \frac{\text{number of operations}}{\Delta t_p(n,k)} = \frac{n}{n+(k-1)}$$

  Note: $\lim_{n\to\infty} b(n,k) = B$
- **Gain (or speed-up)**

$$s(n,k) = \frac{\text{scalar execution time}}{\text{pipelined execution time}} = \frac{n\,k}{n+(k-1)}$$

  Note: $\lim_{n\to\infty} s(n,k) = k$
- **Efficiency**

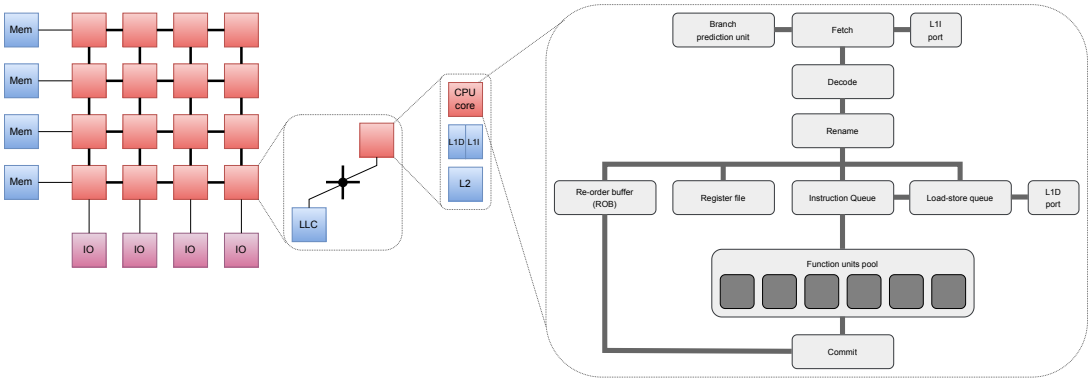
$$\epsilon(n,k) = \frac{1}{k}s(n,k) = \frac{n}{n+(k-1)}$$

Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. **Processor Microarchitecture: Overview**
6. Processor Microarchitecture: Front-end
7. Final

Example: Many-Core CPU Architecture



Instruction Pipeline Stages



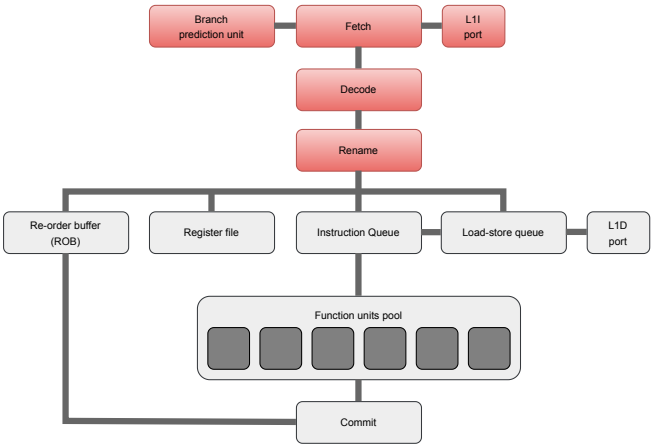
- **Fetch**: Retrieve the instruction from memory/instruction cache
- **Decode**: Interpret the instruction
- **Issue/Dispatch** (optional): Forward the front entry of the instruction pipeline to a free reservation station
- **Execute**: Execute the instruction
- **Write**: Write the results
- **Commit/Retire** (optional): Make results visible

Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. **Processor Microarchitecture: Front-end**
7. Final

Microarchitecture Overview: Front-End



Instruction Fetch and Decode



- **Fetching** typically means reading a cache line from L1I cache and pushing the instructions into a pipeline
  - The throughput per cycle is limited
  - Branches may result in long latencies due to
    - Refilling the pipeline
    - Instruction cache misses
- Instruction **decoding** is usually parallelised
  - Decoding of complex instructions may take longer
  - Instructions may be decoded to multiple **micro-instructions** (also known as **micro-ops**)
- **Performance engineering potential**: Low

Branching and Speculative Execution



- **Branch instructions** cause interrupts in sequential execution of instructions
  - Branches may be **unconditional** or **conditional**
- Mitigation strategy: **branch prediction** in combination with **speculative execution**
  - **Branch prediction** = Mechanisms by which processors predict the outcome of a conditional branch before the actual condition is fully evaluated
  - **Speculative execution** = Facilitate execution of an instruction based on branch prediction by ensuring that commit only happens once condtion is fully evaluated
- **Performance engineering potential**: High

Branch Prediction



- Branch prediction mechanisms
  - Branch hint instructions (software)
  - Hardware that allows processor to learn from past behaviour
    - Information stored in Branch History Tables (BHT)
    - Today, multi-level predictors are used
- Hardware branch predictors will struggle in the following situations:
  - Highly irregular branches
  - First-time branches
  - Indirect branches: branch target is not fixed
  - Difficult to detect data dependencies
- **Performance engineering potential**: Low

Register Renaming



- **Architected registers** = Set of registers foreseen by the ISA
- **Physical registers** = Set of registers implemented in hardware

$$N_{\text{architected registers}} \leq N_{\text{physical registers}}$$

- Hardware may support renaming of architected registers such that all physical registers are used
- **Performance engineering potential**: Low

Content



1. Parallel Performance
2. Queueing Theory
3. Simulation Models
4. Relevant Hardware Architecture Concepts
5. Processor Microarchitecture: Overview
6. Processor Microarchitecture: Front-end
7. **Final**

Summary Lecture #06



- Modelling techniques
  - Parallel performance: Amdahl's Law, Gustafson's Law, work-span model
  - Queuing theory models
  - Simulation models
- Reviewed key architectural concepts like pipelining, parallelisation
- Review of key elements of processor microarchitectures with the following having the highest performance engineering potential
  - Front-end: Branching

Questions?



[Wiki Media]



university of groningen

Performance Engineering: Lecture #07

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-23

Overview



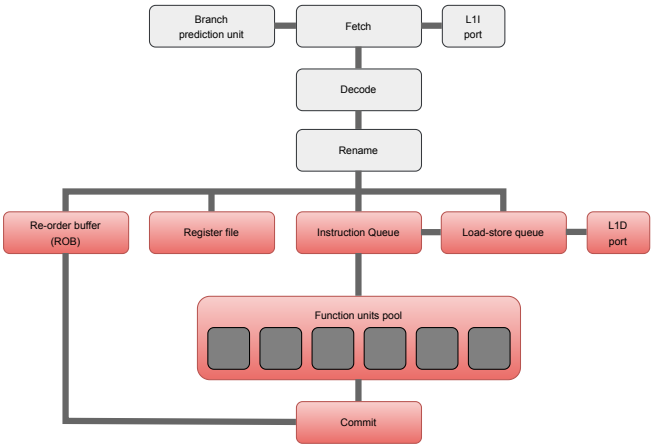
1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. Top-down Microarchitecture Analysis
5. Loop Optimizations (1)
6. Final

Content



1. **Processor Microarchitecture: Back-end**
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. Top-down Microarchitecture Analysis
5. Loop Optimizations (1)
6. Final

Microarchitecture Overview: Back-End



In-Order and Out-of-Order



- Back-end instruction processing may be **in-order** or **out-of-order**
- **Out-of-order instruction processing** = Allow instructions to be executed in an order differently from program order
  - The microarchitecture must guarantee that execution appears like instructions being executed in the original order
  - Reordering depth depends on microarchitecture implementation
  - The front-end is always in-order
- **Performance engineering potential**: Depends on re-ordering depth

Execution Units



- Challenge of **unfilled execution pipelines**
  - Typical issue: data hazards that cause instructions to be stalled
- Modern microarchitectures feature a large number of execution units
  - Exploiting **instruction-level parallelism** is key
  - Relevant metric: **instructions per cycle (IPC)**
- Different execution units have different characteristics
  - Typically different depth
  - Some execution units may not support pipelining
- **Performance engineering potential**: High

Data hazards



- Assume that in program order instruction *i* occurs before instruction *j*
- The instruction *j* is **data dependent** on the instruction *i* if either
  - Instruction *i* produces a result that may be used by instruction *j*
  - Instruction *j* is data dependent on instruction *k*, and instruction *k* is data dependent on instruction *i*
- **Read-after-write (RAW) hazard**
  Instruction *j* tries to read a source before it is updated by instruction *i*

  Example for wrong scheduling of instructions:

```
imul R0, R1, R5 !
isub R6, R3, R7 ! Register R6 not written, yet
iadd R5, R2, R6 !
```

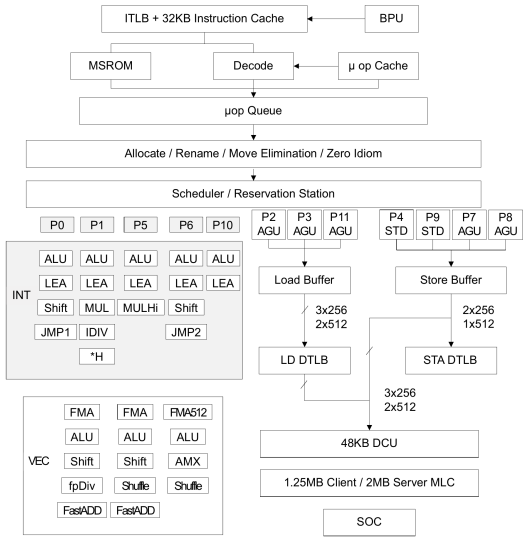
Content



1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. Top-down Microarchitecture Analysis
5. Loop Optimizations (1)
6. Final

Intel Golden Cove (1/2)

[Intel, “Optimization Reference Manual”, v49]



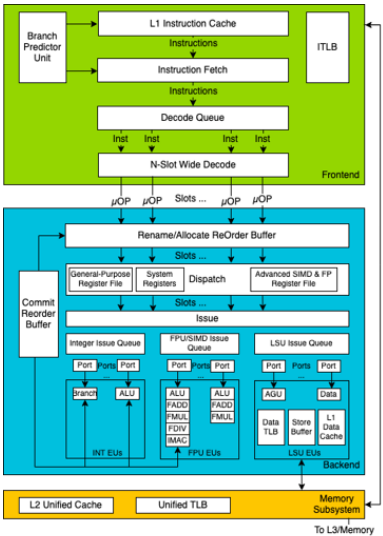
Intel Golden Cove (2/2)



- Front-end fetch bandwidth: $32\,\mathrm{B/cycle}$
- Decoder: 6 instructions/cycle
- Reservation station capacity: 144 micro-instructions
- Number of execution ports: 12
- Floating-point arithmetics latency: [Intel; March 2022]
  - Addition (`ADDPD`): 3 cycles
  - Multiplication (`MULSD`): 4 cycles

Arm Neoverse v2 (1/2)

["Arm Neoverse V2 Core Telemetry Specification", version 2.0]



Arm Neoverse v2 (2/2)



- Front-end fetch and decode bandwidth: implementation specific
- Decoder: 6 instructions/cycle
- Reservation station capacity: implementation specific
- Number of execution ports: 17
- Floating-point arithmetics latency: ["Arm Neoverse V2 Software Optimization Guide", December 2022]
  - Addition (`FADD`): ≥ 2 cycles
  - Multiplication (`FMUL`): ≥ 3 cycles
  - Multiply-accumulate (`FMADD`): ≥ 4 cycles

Content



1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. **Machine Code Analysis: `llvm-mca`**
4. Top-down Microarchitecture Analysis
5. Loop Optimizations (1)
6. Final

`llvm-mca`: LLVM Machine Code Analyzer



- Tool to **statically model** the performance of machine code
  - Given an assembly code sequence, the tool estimates the IPC, hardware resource pressure, and other quantities
  - Requires model to be available in LLVM
  - Code may be compiled with GCC
- **Analysis steps**:
  - Instruction information (default)
  - Resource pressure (default)
  - Throughput Bottlenecks (`-bottleneck-analysis`)
  - Critical sequence (`-bottleneck-analysis`)
- **Choice of architecture**:
  - `-mtriple=<target triple>`: `x86_64-unknown-unknown`, `aarch64-unknown-unknown`
  - `-mcpu=<cpu>`: Specify the processor for which to analyse the code (use `help` to list supported processors)

Example Code


```
1  double a[8], b[8], c[8];
2
3  void kernel(void) {
4      c[0] = a[0] + b[0];
5      c[1] = a[1] + b[1];
6      c[2] = a[2] + b[2];
7      c[3] = a[3] + b[3];
8      c[4] = a[4] + b[4];
9      c[5] = a[5] + b[5];
10     c[6] = a[6] + b[6];
11     c[7] = a[7] + b[7];
12 }
```

Compiler command line: `gcc-14 -O1`

`llvm-mca` on x86 (1/4)



Tool command line: `llvm-mca-20 \`
`-mtriple=x86_64-unknown-unknown \`
`-mcpu=tigerlake \`
`-bottleneck-analysis`

Header:

```
1 Iterations:        100
2 Instructions:      2600
3 Total Cycles:      1353
4 Total uOps:        4400
5
6 Dispatch Width:    6
7 uOps Per Cycle:    3.25
8 IPC:               1.92
9 Block RThroughput: 8.5
```

`llvm-mca` on x86 (2/4)



Instruction information:

```
Instruction Info:
[1]: #uOps
[2]: Latency
[3]: RThroughput
[4]: MayLoad
[5]: MayStore
[6]: HasSideEffects (U)

[1]    [2]    [3]    [4]    [5]    [6]    Instructions:
 1      100    0.25                  U     endbr64
 1      5      0.50    *                   movsd a(%rip), %xmm0
 2      9      0.50    *                   addsd b(%rip), %xmm0
 2      1      0.50            *           movsd %xmm0, c(%rip)
 1      5      0.50    *                   movsd 8+a(%rip), %xmm0
 2      9      0.50    *                   addsd 8+b(%rip), %xmm0
 2      1      0.50            *           movsd %xmm0, 8+c(%rip)
```

`llvm-mca` on x86 (3/4)



Resource pressure:

```
89  Resources:
90  [0]   - ICXDivider
91  [1]   - ICXFPDivider
92  [2]   - ICXPort0
93  [3]   - ICXPort1
94  [4]   - ICXPort2
95  [5]   - ICXPort3
96  [6]   - ICXPort4
97  [7]   - ICXPort5
98  [8]   - ICXPort6
99  [9]   - ICXPort7
100 [10]  - ICXPort8
101 [11]  - ICXPort9
102
103
104 Resource pressure per iteration:
105 [0]    [1]    [2]    [3]    [4]    [5]    [6]    [7]    [8]    [9]    [10]   [11]
106 -      -      4.43   4.44   8.50   8.50   4.00   1.00   1.13   4.00   4.00   4.00
107
108 Resource pressure by instruction:
109 [0]    [1]    [2]    [3]    [4]    [5]    [6]    [7]    [8]    [9]    [10]   [11]   Instructions:
110 -      -      -      0.87   -      -      -      -      0.13   -      -      -      endbr64
111 -      -      -      -      0.50   0.50   -      -      -      -      -      -      movsd      a(%rip), %xmm0
112 -      -      0.93   0.07   0.50   0.50   -      -      -      -      -      -      addsd      b(%rip), %xmm0
113 -      -      -      -      -      -      -      -      -      -      1.00   1.00   movsd      %xmm0, c(%rip)
```

`llvm-mca` on x86 (4/4)



Bottleneck analysis:

```
12 Cycles with backend pressure increase [ 13.16% ]
13 Throughput Bottlenecks:
14   Resource Pressure       [ 5.76% ]
15   - ICXPort2  [ 5.76% ]
16   - ICXPort3  [ 5.76% ]
17   Data Dependencies:      [ 13.16% ]
18   - Register Dependencies [ 13.16% ]
19   - Memory Dependencies   [ 0.00% ]
```

`llvm-mca` on Arm (1/3)



Tool command line: `llvm-mca-20 \`
`-mtriple=aarch64-unknown-unknown \`
`-mcpu=neoverse-v2 \`
`-bottleneck-analysis`

Header:

```
1 Iterations:        100
2 Instructions:      2500
3 Total Cycles:      412
4 Total uOps:        3700
5
6 Dispatch Width:    16
7 uOps Per Cycle:    8.98
8 IPC:               6.07
9 Block RThroughput: 2.7
```

`llvm-mca` on Arm (2/3)

Instruction information:

```
63 Instruction Info:
64 [1]: #uOps
65 [2]: Latency
66 [3]: RThroughput
67 [4]: MayLoad
68 [5]: MayStore
69 [6]: HasSideEffects (U)
70
71 [1]    [2]    [3]    [4]    [5]    [6]    Instructions:
72  1      1     0.25                        adrp   x0, .LANCHOR0
73  1      1     0.17                        add    x0, x0, :lo12:.LANCHOR0
74  1      1     0.17                        add    x2, x0, #64
75  1      1     0.17                        add    x1, x0, #128
76  2      6     0.33    *                   ldp    d31, d30, [x0, #64]
77  2      6     0.33    *                   ldp    d28, d29, [x0, #128]
78  1      2     0.25                        fadd   d31, d31, d28
79  1      2     0.25                        fadd   d30, d30, d29
80  2      2     0.50           *            stp    d31, d30, [x0]
```

`llvm-mca` on Arm (3/3)



Resource pressure:

```
Resources:
[0.0] - V2UnitB
[0.1] - V2UnitB
[1.0] - V2UnitD
[1.1] - V2UnitD
[2.0] - V2UnitFlg
[2.1] - V2UnitFlg
[2.2] - V2UnitFlg
[3]   - V2UnitL2
[4.0] - V2UnitL01
[4.1] - V2UnitL01
[5]   - V2UnitM0
[6]   - V2UnitM1
[7]   - V2UnitS0
[8]   - V2UnitS1
[9]   - V2UnitS2
[10]  - V2UnitS3
[11]  - V2UnitV0
[12]  - V2UnitV1
[13]  - V2UnitV2
[14]  - V2UnitV3


Resource pressure per iteration:
[0.0]  [0.1]  [1.0]  [1.1]  [2.0]  [2.1]  [2.2]  [3]    [4.0]  [4.1]  [5]    [6]    [7]    [8]    [9]    [10]   [11]   [12]   [13]   [14]
0.50   0.50   -      -      -      -      -      3.97   4.01   4.02   0.66   0.66   0.67   0.67   0.67   0.67   3.00   3.01   2.99   3.00

Resource pressure by instruction:
[0.0]  [0.1]  [1.0]  [1.1]  [2.0]  [2.1]  [2.2]  [3]    [4.0]  [4.1]  [5]    [6]    [7]    [8]    [9]    [10]   [11]   [12]   [13]   [14]   Instructions:
-      -      -      -      -      -      -      -      -      -      -      0.33   -      0.67   -      -      -      -      -      -      adrp    x0, .LANCHOR0
-      -      -      -      -      -      -      -      -      -      0.33   -      0.33   -      -      0.34   -      -      -      -      add x0, x0, :lo12:.LANCHOR0
-      -      -      -      -      -      -      -      -      -      -      0.33   -      -      0.34   0.33   -      -      -      -      add x2, x0, #64
```

Content



1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. **Top-down Microarchitecture Analysis**
5. Loop Optimizations (1)
6. Final

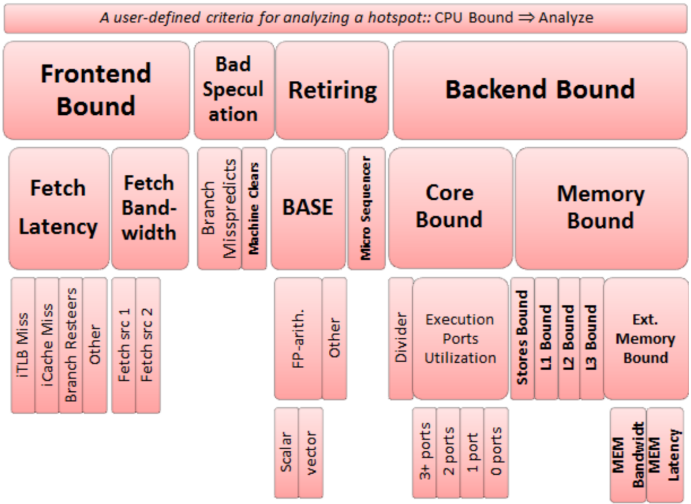
Top-Down Microarchitecture Analysis (TMA)



- **Idea**: Identify and group hardware performance counters to facilitate a top-down microarchitecture performance analysis
- This strategy has first been implemented for Intel processors with the following **top-level analysis categories**: [A. Yasin; 2014]
  - **Front-end bound**: Fraction of slots where front-end undersupplies back-end
  - **Bad speculation**: Fraction of slots wasted due to incorrect speculations
  - **Retiring**: Fraction of slots utilized by useful work (committed instructions)
  - **Back-end bound**: Fraction of slots where no micro-instructions are being delivered due to a lack of required resources

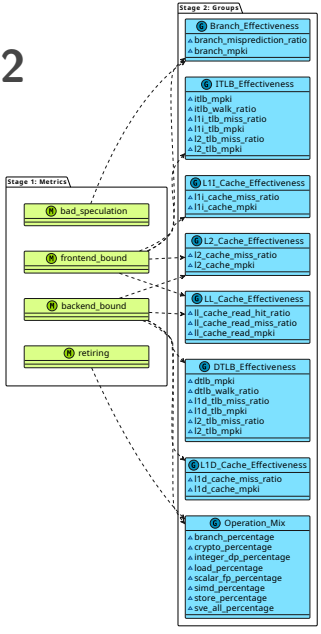
TMA on Intel Processors

[A. Yasin; 2014]



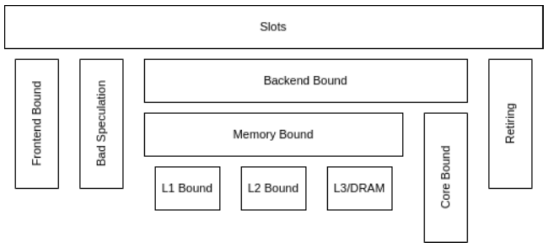
TMA on Arm Neoverse v2

"Arm Neoverse V2 Core Telemetry Specification", version 2.0



TMA on Kunpeng 920

[F. Banchelli et al.; 2023]



Front-end bound	$\texttt{FETCH_BUBBLE}/(4 \cdot \texttt{CPU_CYCLES})$
Bad speculation	$(\texttt{INST_SPEC} - \texttt{INST_RETIRED})/(4 \cdot \texttt{CPU_CYCLES})$
Retiring	$\texttt{INST_RETIRED}/(4 \cdot \texttt{CPU_CYCLES})$
Backend bound	1 - (Front-end bound + Bad speculation + Retiring)

TMA using `perf` (1/2)



Example code:

```c
double a[8], b[8], c[8];

void kernel(void) {
    c[0] = a[0] + b[0];
    c[1] = a[1] + b[1];
    c[2] = a[2] + b[2];
    c[3] = a[3] + b[3];
    c[4] = a[4] + b[4];
    c[5] = a[5] + b[5];
    c[6] = a[6] + b[6];
    c[7] = a[7] + b[7];
}

int main() {
    for (int i = 0; i < 100000000; i++)
        kernel();
}
```

Compiler command: `gcc-14 -O1`

TMA using `perf` (2/2)



Perf output on Intel i7-1165G7 (Tiger Lake):

```
Performance counter stats for './kernel1-x86.x':

            252.10 msec task-clock                #    0.998 CPUs utilized
                 2      context-switches          #    7.933 /sec
                 2      cpu-migrations            #    7.933 /sec
                54      page-faults               #  214.201 /sec
     1,154,029,248      cycles                    #    4.578 GHz
     2,902,969,578      instructions              #    2.52  insn per cycle
       300,499,859      branches                  #    1.192 G/sec
            10,538      branch-misses             #    0.00% of all branches
                        TopdownL1                 #     47.0 %  tma_backend_bound
                                                  #      1.2 %  tma_bad_speculation
                                                  #      0.1 %  tma_frontend_bound
                                                  #     51.7 %  tma_retiring

       0.252639477 seconds time elapsed

       0.252563000 seconds user
       0.000000000 seconds sys
```

Content



1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. Top-down Microarchitecture Analysis
5. **Loop Optimizations (1)**
6. Final

Introduction to Loop Optimization



- Loop optimization is a typical performance engineering effort
- The following loop optimization techniques will be covered:
  - Loop fusion
  - Loop unroll
  - Loop unroll and jam
- The following analysis techniques will be introduced:
  - Identification of loop carry dependencies
  - Critical path analysis

Case Study: `field_copy`



- **Purpose** of this case study: Explore impact of loop fusion
- **Specification** for `field_copy`:
  - Copy a field of 3-dimensional position variables
  - The variables are double-precision numbers

`field_copy`: Initial Implementation



```c
#include <stdlib.h>

typedef struct {
  double x;
  double y;
  double z;
} pos_t;

void field_copy(pos_t *restrict a,
                pos_t *restrict b,
                int n) {
  for (int i = 0; i < n; i++)
    a[i].x = b[i].x;

  for (int i = 0; i < n; i++)
    a[i].y = b[i].y;

  for (int i = 0; i < n; i++)
    a[i].z = b[i].z;
}
```

```c
int main() {
  const int n = 1048576;
  pos_t *a   = (pos_t *) calloc(n, sizeof(pos_t));
  pos_t *b   = (pos_t *) calloc(n, sizeof(pos_t));

  for (int i = 0; i < 1000; i++)
    field_copy(a, b, n);
}
```

Loop Fusion



- **Loop fusion** = Loop optimization technique where separate loops are merged
- **Prerequisites**:
  - All loops iterate over the same range
  - Loops do not reference each other's data
- **Target benefit(s)**
  - Reduce loop control overhead (branching, condition checks)
  - Enable further optimizations within larger basic blocks
  - Reduced data transfer
- **Possible drawback(s)**
  - Large code size
  - More instruction cache misses

`field_copy`: Fused Loops



```c
#include <stdlib.h>

typedef struct {
  double x;
  double y;
  double z;
} pos_t;

void field_copy_fused(
        pos_t *restrict a,
        pos_t *restrict b,
        int n) {
  for (int i = 0; i < n; i++) {
    a[i].x = b[i].x;
    a[i].y = b[i].y;
    a[i].z = b[i].z;
  }
}
```

```c
int main() {
  const int n = 1048576;
  pos_t *a   = (pos_t *) calloc(n, sizeof(pos_t));
  pos_t *b   = (pos_t *) calloc(n, sizeof(pos_t));

  for (int i = 0; i < 1000; i++)
    field_copy_fused(a, b, n);
}
```

`field_copy`: Kunpeng 920 Results

	N_{cycle}	N_{instr}
Original, `-O1`	$14.52 \cdot 10^9$	$15.73 \cdot 10^9$
Fused, `-O1`	$3.44 \cdot 10^9$	$8.39 \cdot 10^9$
Original, `-O3`	$2.62 \cdot 10^5$	$1.55 \cdot 10^5$
Fused, `-O3`	$2.57 \cdot 10^5$	$1.55 \cdot 10^5$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - Using compiler optimization level `-O1`: Number of cycles and instructions improve by $4.2\times$ and $1.9\times$
  - Using compiler optimization level `-O3`: Explicit optimization is not required

Content



1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. Top-down Microarchitecture Analysis
5. Loop Optimizations (1)
6. **Final**

Summary Lecture #07



- Review of key elements of processor microarchitectures with the following having the highest performance engineering potential
  - Back-end: Out-of-order execution, execution units
- Overview of different processor microarchitectures:
  - Intel Golden Cove
  - Arm Neoverse v2
- Static code analysis using `llvm-mca`
- Dynamic top-down microarchitecture analysis (TMA):
  - Front-end bound
  - Bad speculation
  - Retiring
  - Back-end bound
- Loop optimizations: Loop fusion

Questions?



[Wiki Media]



university of groningen

Performance Engineering: Lecture #08

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-24

Overview



1. Loop Optimizations (2)
2. SIMD and Vector Instructions
3. Auto-Vectorization (1)
4. Final

Content



1. Loop Optimizations (2)

2. SIMD and Vector Instructions

3. Auto-Vectorization (1)

4. Final

Case Study: `mat_accum`



- **Purpose** of this case study: Explore impact of (outer) loop unroll and jam
- **Specification** for `mat_accum`:
  - Implement the following task:

    $$A_{i,j} \leftarrow A_{i,j-1} + b_j$$

    for $i = 0, \ldots, N-1$ and $j = 1, \ldots, M-1$
  - The variables are double-precision numbers

mat_accum: Initial Implementation



```c
#include <stdlib.h>

void kernel(double *restrict a, double *restrict b, int n, int m) {
    for (int i = 0; i < n; i++)
      for (int j = 1; j < m; j++)
          a[i*m + j] = a[i*m + (j-1)] + b[j];
}

int main() {
  const int n = 1024;
  const int m = 2048;
  double *a   = (double *) calloc(n*m, sizeof(double));
  double *b   = (double *) calloc(m, sizeof(double));

  for (int i = 0; i < 1000; i++)
    kernel(a, b, n, m);
}
```

Loop Unroll



- **Loop unroll** = A loop optimization that copies the loop body to reduce the number of iterations
- **Target benefit(s)**
  - Reduce loop control overhead (branching, condition checks)
  - Enable further optimizations within larger basic blocks
- **Possible drawback(s)**
  - Large code size
  - More instruction cache misses

mat_accum: Implementation with Unroll



```c
#include <stdlib.h>

void kernel_unroll( double *restrict a, double *restrict b, int n, int m) {
  for (int i = 0; i < n; i += 2)
    for (int j = 1; j < m; j++)
      a[i*m + j] = a[i*m + (j-1)] + b[j];

  for (int i = 1; i < n; i += 2)
    for (int j = 1; j < m; j++)
      a[i*m + j] = a[i*m + (j-1)] + b[j];
}

int main() {
  const int n = 1024;
  const int m = 2048;
  double *a   = (double *) calloc(n*m, sizeof(double));
  double *b   = (double *) calloc(m, sizeof(double));

  for (int i = 0; i < 1000; i++)
    kernel_unroll(a, b, n, m);
}
```

Question: Why not unroll loop over j?

Loop Carry Dependence



- **Loop carry dependence** = Next loop iteration depends on the previous loop iteration
  - Also known as **loop-carried dependency**
- **Example**:

```
1 double a[N];
2 double s = 0.0;
3 for (int i = 0; i < N; i++) {
4     s = s + a[i];
5 }
```

Loop Unroll and Jam



- **Loop unroll and jam** = An optimization technique where an outer loop is unrolled, then (multiple) inner loops are jammed (fused)

mat_accum: Implementation with Unroll and Jam



```c
#include <stdlib.h>

void kernel_unroll_jam( double *restrict a, double *restrict b, int n, int m) {
  for (int i = 0; i < n; i += 2)
    for (int j = 1; j < m; j++) {
      a[(i+0)*m + j] = a[(i+0)*m + (j-1)] + b[j];
      a[(i+1)*m + j] = a[(i+1)*m + (j-1)] + b[j];
    }
}

int main() {
  const int n = 1024;
  const int m = 2048;
  double *a   = (double *) calloc(n*m, sizeof(double));
  double *b   = (double *) calloc(m, sizeof(double));

  for (int i = 0; i < 1000; i++)
    kernel_unroll_jam(a, b, n, m);
}
```

`mat_accum`: Kunpeng 920 Results

	N_{cycle}	N_{instr}
Original, `-O1`	$2.92 \cdot 10^{10}$	$1.47 \cdot 10^{10}$
Unroll, `-O1`	$2.93 \cdot 10^{10}$	$1.47 \cdot 10^{10}$
Unroll+jam, `-O1`	$1.47 \cdot 10^{10}$	$1.05 \cdot 10^{10}$
Original, `-O3`	$2.53 \cdot 10^{5}$	$1.55 \cdot 10^{5}$
Unroll, `-O3`	$2.21 \cdot 10^{5}$	$1.55 \cdot 10^{5}$
Unroll+jam, `-O3`	$2.12 \cdot 10^{5}$	$1.55 \cdot 10^{5}$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - Using compiler optimization level `-O1`: Out loop unroll does not help, jamming is important
  - Using compiler optimization level `-O3`: Explicit optimization can still be helpful

Case Study: Squared Sums



- **Purpose** of this case study: Explore a strategy to unroll loops with loop-carry-dependences
- **Specification** for `vreduce2`:
  - Implement the following task:

$$s \leftarrow \sum_{i=0}^{N-1} a_i^2$$

  - The variables are double-precision numbers

Squared Sums: Initial Implementation



```c
#include <stdlib.h>

double sqsum(double *restrict a, int n) {
  double s = 0.0;
  for (int i = 0; i < n; i++)
    s += a[i] * a[i];
  return s;
}

double sum;

int main() {
  int n = 8192;
  double *a = (double *)calloc(n, sizeof(double));
  for (int i = 0; i < 10000; i++)
    sum = sqsum(a, n);
}
```

Loop Unroll with Dependency Break



- **Loop unroll with dependency break**: Optimization technique for unrolling loops with loop-carry-dependences
- Option to break dependence: **Partial reductions**

Squared Sums: Unrolled Implementation



```c
#include <stdlib.h>

double sqsum_unrolled(double *restrict a, int n) {
  double s0 = 0.0;
  double s1 = 0.0;
  double s2 = 0.0;
  double s3 = 0.0;
  for (int i = 0; i < n; i += 4) {
    s0 += a[i+0] * a[i+0];
    s1 += a[i+1] * a[i+1];
    s2 += a[i+2] * a[i+2];
    s3 += a[i+3] * a[i+3];
  }
  return (s0 + s1 + s2 + s3);
}
```

The remaining code remains unchanged

Squared Sums: Kunpeng 920 Results



	N_{cycle}	N_{instr}
Original, `-O1`	$3.28 \cdot 10^8$	$4.10 \cdot 10^8$
Unrolled, `-O1`	$1.36 \cdot 10^8$	$2.67 \cdot 10^8$
Original, `-O3`	$4.09 \cdot 10^8$	$3.28 \cdot 10^8$
Unrolled, `-O3`	$1.03 \cdot 10^8$	$1.03 \cdot 10^8$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - Using compiler optimization level `-O1`: Number of cycles and instructions improve by $2.4\times$ and $1.5\times$
  - Using compiler optimization level `-O3`: Compiler is not able to reach similar performance without code modifications

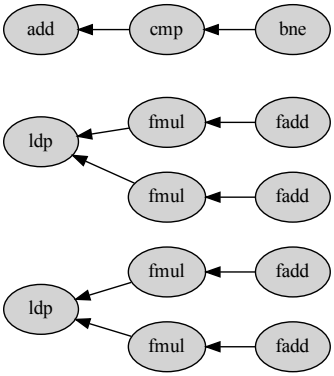
Squared Sums: Unrolled Implementation Assembler



```
8   for (int i = 0; i < n; i += 4) {
9     s0 += a[i+0] * a[i+0];
10    s1 += a[i+1] * a[i+1];
11    s2 += a[i+2] * a[i+2];
12    s3 += a[i+3] * a[i+3];
13  }
```

```
21  .L3:
22          ldp     d30, d31, [x2]
23          fmul    d30, d30, d30
24          fadd    d29, d29, d30
25          fmul    d31, d31, d31
26          fadd    d27, d27, d31
27          ldp     d30, d31, [x2, 16]
28          fmul    d30, d30, d30
29          fadd    d28, d28, d30
30          fmul    d31, d31, d31
31          fadd    d0, d0, d31
32          add     x2, x2, 32
33          cmp     x2, x0
34          bne     .L3
```

Squared Sums: Instruction Dependencies




```
21 .L3:
22         ldp     d30, d31, [x2]
23         fmul    d30, d30, d30
24         fadd    d29, d29, d30
25         fmul    d31, d31, d31
26         fadd    d27, d27, d31
27         ldp     d30, d31, [x2, 16]
28         fmul    d30, d30, d30
29         fadd    d28, d28, d30
30         fmul    d31, d31, d31
31         fadd    d0, d0, d31
32         add     x2, x2, 32
33         cmp     x2, x0
34         bne     .L3
```

Critical Path Analysis



- **Critical path** = Longest sequence of dependent instructions that must be executed sequentially to execute a task
- The critical path determines the **theoretical minimum execution time** for that task
- To compute the critical path the edges between dependent instructions are **weighted by instruction latency**

Case Study: `vadd_condzero`



- **Purpose** of this case study: Explore impact of conditions within loops
- **Specification** for `vadd_condzero`:
  - Implement the following task:

$$
\begin{aligned}
a_i &\leftarrow a_i + b_i \\
b_i &\leftarrow 0 \quad \text{if} \quad c > 0.4
\end{aligned}
$$

  - The variables a_i, b_i, and c are double-precision numbers

vadd_condzero: Initial Implementation



```c
#include <stdlib.h>

void kernel(double *restrict a, double *restrict b, double c, int n) {
  for (int j = 0; j < n; j++) {
    a[j] += b[j];
    if (c > 0.4) b[j] = 0;
  }
}

int main() {
  const int n = 16384;
  double *a   = (double *) calloc(n, sizeof(double));
  double *b   = (double *) calloc(n, sizeof(double));

  for (int i = 0; i < 100000; i++) {
    double c = rand() / (double) RAND_MAX;
    kernel(a, b, c, n);
  }
}
```

Loop Unswitching



- **Loop unswitching** = Optimization technique that moves conditional statements outside of loops when the condition does not change during loop execution
- **Target benefit(s)**
  - Performance benefits (at least for large loops)
  - Enable further optimizations within larger basic blocks
- **Possible drawback(s)**
  - Code duplication
  - Reduced code readability

vadd_condzero: Loop Unswitching



```c
#include <stdlib.h>

void kernel(double *restrict a, double *restrict b, double c, int n) {
  if (c > 0.4)
    for (int j = 0; j < n; j++) {
      a[j] += b[j];
      b[j] = 0;
    }
  else
    for (int j = 0; j < n; j++)
      a[j] += b[j];
}

int main() {
  const int n = 16384;
  double *a   = (double *) calloc(n, sizeof(double));
  double *b   = (double *) calloc(n, sizeof(double));

  for (int i = 0; i < 100000; i++) {
    double c = rand() / (double) RAND_MAX;
    kernel(a, b, c, n);
  }
}
```

`vadd_condzero`: Kunpeng 920 Results



	N_{cycle}	N_{instr}
Original, `-O1`	$5.4 \cdot 10^9$	$17.4 \cdot 10^9$
Unswitched, `-O1`	$3.3 \cdot 10^9$	$6.2 \cdot 10^9$
Original, `-O3`	$4.1 \cdot 10^9$	$12.5 \cdot 10^9$
Unswitched, `-O3`	$2.3 \cdot 10^9$	$6.2 \cdot 10^9$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - Using compiler optimization level `-O1`: Number of cycles and instructions improve by $1.6\times$ and $2.8\times$
  - Using compiler optimization level `-O3`: Compiler is not able to reach similar performance, but explicit optimization still helps ($1.4\times$ faster)

Content



1. Loop Optimizations (2)

2. SIMD and Vector Instructions

3. Auto-Vectorization (1)

4. Final

SIMD and Vector Instructions: Introduction



- **Key idea**: Use a single instructions to perform multiple operations
- **SIMD** = Single Instruction Multiple Data
  - Part of Flynn's taxonomy
- SIMD and vector instructions aim on leveraging data parallelism
  - Early focus area: image processing involving processing of many independent pixels
- Example: vector add with 4 **lanes**

$$\begin{pmatrix} z_0 \\ z_1 \\ z_2 \\ z_3 \end{pmatrix} \leftarrow \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{pmatrix}$$

SIMD versus Vector Instructions



- **SIMD instructions** read input operands and write results in single clock cycle
  - Operand width is limited by hardware constraints
    - Register size
    - Bus update rate
  - Currently, vendors support SIMD instructions with an operand width of **up to** $512\,\text{bit}$
- **Vector instructions** repeat instruction execution multiple times until full vector is processed
  - Vectors can be much larger compared to SIMD operands
  - Vector instructions may process multiple data items per clock cycle

Masking



- **Problem**: What happens if data is not a multiple of the SIMD operand width or vector length?
- **Solution 1**: Scalar **tail loop**
  - Process bulk using SIMD or vector instructions
  - Process remainder using scalar instructions
- **Solution 2**: **Masking**
  - Define a mask where the size is equal to the number of lanes
  - If a lane is masked the result is not written

Gather Load and Scatter Store



- **Gather load** = Load multiple data items from scattered memory locations into a consecutive range of addresses (e.g., a SIMD register)

```
1 for (int i = 0; i < N; i++)
2     x[i] = y[idx[i]];
```

- **Scatter store** = Store multiple data items from a consecutive range of addresses (e.g., a SIMD register) to scattered memory locations

```
1 for (i = 0; i < N; i++)
2     y[idx[i]] = x[i];
```

- Variant: **Structured load** and **structured store** instructions
  - These instructions load or store multiple data items with a fixed stride

SIMD/Vector ISAs



- **x86**
  - Streaming SIMD Extensions (SSE), 128-bit
  - Advanced Vector Extensions (AVX, AVX2), 256-bit
  - AVX-512, 512-bit
- **Arm**
  - NEON, 128-bit
  - Scalable Vector Extension (SVE), 128 to 2048-bit
- **Power**
  - VMX/AltiVec, 128-bit
  - VSX, 128-bit
- **RISC-V**
  - RISC-V (vector ISA)

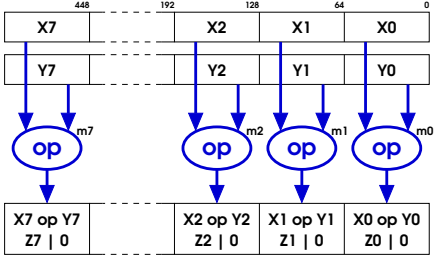
AVX-512 (1/2)

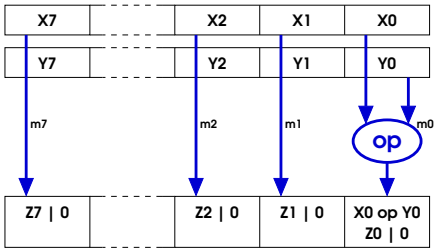


- **AVX-512** refers to a **familiy of instructions**
  - Most recent: Intel Advanced Vector Extensions 10.2 [Intel; 2025]
  - Most widely used encoding: EVEX
  - On Linux see `/proc/cpuinfo` to identify supported ISA
- Number of registers (assuming EVEX):
  - Vector registers: 32 (`ZMM0`, ..., `ZMM31`)
  - Mask registers: 8
- Number of lanes: Depends on data type
  - `double`/64-bit integers: 8
  - `float`/32-bit integers: 16
  - FP16: 32

AVX-512 (1/2)

- AVX-512 supports multi-lane (P=packed) and single-lane (S=scalar) processing





AVX512: Load and Store

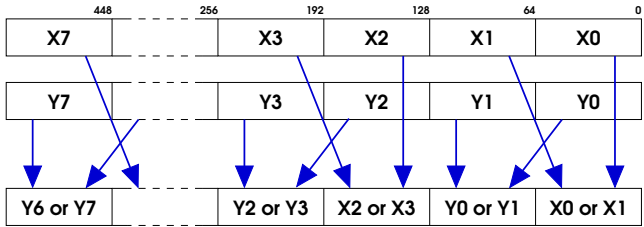


- Relevance of load and store instructions
  - Not all AVX-512 instruction operands can be memory operands
  - Need for gather load and scatter store
- Example instructions:

`VMOVQ xmm1, xmm2/m64`	Move quadword from xmm2/m64 to xmm1
`VMOVQ xmm1/m64, xmm2`	Move quadword from xmm2 to xmm1/m64
`VGATHERQPD zmm1 {k1}, vm64z`	Gather quadword vector
`VSCATTERQPD vm64z {k1}, zmm1`	Scatter quadword vector

AVX512: Shuffle

- `VSHUFPD zmm1{k1}{z}, zmm2, zmm3/m512, imm8`
  Re-order double precision floats depending on immediate value



- Immediate value selects higher (1) or lower (0) part
  - Example: `imm8 = 0x55 = b'01 01 01 01` will select (`Y6,X7,Y4,X5,Y2,X3,Y0,X1`)
- Use case: Swap two elements representing real and imaginary part of double precision complex number

AVX512: Floating-Point Instructions



`VADDPD zmm1 {k1}{z}, zmm2, zmm3/m512/m64bcst{er}`	Add packed double precision floating-point, store result in `zmm1` (writemask `k1`)
`VMULPD zmm1 {k1}{z}, zmm2, zmm3/m512/m64bcst{er}`	Multiply packed double precision floating-point, store result in `zmm1`
`VFMADD132PD zmm1 {k1}{z}, zmm2, zmm3/m512/m64bcst{er}`	Multiply packed double precision floating-point from `zmm1` and `zmm3/m512/m64bscst`, add `zmm2`, store result in `zmm1`

Scalable Vector Extension (SVE/SVE2)



- SVE was introduced as an extension of Arm-v8
- Special feature: **Vector Length Agnostic (VLA)**
  - SIMD operand width can be $128, 256, \ldots, 2048$ bit
  - Compiler does not have to now SIMD operand width at compile time
- Number of registers
  - Vector registers: 32 (`Z0`, ..., `Z31`)
  - Mask/predicate registers: 16 (`P0`, ..., `P15`)

SVE: Load and Store



- Recall: Arm is a load-store ISA
- Example instructions that also support gather/scatter + structured load/store:
  - `LD1D`: Contiguous load doublewords to vector
  - `ST1D`: Contiguous store doublewords from vector
  - `LD2D`: Contiguous load two-doubleword structures to two vectors
  - `ST2D`: Contiguous store two-doubleword structures from two vectors

SVE: Floating-Point Instructions



- `FADD`: Floating-point add vector
- `FADDA`: Horizontal floating-point strictly-ordered reduction (accumulating scalar)
- `FMUL`: Floating-point multiply vectors
- `FMAD`: Floating-point fused multiply-add vectors (predicated), writing multiplicand
- `FCMLA`: Floating-point complex multiply-add with rotate

All instructions support masking/predication

RISC-V “V” Vector Extension (RVV)



- Vector ISA extension with vectors up to $16\,\mathrm{k}$ bit
- RVV defines 32 vector registers of size `VLEN` bits
  - `VLEN` must be a power of 2
- Vectors in RVV are divided in elements
  - The size of elements in bits is at least 8 bits up to `ELEN` bits
- Support of masking
- Today still a few CPUs available supporting RVV
  - Example: SiFive X288 core with vector length up to 4096 bit

Flop per Cycle and Core



- Assuming support of fused multiply-add instructions:

$$B_{\text{fp}} = N_{\text{core}} \cdot N_{\text{SIMD}} \cdot N_{\text{lane}} \cdot 2 \ \frac{\text{Flop}}{\text{cycle}}$$

N_{core}: Number of cores
N_{SIMD}: Number of SIMD pipelines
N_{lane}: Number of SIMD lanes

- Examples for double-precision fused multiply-add instructions:
  - **Intel Golden Cove**: $B_{\text{fp}} = N_{\text{core}} \cdot 2 \cdot 8 \cdot 2 = 32 \cdot N_{\text{core}} \ \frac{\text{Flop}}{\text{cycle}}$
  - **Kunpeng 920**: $B_{\text{fp}} = N_{\text{core}} \cdot 1 \cdot 2 \cdot 2 = 4 \cdot N_{\text{core}} \ \frac{\text{Flop}}{\text{cycle}}$

Content



1. Loop Optimizations (2)

2. SIMD and Vector Instructions

3. **Auto-Vectorization (1)**

4. Final

Compiler Auto-Vectorization



- **Auto-vectorization** = Compiler technique to convert a **scalar implementation** into a **vector implementation**
- Approaches:
  - **Loop-based Vectorization**: Group scalar instructions after loop unrolling
  - **Superword-Level Parallelism (SLP)**: Group similar independent operations together
- Auto-vectorization is performed using intermediate representations
  - GCC: GIMPLE IR

SLP Auto-Vectorization Steps



[Ira Rosen et al.; 2007]

- **Loop unrolling**: Unroll loops
- **Data dependence analysis**: Look for instructions that are not data dependent
- **Clustering**: Identify groups of accesses to adjacent memory addresses to be packed in a vector load
  - Possible use of gather load
  - Note the key importance of data layout
- **Search** for computation chains that can be vectorized
- **Decide** whether to apply vectorization

Auto-Vectorization: Compiler Options



- **GCC**:
  - Enable: `-O3`
  - Reports: `-fopt-info-all` or `-fopt-info-vec-missed`
  - More verbose reports: `-ftree-vectorizer-verbose=3`
- **clang**:
  - Enable: `-O3`
  - Reports:
    - `-Rpass=loop-vectorize`: Identify loops that were successfully vectorized
    - `-Rpass-missed=loop-vectorize`: Identify loops that failed vectorization and indicates if vectorization was specified
    - `-Rpass-analysis=loop-vectorize`: Identify the statements that caused vectorization to fail
  - Tune: `-force-vector-width=<n>`

Memory Alignment (1/2)



- **Memory alignment** = Choice of memory addresses that are multiples of a specific value
  - **Aligned memory address** = memory address that is divisible by a certain value
- Depending on the CPU architecture
  - Alignment is required
  - Special load/store instructions have to be used
  - Performance penalties occur
- Function to test alignment:

```c
#include <stddef.h>
#include <stdint.h>

int is_aligned(const void* ptr, size_t alignment) {
    return ((uintptr_t)ptr % alignment) == 0;
}
```

Memory Alignment (2/2)



- POSIX-compliant aligned memory allocation: `posix_memalign()`

```c
#include <stdlib.h>

double *a;

int main() {
  posix_memalign((void **)&a, 64, 512 * sizeof(double));
  return 0;
}
```

- **Challenge**: How to propagate alignment information?
  - Possible GCC-specific solution: `__builtin_assume_aligned()`

```c
void vmul(double *restrict c, double *restrict a, double *restrict b, int n)
{
  a = __builtin_assume_aligned(a, 64);
  b = __builtin_assume_aligned(b, 64);
  c = __builtin_assume_aligned(c, 64);

  for (int i = 0; i < n; i++) c[i] = a[i] * b[i];
}
```

GCC Pragma `ivdep`



- **Challenge**: The compiler may not be able to determine absence of loop carry dependencies
- **GCC-specific solution**: Use the pragma `ivdep`
  - Programmer asserts that there are no loop carry dependencies
- **Example**:

```
1 void kernel(double *a, double c, int k, int n)
2 {
3 #pragma GCC ivdep
4   for (int i = 0; i < n; i++)
5     a[i] = a[i + k] * c;
6 }
```

  - With this pragma, the programmer asserts that $k \geq 0$

Case Study: Scaling a Vector (`vscale`)



- **Purpose** of this case study: Simple test case for auto-vectorization
- **Specification**: Implement a function that implements

$$\vec{y} \leftarrow \alpha \cdot \vec{x}$$

  using double-precision floating-point numbers

vscale: Scalar Implementation



Benchmark routine:

```
1 void vscale(double *restrict y,
2              double *restrict x,
3              double a, int n) {
4   for (int i = 0; i < n; i++)
5     y[i] = a * x[i];
6 }
```

Benchmark driver:

```
1 #include <stdlib.h>
2
3 void vscale(double *restrict,
4              double *restrict,
5              double, int);
6
7 int main() {
8   int n     = 16384;
9   double *x = (double *) calloc(n, sizeof(double));
10  double *y = (double *) calloc(n, sizeof(double));
11  double a  = 1.0;
12  for (int k = 0; k < 2000000; k++)
13    vscale(y, x, a, n);
14 }
```

vscale: Auto-Vectorized



Compilation using `gcc -O3 -march=native`

Benchmark routine:

```
1 void vscale(double *restrict y,
2              double *restrict x,
3              double a, int n) {
4   for (int i = 0; i < n; i++)
5     y[i] = a * x[i];
6 }
```

Assembler:

```
6  vscale:
7  .LFB0:
8          .cfi_startproc
9          endbr64
10         movq    %rdi, %rcx
11         testl   %edx, %edx
12         jle     .L17
13         leal    -1(%rdx), %eax
14         cmpl    $2, %eax
15         jbe     .L8
16         movl    %edx, %edi
17         vbroadcastsd    %xmm0, %ymm2
18         xorl    %eax, %eax
19         shrl    $2, %edi
20         salq    $5, %rdi
21         .p2align 5
22         .p2align 4,,10
23         .p2align 3
24 .L4:
25         vmulpd  (%rsi,%rax), %ymm2, %ymm1
26         vmovupd %ymm1, (%rcx,%rax)
27         addq    $32, %rax
28         cmpq    %rdi, %rax
29         jne     .L4
```

`vscale`: Results on Intel i7-1165G7



	N_{cycle}	N_{instr}
`-O1`	$4.9 \cdot 10^{10}$	$19.7 \cdot 10^{10}$
`-O3`	$2.8 \cdot 10^{10}$	$9.8 \cdot 10^{10}$
`-O3 -march=native`	$2.2 \cdot 10^{10}$	$4.1 \cdot 10^{10}$

- **Compiler**: GCC 14.2.0
- **Observations**:
  - Latency and number of instructions reduces by about $2.2\times$ and 4.8, respectively, comparing `-O1` and `-O3 -march=native`
- **Crosschecks**:
  - Benchmark is repeated $2 \cdot 10^6\times$ using $N = 16384$ ☞ 0.7 cycles per array element ✔

Content



1. Loop Optimizations (2)

2. SIMD and Vector Instructions

3. Auto-Vectorization (1)

4. Final

Summary Lecture #08



- **Loop optimizations**:
  - Loop unroll, loop unroll and jam, loop unswitching
  - Loop-carry-dependencies and a strategy to break such dependencies
  - Instruction dependency graphs and critical path analysis
- **SIMD and vector instructions**: AVX-512
- Extended overview of **SIMD and vector instructions**: SVE/SVE2, RVV
- **Auto-vectorization**:
  - Compiler auto-vectorization algorithms, options, and reports
  - Possible issues: memory alignment, loop carry dependence
  - Case study: vector scale

Questions?



[Wiki Media]



university of groningen

Performance Engineering: Lecture #09

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-28

Overview



1. Auto-Vectorization (2)
2. SIMD Programming Models
3. Branch Optimizations
4. Memory Architecture (1)
5. Final

Content



1. **Auto-Vectorization (2)**
2. SIMD Programming Models
3. Branch Optimizations
4. Memory Architecture (1)
5. Final

Case Study: Matrix-Vector Multiplication (`matmulv`)



- **Purpose** of this case study: Vectorization of loops with loop carry dependencies
- **Specification**: Implement multiplication of a matrix and a vector:

$$\vec{y} \leftarrow A\,\vec{x}$$

  - A is a square matrix of size $N = 2048$
  - Matrix and vector elements are double-precision numbers

matmulv: Scalar Implementation



Benchmark routine:

```
1  void matmulv(double *restrict y,
2               double *restrict a,
3               double *restrict x,
4               const int n) {
5    for (int i = 0; i < n; i++) {
6      y[i] = 0.0;
7      for (int j = 0; j < n; j++)
8        y[i] += a[i * n + j] * x[j];
9    }
10 }
```

Benchmark driver:

```
1  #include <stdlib.h>
2
3  void matmulv(double *restrict,
4               double *restrict,
5               double *restrict,
6               const int);
7
8  int main() {
9    int n    = 2048;
10   double *a = (double *) calloc(n*n, sizeof(double));
11   double *x = (double *) calloc(n,   sizeof(double));
12   double *y = (double *) calloc(n,   sizeof(double));
13   for (int k = 0; k < 2000; k++)
14     matmulv(y, a, x, n);
15 }
```

`matmulv`: Vectorized



- Manual unrolling by $4\times$
- Compilation using
  `gcc -O3 -march=native`

Benchmark routine:

```
1  void matmulv(double *restrict y,
2               double *restrict a,
3               double *restrict x,
4               const int n) {
5    for (int i = 0; i < n; i++) {
6      double s[4] = {0.0, 0.0, 0.0, 0.0};
7      for (int j = 0; j < n; j += 4) {
8        s[0] += a[i * n + j+0] * x[j+0];
9        s[1] += a[i * n + j+1] * x[j+1];
10       s[2] += a[i * n + j+2] * x[j+2];
11       s[3] += a[i * n + j+3] * x[j+3];
12     }
13     y[i] = s[0] + s[1] + s[2] + s[3];
14   }
15 }
```

Assembler:

```
21 .L3:
22          xorl    %eax, %eax
23          xorl    %edx, %edx
24          vxorpd  %xmm0, %xmm0, %xmm0
25          .p2align 5
26          .p2align 4,,10
27          .p2align 3
28 .L4:
29          vmovupd (%r8,%rax), %ymm3
30          incl    %edx
31          vfmadd231pd     (%rsi,%rax), %ymm3, %ymm0
32          addq    $32, %rax
33          cmpl    %ecx, %edx
34          jb      .L4
35          vextractf64x2   $1, %ymm0, %xmm2
36          vunpckhpd       %xmm0, %xmm0, %xmm1
37          addq    $8, %rdi
38          addq    %r9, %rsi
39          vaddsd  %xmm0, %xmm1, %xmm1
40          valignq $3, %ymm0, %ymm0, %ymm0
41          vaddsd  %xmm2, %xmm1, %xmm1
42          vaddsd  %xmm0, %xmm1, %xmm0
43          vmovsd  %xmm0, -8(%rdi)
44          cmpq    %rdi, %r10
45          jne     .L3
```

`matmulv`: Vectorized with Outer-Loop Unrolling



Benchmark routine:

```
1  void matmulv(double *restrict y,
2               double *restrict a,
3               double *restrict x,
4               const int n) {
5    for (int i = 0; i < n; i += 4) {
6      y[i + 0] = y[i + 1] = y[i + 2] = y[i + 3] = 0.0;
7      for (int j = 0; j < n; j++) {
8        y[i+0] += a[(i+0) * n + j] * x[j];
9        y[i+1] += a[(i+1) * n + j] * x[j];
10       y[i+2] += a[(i+2) * n + j] * x[j];
11       y[i+3] += a[(i+3) * n + j] * x[j];
12     }
13   }
14 }
```

- **Issue**: Requires gather load for A

`matmulv`: Results on Intel i7-1165G7

	N_{cycle}
`-O1`	$33.3 \cdot 10^9$
`-O3`	$33.2 \cdot 10^9$
`-O3 -march=native`	$32.9 \cdot 10^9$
unroll4, `-O1`	$9.0 \cdot 10^9$
unroll4, `-O3`	$8.4 \cdot 10^9$
unroll4, `-O3 -march=native`	$8.1 \cdot 10^9$
unroll8, `-O3 -march=native`	$4.2 \cdot 10^9$
unroll4-outer, `-O3 -march=native`	$10.7 \cdot 10^9$

- **Compiler**: GCC 14.2.0
- **Observation**: Latency reduces by up to $7.9\times$
- **Cross-check**: Best performance at 4 Flop/cycle ✔

Mathematical Functions



- **Challenge**: Standards for languages like C foresee a scalar API for the mathematical functions and, therefore, make it difficult to exploit SIMD/vector instructions
  - Example:

$$\vec{y} \leftarrow \sin(\vec{x})$$

- **Possible solutions**:
  - Use programming models supporting mathematical functions, e.g. nsimd, Highway
  - Rely on built-in implementations provided by the compiler

Case Study: `sinf`



- **Purpose** of this case study: Check the impact of vectorizing mathematical functions
- **Specification**: Compute the sin function of an array of single-precision floating-point numbers

sinf: Code



Benchmark routine:

```c
#include <math.h>

void vsinf(float *restrict y,
           float *restrict x,
           int n)
{
  for (int i = 0; i < n; i++)
    y[i] = sinf(x[i]);
}
```

Benchmark driver:

```c
#include <stdlib.h>

void vsinf(float *restrict,
           float *restrict,
           int);

int main() {
  int n    = 16384;
  float *x = (float *) calloc(n, sizeof(float));
  float *y = (float *) calloc(n, sizeof(float));

  for (int k = 0; k < 100000; k++)
    vsinf(y, x, n);
}
```

`sinf`: Auto-Vectorized



Compilation using `gcc -O3 -ffast-math`

Benchmark routine:

```
1 #include <math.h>
2
3 void vsinf(float *restrict y,
4            float *restrict x,
5            int n)
6 {
7   for (int i = 0; i < n; i++)
8     y[i] = sinf(x[i]);
9 }
```

Assembler:

```
46 .L4:
47         movups  (%rbx), %xmm0
48         addq    $16, %rbx
49         addq    $16, %rbp
50         call    _ZGVbN4v_sinf@PLT
51         movups  %xmm0, -16(%rbp)
52         cmpq    %r12, %rbx
53         jne     .L4
```

`sinf`: Results on Intel i7-1165G7



	N_{cycle}	N_{instr}
`-O1`	$1.5 \cdot 10^{10}$	$3.9 \cdot 10^{10}$
`-O3`	$1.7 \cdot 10^{10}$	$3.9 \cdot 10^{10}$
`-O3 -ffast-math`	$1.0 \cdot 10^{10}$	$2.5 \cdot 10^{10}$

- **Compiler**: GCC 14.2.0
- **Observations**:
  - Latency and number of instructions reduces by about $1.5\times$ comparing `-O1` and `-O3 -ffast-math`
- **Crosschecks**:
  - Benchmark is repeated $10^5\times$ using $N = 16384$ ☞ 36 cycles per call of `sinf` ✔

Content



1. Auto-Vectorization (2)
2. SIMD Programming Models
3. Branch Optimizations
4. Memory Architecture (1)
5. Final

SIMD Programming Models: Introduction



- **Alternative** to auto-vectorization: **Manual vectorization**
- **Options**:
  - In-line assembler (not recommended)
  - Intrinsics
  - SIMD programming models: NSIMD, Highway, ...

Compiler Intrinsics



- **Compiler intrinsics** = Special functions that look like regular functions but are directly implemented by the compiler
- **Target benefit(s)**:
  - Improved performance due to stronger control on code generation
- **Possible drawback(s)**:
  - Compiler- and/or ISA-specific ☞ non-portable code

Intrinsics for AVX-512



- Documentation: Intel Intrinsics Guide
- Selected new data types: `__m512d`
- Selected intrinsics:

`_mm512_loadu_pd`:	Load unaligned doubles
`_mm512_set1_pd`:	Broadcast doubles
`_mm512_mul_pd`:	Multiply packed doubles
`_mm512_storeu_pd`:	Store unaligned doubles

- GCC compilation may require the following options: `-mavx512f -mavx512dq -mfma`

Vector Scale: AVX-512 Intrinsics



Compilation using `gcc-14 -O3 -mavx512f`

Benchmark routine:

```
1  #include <immintrin.h>
2
3  void vscale_avx512(double *restrict y,
4                     const double *restrict x,
5                     double a, int n)
6  {
7    __m512d va = _mm512_set1_pd(a);
8
9    for (int i = 0; i < n; i += 8) {
10       __m512d vx = _mm512_loadu_pd(&x[i]);
11       __m512d vy = _mm512_mul_pd(va, vx);
12       _mm512_storeu_pd(&y[i], vy);
13   }
14 }
```

Assembler:

```
6  vscale_avx512:
7  .LFB6571:
8          .cfi_startproc
9          endbr64
10         vbroadcastsd    %xmm0, %zmm0
11         testl   %edx, %edx
12         jle     .L5
13         xorl    %eax, %eax
14         .p2align 5
15         .p2align 4
16         .p2align 3
17 .L3:
18         vmulpd  (%rsi,%rax,8), %zmm0, %zmm1
19         vmovupd %zmm1, (%rdi,%rax,8)
20         addq    $8, %rax
21         cmpl    %eax, %edx
22         jg      .L3
23 .L5:
24         vzeroupper
25         ret
26         .cfi_endproc
```

Intrinsics for NEON



- Documentation: Arm Neon Intrinsics Reference
- Selected new data types: `float64x2_t`
- Selected intrinsics:

  `vdupq_n_f64`: Duplicate vector element to vector
  `vld1q_f64`: Load multiple single-element structures
  `vmulq_f64`: Multiply packed doubles
  `vst1q_f64`: Store multiple single-element structures

Vector Scale: NEON Intrinsics



Compilation using gcc-14 -O3

Benchmark routine:

```
1  #include <arm_neon.h>
2
3  void vscale_neon(double *restrict y,
4                   const double *restrict x,
5                   double a, int n)
6  {
7    float64x2_t va = vdupq_n_f64(a);
8
9    for (int i = 0; i < n; i += 2) {
10       float64x2_t vx = vld1q_f64(&x[i]);
11       float64x2_t vy = vmulq_f64(va, vx);
12       vst1q_f64(&y[i], vy);
13   }
14 }
```

Assembler:

```
6           .global vscale_neon
7           .type   vscale_neon, %function
8  vscale_neon:
9  .LFB3916:
10          .cfi_startproc
11          dup     v0.2d, v0.d[0]
12          cmp     w2, 0
13          ble     .L1
14          sub     w3, w2, #1
15          mov     x2, 0
16          lsr     w3, w3, 1
17          add     w3, w3, 1
18          lsl     x3, x3, 4
19          .p2align 5,,15
20 .L3:
21          ldr     q31, [x1, x2]
22          fmul    v31.2d, v31.2d, v0.2d
23          str     q31, [x0, x2]
24          add     x2, x2, 16
25          cmp     x3, x2
26          bne     .L3
```

NSIMD



- NSIMD is a vectorization library for C and C++
- Documentation: github.io
- Large range of supported architectures ☞ portability
  - Intel AVX-512 for KNLs
  - Intel AVX-512 for Skylake processors
  - Arm NEON 128 bits (ARMv8 and later)
  - Arm SVE (original sizeless SVE)
  - ...
- Various vectorised fuctions
  - Trigonometric functions
  - Exponential and logarithmic functions
  - Shuffle functions
  - ...

Vector Scale: NSIMD



```cpp
#include <nsimd/nsimd-all.hpp>

using namespace nsimd;

void vscale_nsimd(double* y,
                  const double* x,
                  double a, int n)
{
  using vdouble = pack<double>;
  const auto size = len<vdouble>();
  vdouble va = set1<vdouble>(a);

  for (int i = 0; i < n; i += size) {
    vdouble vx = loadu<vdouble>(&x[i]);
    vdouble vy = mul(va, vx);
    storeu(&y[i], vy);
  }
}
```

Highway



- C++ library that provides portable SIMD/vector intrinsics
- Documentation: github.io
- Large range of supported architectures ☞ portability
  - `AVX10_2`
  - `NEON`
  - `RVV`
  - `...`

Vector Scale: Highway



```cpp
#include <hwy/highway.h>

namespace hw = hwy::HWY_NAMESPACE;

void vscale_highway(double* HWY_RESTRICT y,
                    const double* HWY_RESTRICT x,
                    double a, int n)
{
  const hw::ScalableTag<double> vdouble;
  const auto size = hw::Lanes(vdouble);
  auto va = hw::LoadU(vdouble, &a);

  for (int i = 0; i < n; i += size) {
    auto vx = hw::LoadU(vdouble, &x[i]);
    auto vy = hw::Mul(va, vx);
    hw::StoreU(vy, vdouble, &y[i]);
  }
}
```

Content



1. Auto-Vectorization (2)
2. SIMD Programming Models
3. Branch Optimizations
4. Memory Architecture (1)
5. Final

Introduction to Branch Optimization



- Approaches to branch optimization:
  - Avoid branching
  - Help branch predictor
  - Software support

Case Study: Table Lookup



- **Purpose** of this case study: Explore branching in the context of table lookups
- **Specification** for `table_lookup`:
  - Implement a benchmark for random table lookups based on an index
  - For simplicity consider the index k being an integer number, i.e. $k = 0, 1, \ldots$

Table Lookup: Implementation with Branches


```c
#include <math.h>
#include <stdlib.h>

double x;

int main() {
  for (int i = 0; i < 10000000; i++) {
    int k = rand() % 2;
    switch (k) {
      case 0: x = 2.0; break;
      case 1: x = 2.1; break;
    }
  }
}
```

```c
#include <math.h>
#include <stdlib.h>

double x;

int main() {
  for (int i = 0; i < 10000000; i++) {
    int k = rand() % 10;
    switch (k) {
      case 0: x = 2.0; break;
      case 1: x = 2.1; break;
      case 2: x = 2.2; break;
      case 3: x = 2.3; break;
      case 4: x = 2.4; break;
      case 5: x = 2.5; break;
      case 6: x = 2.6; break;
      case 7: x = 2.7; break;
      case 8: x = 2.8; break;
      case 9: x = 2.9; break;
    }
  }
}
```

Branch Optimization: Table Lookup



- **Branch optimization using table lookups** = Technique that replaces conditional branches (like long if-else or switch statements) with a lookup into an array or dictionary
- **Target benefit(s)**
  - Eliminate branches
- **Possible drawback(s)**
  - Requires storing the lookup table and performing table lookups
  - More data cache misses

Table Lookup: Implementation without Branches



```c
#include <math.h>
#include <stdlib.h>

double x;
const double table[] = { 2.0, 2.1 };

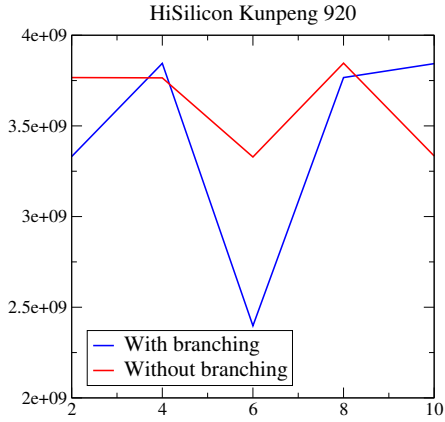
int main() {
  for (int i = 0; i < 10000000; i++) {
    int k = rand() % 2;
    x = table[k];
  }
}
```

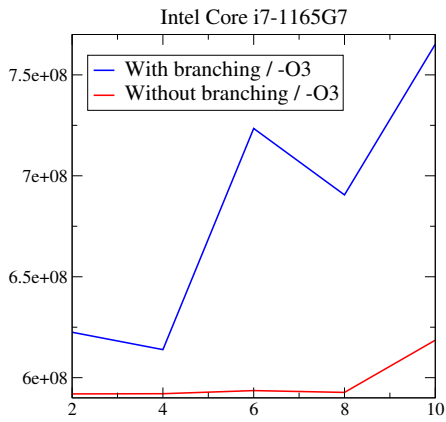
```c
#include <math.h>
#include <stdlib.h>

double x;
const double table[] =
    { 2.0, 2.1, 2.2, 2.3, 2.4,
      2.5, 2.6, 2.7, 2.8, 2.9 };

int main() {
  for (int i = 0; i < 10000000; i++) {
    int k = rand() % 10;
    x = table[k];
  }
}
```

Table Lookup: Results





Other Branch Optimizations



- **Help the branch predictor**, e.g., by
  - Keeping branch patterns stable
  - Reduce branch frequency and branching locations to improve use of Branch History Tables
- **Software support for branch prediction**
  - Branch prediction hints
    - Support has very much reduced
  - Arm v8 (aarch32) supports branch predictor maintenance
  - GCC supports identification of the most likely taken branch (`__builtin_expect()`)

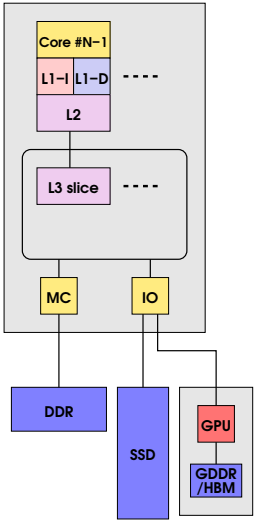
Content



1. Auto-Vectorization (2)
2. SIMD Programming Models
3. Branch Optimizations
4. **Memory Architecture (1)**
5. Final

Memory in Modern Node Architectures

- **Today's compute nodes: different memory types and technologies**
  - **Volatile memories**
    - Main memory (DDR3, DDR4, or DDR5)
    - Caches (SRAM)
    - Accelerator memory (GDDR or HBM)
  - **Non-volatile memories**
    - SSD (NAND flash)
- **Different capabilities**
  - Bandwidth
  - Latency for single transfer
    - Access time
    - Cycle time
  - Capacity



Memory Technologies: DRAM (1/2)



- Data bit storage cell $=$ 1 capacitor $+$ 1 transistor
- Capacitors discharge ☞ periodic **refresh** needed
  - Refresh required in intervals of O(10) ms
  - Possibly significant performance impact
  - Memory access latencies become variable
- DRAM typically packaged on small boards: **DIMM** $=$ Dual Inline Memory Modules
  - ≥ 1 DIMM per memory channel
  - Typical DRAM channel width: $N_{\mathrm{width}} = 64$ bit

Memory Technologies: DRAM (2/2)



- How to determine a theoretic upper bound for memory bandwidth:

$$B_{\text{mem}} = N_{\text{socket}} \cdot N_{\text{channel}} \cdot N_{\text{width}} \cdot B_{\text{rate}}$$

N_{socket}:	Number of sockets
N_{channel}:	Number of memory channels per socket
N_{width}:	Memory channel width
B_{rate}:	Data rate

- Performance range for selected memory technologies:

	DDR3	DDR4	DDR5
Standards release	2007	2012	2020
Data rate [Gbps]	$0.4 - 2.333$	$2.133 - 3.2$	$4 - 8.8$
Bandwidth/DIMM [GByte/s]	$3.2 - 18.6$	$17.1 - 25.6$	$32 - 70.4$

Memory Technologies: SRAM



- Data bit storage cell = (4 + 2) transistors
- Comparison with DRAM

	DRAM	SRAM
Refresh required?	yes	no
Speed	slower	faster
Density	higher	lower

- Typically used for on-chip memory

Data Locality (1/3)



- **Temporal locality** = There is a high probability for a memory location to be accessed again in the near future
  - Observe memory accesses during a time window W_{t}:

$$A_t, A_{t+1}, A_{t+2}, \ldots, A_{t+W_{\mathrm{t}}-1}$$

  - Let $\rho_{\mathrm{t}}(B)$ be the fraction of addresses with

$$A_{t+i} \equiv B$$

    for $B \in \{A_t, \ldots, A_{t+W_{\mathrm{t}}-1}\}$ and $0 \leq i < W_{\mathrm{t}}$
  - **Temporal locality** means that for some B we find $\rho_{\mathrm{t}}(B) \gg 1/W_{\mathrm{t}}$

Data Locality (2/3)



- **Spatial locality** = There is a high probability for a nearby memory location to be accessed in the near future
  - Observe memory accesses during a time window W_{t}:

$$A_t, A_{t+1}, A_{t+2}, \ldots, A_{t+W_{\mathrm{t}}-1}$$

  - Let $\rho_{\mathrm{s}}(\delta)$ be the fraction of pairs (A_{t+i}, A_{t+j}) $(0 \leq i < j < W_{\mathrm{t}})$ with

$$0 < |A_{t+i} - A_{t+j}| \leq \delta$$

    and $\delta > 0$
  - **Spatial locality** means that $\rho_{\mathrm{s}}(\delta)$ is large for small δ

Data Locality (3/3)



Question: Which type of locality applies to which pattern?

Time	Pattern #1	Pattern #2
0	$A + 0$	$A + 0$
1	$A + 1$	$A + 4096$
2	$A + 2$	$A + 0$
3	$A + 3$	$A + 4096$
4	$A + 4$	$A + 8192$
5	$A + 5$	$A + 0$
6	$A + 6$	$A + 4096$
7	$A + 7$	$A + 0$
8	$A + 8$	$A + 0$
9	$A + 9$	$A + 16384$

Data Locality (3/3)



Question: Which type of locality applies to which pattern?

- Pattern #1: Spatial locality
  $\rho_{\mathrm{t}}(A+0) = 1/10$
  $\rho_{\mathrm{s}}(10) = 1$
- Pattern #2: Temporal locality
  $\rho_{\mathrm{t}}(A+0) = 5/10$
  $\rho_{\mathrm{t}}(A+4096) = 3/10$
  $\rho_{\mathrm{s}}(10) = 0$

Time	Pattern #1	Pattern #2
0	$A+0$	$A+0$
1	$A+1$	$A+4096$
2	$A+2$	$A+0$
3	$A+3$	$A+4096$
4	$A+4$	$A+8192$
5	$A+5$	$A+0$
6	$A+6$	$A+4096$
7	$A+7$	$A+0$
8	$A+8$	$A+0$
9	$A+9$	$A+16384$

Reuse Distance



- **Reuse distance** = The number of accesses to distinct addresses made since the last reference to the same address
- **Example**:

Memory address	d	a	c	b	c	c	d	e	a	c
Reuse distance	-	-	-	-	1	0	3	-	4	3

- **Application** of the reuse distance:
  - Assume a cache of size C with Least Recently Used (LRU) replacement policy
  - If reuse distance $d \leq C$ then the address will be a cache hit

Memory and Storage Hierarchy



Computers provide multiple levels of volatile memory and non-volatile storage

- Important because of the "**processor–memory performance gap**"
- Close to the processor: very fast memory, but less capacity

Level	Access time	Capacity
Registers	$O(0.1)$ ns	$O(1)$ Ki B
L1 cache	$O(1)$ ns	$O(10-100)$ Ki B
L2 cache	$O(10)$ ns	$O(100)$ Ki B
L3 cache	$O(10)$ ns	$O(1-100)$ Mi B
Memory	$O(100)$ ns	$O(10-100)$ Gi B
Disk	$O(10)$ ms	$O(100-\ldots)$ Gi B

Content



1. Auto-Vectorization (2)
2. SIMD Programming Models
3. Branch Optimizations
4. Memory Architecture (1)
5. **Final**

Summary Lecture #09



- **Auto-vectorization**:
  - Case study: matrix-vector multiplication
  - Vectorization of mathematical functions
- **Programming models**: Intrinsics, NSIMD, Highway
- **Branch optimizations**: Table lookup and others
- Performance relevant features of memory architectures
  - Memory technologies and their performance
  - Data locality
  - Memory and storage hierarchy

Questions?



[Wiki Media]

These slides contain content and figures from

- D. Bakhvalov, Performance Analysis and Tuning on Modern CPUs (2nd Ed.). 2024. [Online]. Available: `https://github.com/dendibakh/perf-book/releases`



university of groningen

Performance Engineering: Lecture #10

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-30

Overview



1. Memory Architecture (2)
2. Memory Performance Tools
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. Final

Content



1. **Memory Architecture (2)**
2. Memory Performance Tools
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. Final

Cache



Cache miss = Word not found in cache

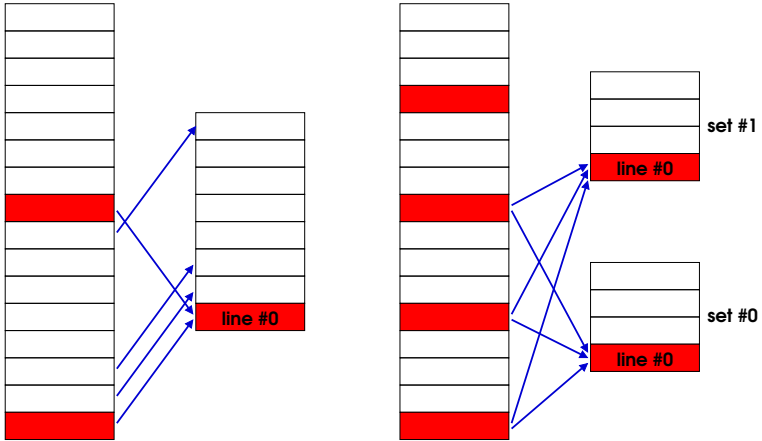
- Word has to be fetched from next memory level
- Typically a full **cache line** (=multiple words) is fetched

Cache organization: **Set associative cache**

- Match line onto a set and then place line within the set
  - Choice of set address: (address) mod (number of sets)
- Types:
  - **Direct-mapped cache**: 1 cache line per set
    - Each line has only one place it can appear in the cache
  - **Fully associative cache**: 1 set only
    - A line can be placed anywhere in the cache
  - **n-way set associative cache**: *n* cache lines per set
    - A line can be placed in a restricted set of places (1 out of *n* different places) in the cache

Cache Organization

Direct-mapped vs. 4-way set associative cache:



Cache Write Policies



Issue: How to keep cache and memory consistent?

- Cache write policy design options
  - **Write-through cache**
    - Update cache and memory
    - Memory write causes significant delay if pipeline must stall
  - **Write-back cache**
    - Update cache only and post-pone update of memory
      ☞ memory and cache will become inconsistent
    - If cache line is **dirty** then it must be committed before being replaced
    - Add dirty bit to manage this task

Issue: How to manage partial cache line updates?

- Typical solution: **write-allocate** (fetch-on-write)
  - Cache line is read before being modified

Digression: Stream Benchmark Performance



- Analyse impact of write-allocate mechanism assuming a cache line size of 64 byte and double-precision numbers
- **Scale benchmark** with arrays of length N:

$$\vec{b} \leftarrow s \cdot \vec{c}$$

  - Cache line loads: $(1+1) \cdot 8 \cdot N/64$
  - Cache line writes: $1 \cdot 8 \cdot N/64$
- **Triad benchmark**:

$$\vec{a} \leftarrow \vec{b} + s \cdot \vec{c}$$

  - Cache line loads: $(1+2) \cdot 8 \cdot N/64$
  - Cache line writes: $1 \cdot 8 \cdot N/64$
- Write-allocate may cause

$$b_{\text{triad}} \simeq \frac{9}{8} b_{\text{scale}}$$

Cache Misses



- Categories of cache misses

  Compulsory: Occurs when cache line is accessed the first time

  Capacity: Re-fetch of cache lines due to lack of cache capacity

  Conflict: Cache line was discharged due to lack of associativity

- **Average memory access time** $=$

$$\text{hit time} + \text{miss rate} \times \text{miss penalty}$$

Hit time ... Time to hit in the cache

Miss penalty ... Time to load cache line from memory

Inclusive versus Exclusive Caches

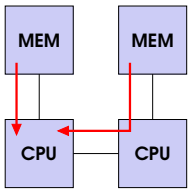


- Policies:
  - **Inclusive cache** = Data present in one cache level also present in higher cache level
    - Example: All data in L1 cache is also present in L2 cache
  - **Exclusive cache** = Data present in at most one cache level
  - Intermediate policies:
    - Data available at higher cache level is allowed to be available at lower cache levels
- Advantages of inclusive cache
  - Management of cache coherence may be simpler
- Advantages of exclusive cache
  - Duplicate copies of data in several cache levels avoided
- Example for intermediate policy:
  - L3 cache which is used as **victim cache**: cache holds data evicted from L2, but not necessarily data loaded to L2

Non-Uniform Memory Access



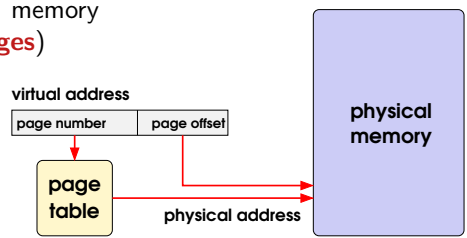
- **Non-Uniform Memory Access (NUMA)** = The data paths to the physical memory are non-uniform
- **Example**: Dual-socket node design with a shared memory space
  - A processor can load from another processor's memory



  - Memory bandwidth and access latency depend on which memory is accessed

Virtual Memory

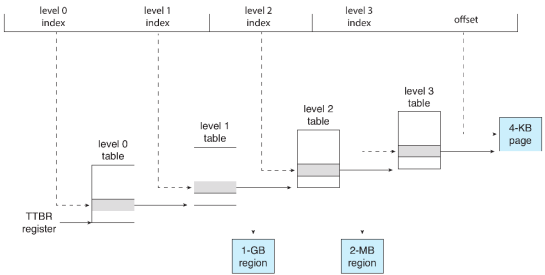
- **Virtual memory** = Memory management technique that allows each process to have its own contiguous memory address space
- Practical implementation:
  - The operating systems (supported by hardware) manages mapping between **physical memory addresses** and **virtual memory addresses**
  - To simplify this mapping the physical (virtual) memory is organised in **memory frames** (**memory pages**)
  - Memory mapping assuming equally-sized frames and pages:



Page Tables Look-up

- Hardware mechanisms that speed-up page table look-ups by
  - **Translation Lookaside Buffers (TLB)**
    - Small cache holding page table entries
  - **Hardware table walk** supporting multi-level page tables
    - Example: Arm-v8 4-Level Lookup

[Silberschatz; 2018]



Demand Paging



- **Demand paging** = Memory management technique that brings pages into memory when an executing process demands them
- Situations where demand paging will occur:
  - Virtual memory page exists but was swapped to storage
    - OS will select a memory frame, read content from storage, and create/update a page table entry
  - Virtual memory page has never been accessed
    - OS will create virtual memory page, assign a memory page, and create a page table entry
    - Note: This does not happen when virtual memory is allocated

    ☞ **First-touch policy**

Content



1. Memory Architecture (2)
2. **Memory Performance Tools**
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. Final

Memory Tools: Introduction



- Tools to discover parameters of the memory subsystem
  - `dmidecode`
  - `hwloc-ls`
- Tools to monitor performance
  - Perf (see tutorial #1)
  - Valgrind Memcheck
  - Valgrind Cachegrind

Discovery of the Memory Subsystem: SMBIOS



- Information about the available hardware is typically stored in the **System Management BIOS (SMBIOS)**
- Linux tool for reading the SMBIOS: `dmidecode`

Example:

```
# dmidecode 3.5
Getting SMBIOS data from sysfs.
SMBIOS 3.2.0 present.
Table at 0x9109F000.

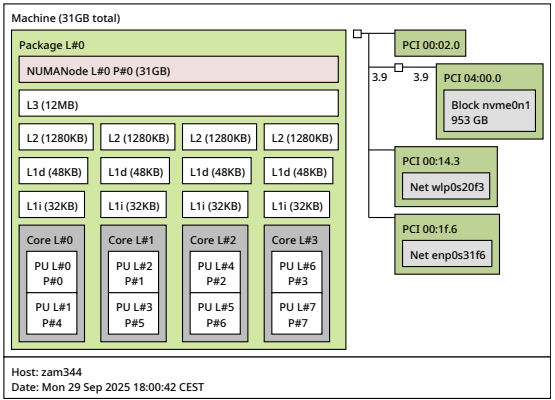
...

Handle 0x0001, DMI type 17, 92 bytes
Memory Device
        ...
        Total Width: 16 bits
        Data Width: 16 bits
        Size: 4 GB
        ...
        Bank Locator: BANK 0
        Type: LPDDR4
        ...
        Speed: 4267 MT/s
        ...
        Rank: 2
        Configured Memory Speed: 4267 MT/s
        ...
```

Discovery of the Memory Subsystem: hwloc

- **Portable Hardware Locality (hwloc)** was developed in the context of OpenMPI as a portable abstraction of the hierarchical topology of modern node architectures
- Showing the topology of the system: `hwloc-ls`

Example: Intel Core i7-1165G7



Valgrind Tools: Memcheck



- The default Valgrind Memcheck tool can be used to obtain the number of memory allocations:

```
==149464== Memcheck, a memory error detector
==149464== Copyright (C) 2002–2022, and GNU GPL'd, by Julian Seward et al.
==149464== Using Valgrind–3.22.0 and LibVEX; rerun with –h for copyright info
==149464== Command: ./a.out
==149464==
==149464==
==149464== HEAP SUMMARY:
==149464==     in use at exit: 0 bytes in 0 blocks
==149464==   total heap usage: 10 allocs, 10 frees, 245,760 bytes allocated
==149464==
==149464== All heap blocks were freed -- no leaks are possible
==149464==
==149464== For lists of detected and suppressed errors, rerun with: -s
==149464== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```c
#include <mcheck.h>
#include <stdlib.h>

double *a, *b;

int main() {
  for (int k = 1; k <= 5; k++) {
    const int m = sizeof(double);
    const int n = k * 1024;

    a = (double *) malloc(n * m);
    b = (double *) malloc(n * m);

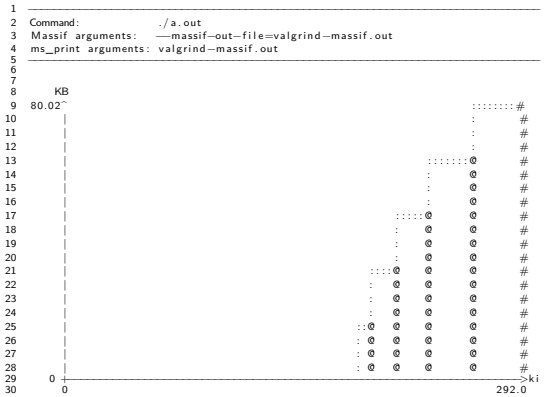
    for (int i = 0; i < n; i++)
      a[i] = b[i];

    free(b);
    free(a);
  }

  return 0;
}
```

Valgrind Tools: Massif

- Massif is a heap profiler
- Command line example: `valgrind --tool=massif ./a.out`
- Output parser: `ms_print`



Valgrind Tools: CacheGrind (1/2)



- Cachegrind is a tracing profiler including a cache simulator
- Command line example:

```
valgrind \
    --tool=cachegrind \
    --cache-sim=yes \
    --D1=65536,4,64 ./a.out
```

```
1  #define N 800000
2
3  double a[N], b[N];
4
5  int main()
6  {
7      for (int i = 0; i < N; i++)
8          a[i] = b[i];
9
10     return 0;
11 }
```

Valgrind Tools: CacheGrind (2/2)

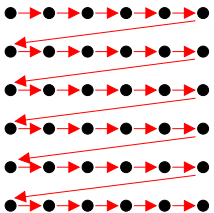
```
1  ==151340== Cachegrind, a high-precision tracing profiler
2  ==151340== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
3  ==151340== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
4  ==151340== Command: ./a.out
5  ==151340==
6  --151340-- warning: L3 cache found, using its data for the LL simulation.
7  ==151340==
8  ==151340== I refs:        4,178,640
9  ==151340== I1  misses:        1,131
10 ==151340== LLi misses:        1,123
11 ==151340== I1  miss rate:      0.03%
12 ==151340== LLi miss rate:      0.03%
13 ==151340==
14 ==151340== D refs:        1,651,120  (839,549 rd   + 811,571 wr)
15 ==151340== D1  misses:      201,495  (101,166 rd   + 100,329 wr)
16 ==151340== LLd misses:      201,421  (101,097 rd   + 100,324 wr)
17 ==151340== D1  miss rate:      12.2% (   12.1%     +    12.4%  )
18 ==151340== LLd miss rate:      12.2% (   12.0%     +    12.4%  )
19 ==151340==
20 ==151340== LL refs:          202,626  (102,297 rd   + 100,329 wr)
21 ==151340== LL misses:        202,544  (102,220 rd   + 100,324 wr)
22 ==151340== LL miss rate:         3.5% (    2.0%     +    12.4%  )
```

Content



1. Memory Architecture (2)
2. Memory Performance Tools
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. Final

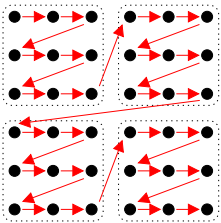
Cache Blocking



- **Cache blocking** is a strategy where data structures or memory access patterns are changed such that capacity cache misses are reduced
  - Optimization strategy sometimes also called **loop tiling** or **loop blocking**
- **Example**: Double-precision matrix-vector multiplication

$$y_i \leftarrow \sum_{j=0}^{N-1} A_{ij} x_j \quad (i = 0, \ldots, N-1)$$

  - Advantage: x needs to be loaded only once per block
  - Disadvantage: Non-linear read of A



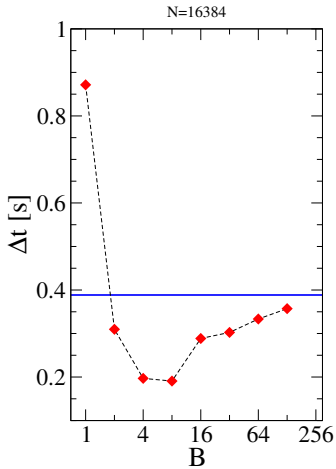
Cache-blocked Matrix-Vector Multiplication



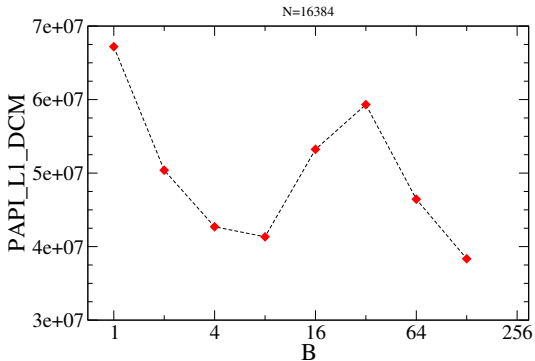
```
1  double x[N], y[N];
2  double A[N][N];
3
4  for (int ib = 0; ib < N/B; ib++) {
5    for (int i = 0; i < B; i++)
6      y[ib*B+i] = 0.0;
7
8    for (int jb = 0; jb < N/B; jb++)
9      for (int i = 0; i < B; i++)
10       for (int j = 0; j < B; j++) {
11         int ii = ib*B + i;
12         int jj = jb*B + j;
13         y[ii] += A[ii][jj] * x[jj];
14       }
15 }
```

Cache-blocked Matrix-Vector Multiplication

- Results obtained for $N = 16384$ on Intel Xeon E5-2623 v3 using a single thread
  - Horizontal line shows results for no cache blocking
- Observations
  - Large number of nested loops generate a significant overhead
  - Up to $2\times$ speed-up compared to no cache blocking
  - Best results are obtained for small block sizes with $B = 8$



Cache-blocked Matrix-Vector Multiplication



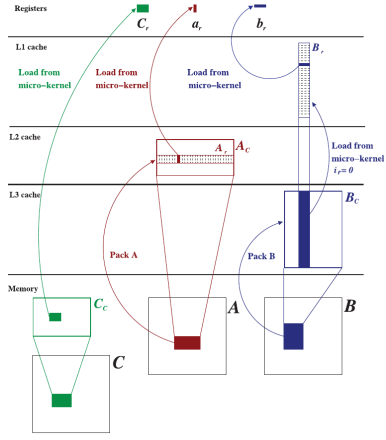
- For small B number of L1 data cache misses drop as expected
- The intermediate behaviour is possibly due to associativity

BLIS Design of gemm (BLAS-3)

- **gemm** = general matrix multiplication

$$C \leftarrow \alpha A B + \beta C$$

- BLIS = open source implementation of BLAS
- BLIS' gemm implementation is based on a systematic approach to multi-level blocking
  - Inner-most computations: hand-tuned micro-kernel
  - 5 outer loops
- Known analytic model for selecting implementation parameters based on hardware parameters
  - Cache: line size, associativity, size
  - SIMD instructions: throughput, width



[T.M. Low et al.]

Content



1. Memory Architecture (2)
2. Memory Performance Tools
3. Cache Blocking
4. **Dynamic Memory Allocation**
5. DTLB Optimization
6. Final

Dynamic Memory Allocation



- Possible issues related to dynamic memory allocation:
  - Increased number of page table updates
  - Lock contentions in case of multi-threading
  - Fragmentation
- Performance impact mitigation strategies
  - Minimize number of allocations/de-allocations
  - Use of optimised memory allocators, e.g. TCmalloc, jemalloc
  - Avoid small memory allocations

Alternative: TCmalloc



- TCMalloc is Google's customized implementation of `malloc()` provided by LIBC
- Benefits
  - Reduced lock contention
  - Improved thread scaling (depending on application)
- User-accessible controls
  - Logical page size: unit in which TCMalloc manages memory
  - Memory releasing
- Usage:
  `LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libtcmalloc.so ./myprog.x`

Alternative: jemalloc



- A replacement of `malloc()` that aims to reduce fragmentation and improve scalability
  - Became default in Facebook software and databases like Cassandra, MariaDB
- Benefits
  - Wall-clock-based purging of dirty pages (improved memory fairness)
  - Improved thread scaling (depending on application)
- User-accessible controls
  - Enabling background threads
  - Defining decay times for returning unused pages
  - Use of huge pages
- Usage:
  `LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libjemalloc.so ./myprog.x`

Content



1. Memory Architecture (2)
2. Memory Performance Tools
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. Final

DTLB Optimizations



- Any algorithm that does random accesses into a large memory region will likely suffer from DTLB misses
- Example TLB sizes for Intel Golden Cover
  - 1st-level DTLB for load+store: 96+96 entries for 4k pages
  - 2nd-level DTLB: 1024 entries
- Performance impact mitigation strategies:
  - Optimize memory pattern (may not be possible)
  - Use of huge pages

Explicit Huge Pages



- Size of huge pages is ISA and/or processor dependent
  - Check `/proc/meminfo`
- Explicit huge pages can be reserved at system boot time or before an application starts
  - Example:

```
echo 20 \
    > /proc/sys/vm/nr_hugepages
```

- Memory allocation using `mmap` with flag `MAP_HUGETLB` set

Example output:

```
MemTotal:       32582900 kB
MemFree:         1834816 kB
MemAvailable:    8545752 kB
Buffers:          323880 kB
Cached:         10578596 kB
SwapCached:       829012 kB
...
HugePages_Total:      20
HugePages_Free:       20
HugePages_Rsvd:        0
HugePages_Surp:        0
Hugepagesize:       2048 kB
Hugetlb:           40960 kB
DirectMap4k:      511424 kB
DirectMap2M:    21225472 kB
DirectMap1G:    12582912 kB
```

Explicit Huge Pages Example



```c
#include <stdlib.h>
#include <stdio.h>
#include <sys/mman.h>

int main()
{
    const int size = (1 << 26);      // 64 MiByte
    void *ptr = mmap(NULL, size, PROT_READ | PROT_WRITE,
                     MAP_PRIVATE | MAP_ANONYMOUS | MAP_HUGETLB, -1, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap() failed");
        return 1;
    }

    /* Some useful work */

    munmap(ptr, size);
    return 0;
}
```

Transparent Huge Pages



- Linux also offers **Transparent Huge Page Support (THP)**
- Check whether huge pages are enabled: `cat /sys/kernel/mm/transparent_hugepage/enabled`
  - `always`: Application requesting THP will stall on allocation failure
  - `madvise`: Like `always` but only for a particular address range
  - `never`
- `int madvise(void addr, size_t length, int advice)`
  - Linux system call used to give advice or directions to the kernel about an address range
  - Address range is defined by beginning (`addr`) and `length`
  - `advice` = `MADV_HUGEPAGE`: Enable THP for given address range

Transparent Huge Tables Example



```c
#include <stdlib.h>
#include <stdio.h>
#include <sys/mman.h>

int main()
{
    const int size = (1 << 26);      // 64 MiByte
    void *ptr = mmap(NULL, size, PROT_READ | PROT_WRITE,
                     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap() failed");
        return 1;
    }

    madvise(ptr, size, MADV_HUGEPAGE);

    /* Some useful work */

    munmap(ptr, size);
    return 0;
}
```

Transparent Huge Tables: jemalloc



- Assume that huge pages are enabled with setting `madvise`
- Set environment variable `MALLOC_CONF="thp:always"` to inform jemalloc about the use of huge pages
  - Example command line:

```
LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libjemalloc.so \
   MALLOC_CONF="thp:always" ./myprog.x
```

Which Huge Page Mechanism to Use?



- **Explicit Huge Pages**
  - Pro: EHPs are not affected by swapping
  - Con: Higher dependence on specific configuration of the target system
- **System-wide Transparent Huge Pages**
  - Pro: Good for quick tests
  - Con: May result in inefficient use of huge pages
- **Per-process Transparent Huge Pages** (`madvise`)
  - Pro: Flexible to use
  - Con: May be affected by swapping and costs of system calls

Content



1. Memory Architecture (2)
2. Memory Performance Tools
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. **Final**

Summary Lecture #10



- Performance relevant features of memory architectures
  - Caches
  - Non-uniform memory architectures
  - Virtual memory
- Tools for hardware discovery and memory-related monitoring and profiling
- Performance engineering topics
  - Cache blocking
  - Dynamic memory allocation
  - DTLB optimization and use of huge pages



[Wiki Media]

Questions?

These slides contain content and figures from

- Christoph Kessler, “Shared memory programming with OpenMP”, Lecture Slides, 2022
- A. Silberschatz, P.B. Galvin, G. Gagne, “Operating System Concepts, 10th Edition,” 2018



university of groningen

Performance Engineering: Lecture #11

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-10-05

Overview



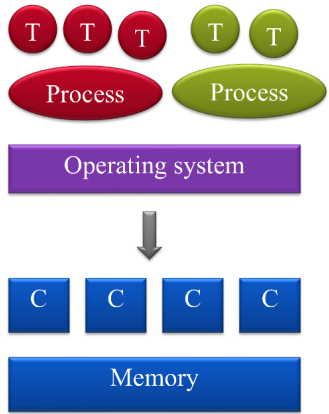
1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. Final

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. Final

Shared-Memory Parallel Programming

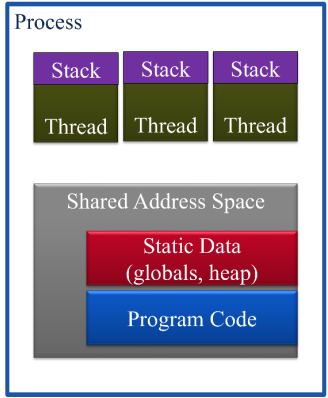


Basic assumptions:

- **Shared memory hardware support**
- An operating system that can provide
  - **Processes** with individual address spaces
  - **Threads** that share address space within a process
  - **Resource scheduling mechanisms**: thread scheduling, memory allocation and mapping

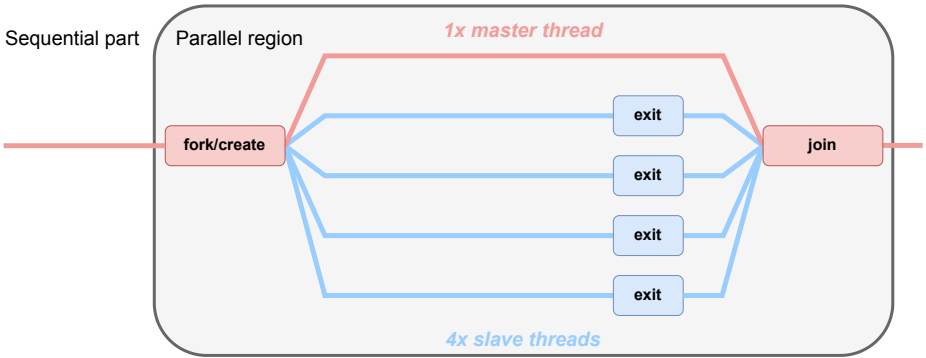
Processes versus Threads

- A process is a container for a program in execution, with state, capabilities, and access rights to resources shared by all its threads:
  - Address space
  - Code
  - Data (static data, heap data)
  - Opened files
- A thread is a unit of control flow and CPU scheduling
  - Each thread has its own PC and stack
  - Any program starts its execution as a single thread calling `main()`
  - New threads can be created through OS calls



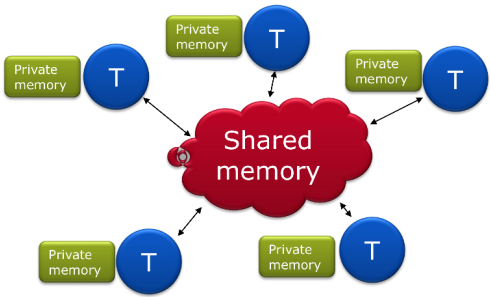
Fork-Join Model

- **Master thread** spawns a **team of threads** as needed



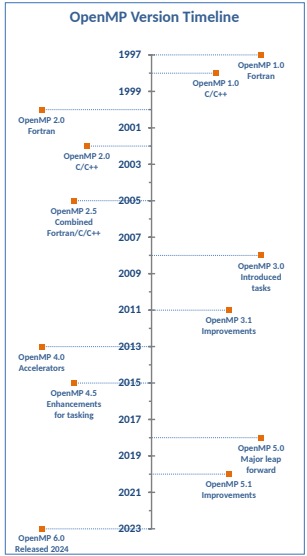
OpenMP: Introduction

- **OpenMP** = Parallel programming API
  - Based on **fork-join multi-threading model**
  - Collection of compiler directives, library routines, and environment variables
  - **Specification** defined by the **"OpenMP Architecture Review Board"**
- OpenMP requires **compiler support**
- Parallelisation is controlled by **compiler directives**
- **Abstract machine model**:
  - Concurrent threads
  - A shared address space
  - Private memory to each thread



OpenMP Evolution

- 1997: First release
- 2023: Latest release (version: 6.0)



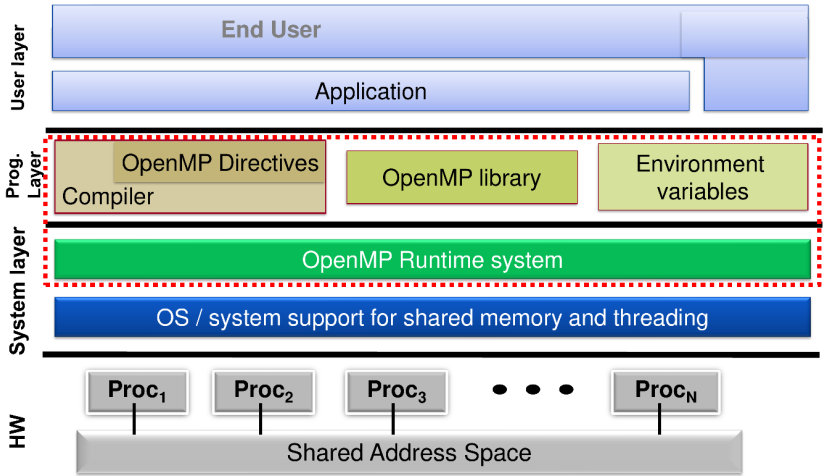
[Cornell Virtual Workshop]

Relevant Official OpenMP Documents



- Relevant documents
  - Specification “OpenMP Application Programming Interface”
  - Examples document: “OpenMP Application Programming Interface. Examples”
- Both documents are available at `https://www.openmp.org/`

OpenMP Software Stack



Number of Threads



- **Environment variable** `OMP_NUM_THREADS`
  - Sets the number of threads to use for parallel regions
  - List of positive integer values: one for each nesting level
  - Example:

    ```
    export OMP_NUM_THREADS=4,3,2
    ```
- **Function**: `void omp_set_num_threads(int)`

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. Final

OpenMP: Core Syntax



- Most of the constructs are **compiler directives**

  `#pragma omp construct [clause [clause]...]`

  - Example

    `#pragma omp parallel num_threads(4)`
- **Function prototypes and types** in the include file `omp.h`:

  `#include <omp.h>`
- Most OpenMP constructs apply to a **structured block**
  - **Structured block**: A block of one or more statements with one point of entry at the top and one point of exit at the bottom
  - Exception: Call of `exit()` within the structured block is allowed

Hello World: Sequential Program (1/3)



```c
#include <stdio.h>


int main()
{


    int id = 0;

    printf("Hello (id=%d) ", id);
    printf("world (id=%d)\n", id);

  return 0;
}
```

Hello World: Parallel Program (2/3)



```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main()
5  {
6  #pragma omp parallel
7    {
8      int id = omp_get_thread_num();
9
10     printf("Hello (id=%d) ", id);
11     printf("world (id=%d)\n", id);
12   }
13   return 0;
14 }
```

Hello World: Compile and Run (3/3)



Compilation:

```
gcc -fopenmp hello-par.c
```

Output on Kunpeng 920:

```
Hello (id=23) world (id=11)
world (id=7)
Hello (id=20) Hello (id=25) world (id=19)
world (id=124)
Hello (id=21) Hello (id=27) world (id=22)
world (id=20)
Hello (id=28) world (id=25)
Hello (id=26) world (id=21)
world (id=6)
world (id=23)
world (id=9)
world (id=28)
world (id=26)
Hello (id=30) Hello (id=32) world (id=32)
world (id=27)
Hello (id=31) world (id=31)
world (id=30)
Hello (id=29) Hello (id=14) world (id=29)
world (id=14)
world (id=15)
Hello (id=41) world (id=41)
Hello (id=42) world (id=42)
Hello (id=44) world (id=44)
Hello (id=46) world (id=46)
Hello (id=52) world (id=52)
Hello (id=54) world (id=54)
```

Parallel Regions (1/2)

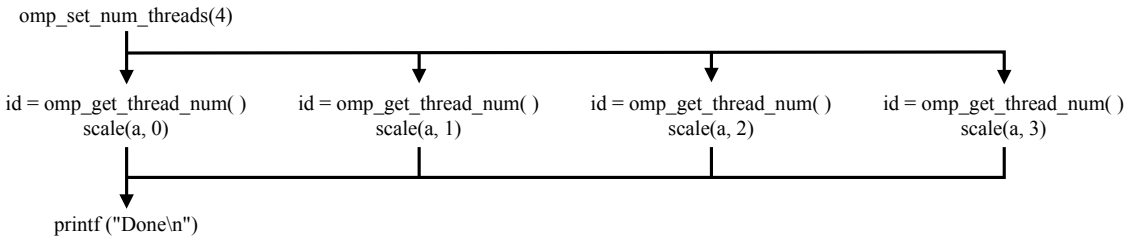


- Threads in OpenMP are created using a **`parallel` construct**
- **Example**: Parallel region with 4 threads

```
1  #include <omp.h>
2
3  double a[1000];
4
5  int main() {
6    omp_set_num_threads(4);
7
8  #pragma omp parallel
9    {
10     int id = omp_get_thread_num();
11     scale(a, id);
12   }
13
14   printf("Done\n");
15 }
```

- Each thread will call `scale(a,id)` with
  - The same value of `a`
  - Different values of `id` $= 0, 1, 2, 3$

Parallel Regions (2/2)



Shared versus Private Variables



- **Shared variables**

  `#pragma omp parallel shared (varlist)`

  - A change made to a shared variable by one thread is **eventually** visible for all threads in the program
  - Memory consistency is implementation dependent
- **Private variables**

  `#pragma omp parallel private (varlist)`

  - One local instance per thread and exclusive access by that thread
  - Allocation on the thread's run-time stack

Synchronisation: General Introduction



- Unintended sharing of data causes race conditions
  - **Race condition** = A situation in which two threads are concurrently trying to change the value of a variable
  - **No problem** if all threads do read-only accesses
  - **Problem** if read-after-write dependences exist, e.g.
    - Thread T_0 updates the shared variable x
    - Thread T_1 is expected to read the updated value of x
- Need for **synchronisation**
  - Synchronisation can be expensive ☞ **performance** engineering aims for less synchronisation
  - **Correctness** requires to implement all necessary synchronisation
    - **Beware**: The presence of race conditions is not a sufficient criterion for erroneous results

OpenMP Synchronisation



- High level synchronization
  - `critical` construct
  - `atomic` construct
  - `barrier` construct
  - `ordered` construct
- Low level synchronization flush
  - `flush` construct
  - Lock routines

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. Final

Work Scheduling



- **Challenge**: How to distribute work to the different threads?
  - Possibly work items of different size need to be distributed
  - The size of the work items might not be known at compile time
- **Scheduling strategies** implemented in OpenMP
  - **Static**: Work items are known by the programmer to be (almost) equal
  - **Dynamic**: Work items' sizes are unpredictable ☞ need for **dynamic load balancing**
    - Dynamic scheduling will come with an overhead
  - **Guided**: Special case of dynamic scheduling to reduce scheduling overhead
  - **Auto**: The run-time system tries to learn from previous executions of the same loop
- **Additional strategy**: Dynamically adjust the number of threads

`schedule` Clause



The schedule clause affects how loop iterations are mapped onto threads

- `schedule(static [,chunk])`
  - At loop entry, deal-out blocks of iterations of size `chunk` to each thread
- `schedule(dynamic[,chunk])`
  - Each thread grabs `chunk` work items (e.g., loop iterations) off a (conceptually) shared work queue, executes them and repeats, until all items have been handled
- `schedule(guided[,chunk])`
  - Dynamic scheduling where the size of the grabbed block starts large and exponentially shrinks down to minimum size `chunk` as the calculation proceeds.
- `schedule(runtime)`
  - Schedule and chunk size taken from the `OMP_SCHEDULE` environment variable
- `schedule(auto)`
  - Schedule is up to the run-time to choose (does not have to be any of the above)

Work Scheduling: Environment Variables

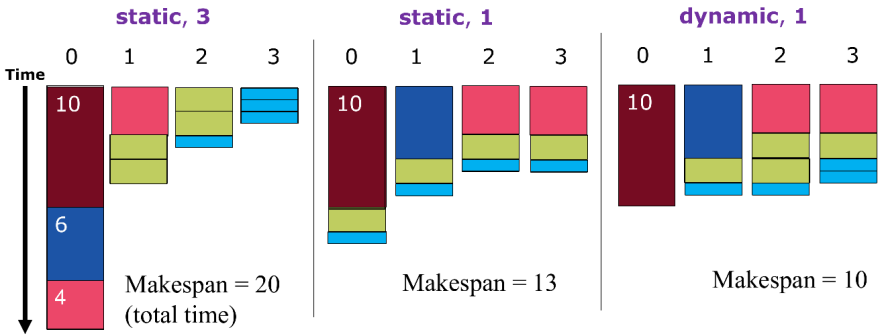


Instead of using clauses in the program, also environment variables can be used to set OpenMP **Internal Control Variables (ICV)**:

- `OMP_SCHEDULE`: `type[, chunk]`
  - Affects all OpenMP parallelised loops
  - `type`: `static`, `dynamic`, `guided`, or `auto`
  - `chunk`: An optional positive integer that specifies the chunk size
- `OMP_DYNAMIC`: `true` or `false`
  - Controls dynamic adjustment of the number of threads to use for executing parallel regions

Why Use Different Scheduling Strategies?

- Example work queue with 12 entries and different execution times: $10, 6, 4, 4, 2, 2, 2, 2, 1, 1, 1, 1$
- Assume 4 threads



Threads Placement



- **Challenge**: How to allocate hardware resources to threads?
  - Placement of threads on cores
  - Use of physical memory by each thread
- **Strategy**: Define policies
  - Allow threads to be migrated dynamically or request a fix placement
  - Placement following an affinity policy
    - **Processor affinity** = A scheduling attribute that specifies a thread's affinity for a processor core for execution
  - Physical memory allocation will follow based on OS allocation rules
- **Affinity policies**
  - `master`: Policy instructs the execution environment to assign every thread in the team to the same place as the master thread
  - `close`: Assign the threads in the team to places close to the place of the parent thread
  - `spread`: Create a sparse distribution for a team of threads among the available places

Tune Threads Placement



- **OpenMP environment variables**
  - `OMP_PROC_BIND`: Defines the affinity policies and can have the following values
    - `true`: Threads should not be moved
    - `false`: Threads may be moved
    - Comma separated list of `master`, `close`, or `spread`: Define thread affinity policy
  - `OMP_PLACES`: List of locations for placing threads
- **OpenMP clauses**
  - The `proc_bind` clause can be used in a `parallel` construct

OMP_DISPLAY_ENV



- Environment variable controls on information displayed by the run-time system at the start
- Possible values: `true`, `false`, `verbose`
- Output is implementation specific

```
1  OPENMP DISPLAY ENVIRONMENT BEGIN
2    _OPENMP = '201511'
3    [host] OMP_DYNAMIC = 'FALSE'
4    [host] OMP_NESTED = 'FALSE'
5    [host] OMP_NUM_THREADS = '1'
6    [host] OMP_SCHEDULE = 'DYNAMIC'
7    [host] OMP_PROC_BIND = 'FALSE'
8    [host] OMP_PLACES = ''
9    [host] OMP_STACKSIZE = '0'
10   [host] OMP_WAIT_POLICY = 'PASSIVE'
11   [host] OMP_THREAD_LIMIT = '4294967295'
12   [host] OMP_MAX_ACTIVE_LEVELS = '1'
13   [host] OMP_NUM_TEAMS = '0'
14   [host] OMP_TEAMS_THREAD_LIMIT = '0'
15   [all] OMP_CANCELLATION = 'FALSE'
16   [all] OMP_DEFAULT_DEVICE = '0'
17   [all] OMP_MAX_TASK_PRIORITY = '0'
18   [all] OMP_DISPLAY_AFFINITY = 'FALSE'
19   [host] OMP_AFFINITY_FORMAT = 'level %L thread %i affinity %A'
20   [host] OMP_ALLOCATOR = 'omp_default_mem_alloc'
21   [all] OMP_TARGET_OFFLOAD = 'DEFAULT'
22   [host] GOMP_CPU_AFFINITY = ''
23   [host] GOMP_STACKSIZE = '0'
24   [host] GOMP_SPINCOUNT = '300000'
25 OPENMP DISPLAY ENVIRONMENT END
```

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. Final

Ad-Hoc Checklist Methodology for Loops



1. Find compute-intensive loops
2. Check if loop iterations are
   - independent, or
   - comprise a reduction operation
3. Place the appropriate OpenMP directives
4. Check performance impact

OpenMP: Parallel Loops (1/3)



- **Step #1**: Find compute-intensive loop
- **Step #2**: Check for loop carry dependences

```
1  #include <stdlib.h>
2  #include <math.h>
3
4  double *a;
5  double *b;
6
7  int main(int argc, char *argv[])
8  {
9    const int n = atoi(argv[1]);
10   a = (double *) malloc(n * sizeof(double));
11   b = (double *) malloc(n * sizeof(double));
12
13
14   for (int i = 0; i < n; i++)
15     a[i] = 1.0 + 0.001 * i;
16
17
18   for (int i = 0; i < n; i++)
19     b[i] = exp(a[i]);
20
21   return 0;
22 }
```

OpenMP: Parallel Loops (2/3)



- **Step #3**: Place directives
  - `for` **construct**

```
1  #include <stdlib.h>
2  #include <math.h>
3
4  double *a;
5  double *b;
6
7  int main(int argc, char *argv[])
8  {
9    const int n = atoi(argv[1]);
10   a = (double *) malloc(n * sizeof(double));
11   b = (double *) malloc(n * sizeof(double));
12
13
14   for (int i = 0; i < n; i++)
15     a[i] = 1.0 + 0.001 * i;
16
17 #pragma omp parallel for
18   for (int i = 0; i < n; i++)
19     b[i] = exp(a[i]);
20
21   return 0;
22 }
```

OpenMP: Parallel Loops (2/3)



- **Step #3**: Place directives
  - `for` **construct**

Question: Would code run well on a NUMA architecture?

```
1  #include <stdlib.h>
2  #include <math.h>
3
4  double *a;
5  double *b;
6
7  int main(int argc, char *argv[])
8  {
9    const int n = atoi(argv[1]);
10   a = (double *) malloc(n * sizeof(double));
11   b = (double *) malloc(n * sizeof(double));
12
13
14   for (int i = 0; i < n; i++)
15     a[i] = 1.0 + 0.001 * i;
16
17 #pragma omp parallel for
18   for (int i = 0; i < n; i++)
19     b[i] = exp(a[i]);
20
21   return 0;
22 }
```

OpenMP: Parallel Loops (2/3)



- **Step #3**: Place directives
  - `for` **construct**

Question: Would code run well on a NUMA architecture?

No: Memory is touched first by master thread

```
1  #include <stdlib.h>
2  #include <math.h>
3
4  double *a;
5  double *b;
6
7  int main(int argc, char *argv[])
8  {
9    const int n = atoi(argv[1]);
10   a = (double *) malloc(n * sizeof(double));
11   b = (double *) malloc(n * sizeof(double));
12
13
14   for (int i = 0; i < n; i++)
15     a[i] = 1.0 + 0.001 * i;
16
17 #pragma omp parallel for
18   for (int i = 0; i < n; i++)
19     b[i] = exp(a[i]);
20
21   return 0;
22 }
```

OpenMP: Parallel Loops (3/3)



- **Solution**: Ensure that first access to memory is the same as later accesses
- **Note**: Need to use a memory allocator that does not initialise memory

```
1  #include <stdlib.h>
2  #include <math.h>
3
4  double *a;
5  double *b;
6
7  int main(int argc, char *argv[])
8  {
9    const int n = atoi(argv[1]);
10   a = (double *) malloc(n * sizeof(double));
11   b = (double *) malloc(n * sizeof(double));
12
13 #pragma omp parallel for
14   for (int i = 0; i < n; i++)
15     a[i] = 1.0 + 0.001 * i;
16
17 #pragma omp parallel for
18   for (int i = 0; i < n; i++)
19     b[i] = exp(a[i]);
20
21   return 0;
22 }
```

OpenMP: Reduction Loops (1/2)



- **Challenge**: Loop carry dependence

```
1  double dot_product(double *restrict a,
2                     double *restrict b,
3                     int n)
4  {
5    double s = 0.0;
6
7
8    for (int i = 0; i < n; i++)
9      s += a[i] * b[i];
10
11   return s;
12 }
```

OpenMP: Reduction Loops (2/2)



- **Solution**: Use reduction clause

```
1  double dot_product(double *restrict a,
2                     double *restrict b,
3                     int n)
4  {
5    double s = 0.0;
6
7  #pragma omp parallel for reduction (+:s)
8    for (int i = 0; i < n; i++)
9      s += a[i] * b[i];
10
11   return s;
12 }
```

OpenMP Nested Loops (1/4)



- Consider addition of two matrices
- **Challenge**: How to parallelise nested loops?

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6
7    for (int i = 0; i < n; i++) {
8
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (2/4)



- **Solution #1**: Parallelise inner loop

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6
7    for (int i = 0; i < n; i++) {
8  #pragma omp parallel for
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (2/4)



- **Solution #1**: Parallelise inner loop

Question: Is there a multi-threading overhead problem?

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6
7    for (int i = 0; i < n; i++) {
8  #pragma omp parallel for
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (2/4)



- **Solution #1**: Parallelise inner loop

Question: Is there a multi-threading overhead problem?

Yes: Enter parallel region n times

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6
7    for (int i = 0; i < n; i++) {
8  #pragma omp parallel for
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (3/4)



- **Solution #2**: Parallelise outer loop

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6  #pragma omp parallel for
7    for (int i = 0; i < n; i++) {
8
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (3/4)



- **Solution #2**: Parallelise outer loop

Question: What if $n < N_{\text{thread}}$?

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6  #pragma omp parallel for
7    for (int i = 0; i < n; i++) {
8
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (3/4)



- **Solution #2**: Parallelise outer loop

Question: What if $n < N_{\text{thread}}$?

At most n threads will be used

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6  #pragma omp parallel for
7    for (int i = 0; i < n; i++) {
8
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

OpenMP Nested Loops (4/4)



- **Solution #3**: Use `collapse` clause

```
1  #include <stdlib.h>
2
3  void madd(double *restrict a,
4            double *restrict b,
5            int n, int m) {
6  #pragma omp parallel for collapse(2)
7    for (int i = 0; i < n; i++) {
8
9      for (int j = 0; j < m; j++)
10       a[i * m + j] += b[i * m + j];
11   }
12 }
13
14 int main() {
15   const int n = 4;
16   const int m = 2048;
17
18   double *a = (double *) malloc(n*m * sizeof(double));
19   double *b = (double *) malloc(n*m * sizeof(double));
20
21   madd(a, b, n, m);
22
23   return 0;
24 }
```

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. **Cache Line Optimization**
6. Final

Cache Line Optimization: Introduction



- Cache line optimization is about inefficient use of loaded cache lines
- Main issues:
  - Poor spatial data locality
  - False sharing of cache lines (multithreaded applications only)

Poor Spatial Locality (1/2)



- **Problem**: **Poor spatial data locality** leads to not all data of a loaded cache line being used
- **Example**:

```
1  #define N 8192
2
3  double a[N][N];
4  double b[N][N];
5
6  int main() {
7    for (int j = 0; j < N; j++)
8      for (int i = 0; i < N; i++)
9        b[i][j] = a[i][j];
10 }
```

- **Mitigation strategies**:
  - Change memory access pattern
  - Adjust data structures

Poor Spatial Locality (2/2)



- **Optimised example**: Change from column-major to row-major traversal

```c
#define N 8192

double a[N][N];
double b[N][N];

int main() {
  for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)          // Now j is fastest running index
      b[i][j] = a[i][j];
}
```

- **Note**: This may not always be possible
  - Example: Matrix-matrix multiplication
    - One matrix is traversed row-major and the other column-major (or vice versa)

Case Study: Strided Vector Copy



- **Purpose** of this case study: Measure the impact of strided memory access
- **Specification**: Perform a strided copy of an array:

$$y_i \leftarrow x_i, \quad i = 0, 1s, 2s, 3s, \ldots$$

with stride $s \geq 1$

Strided Vector Copy: Code



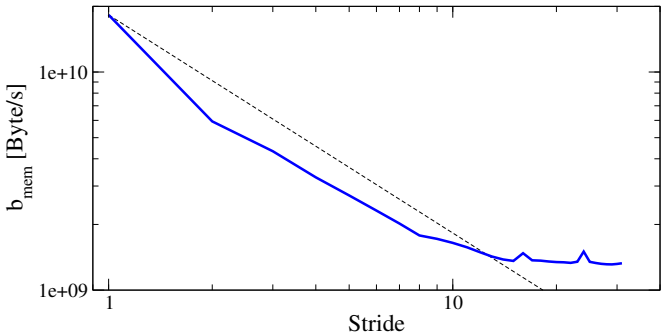
Benchmark routine:

```
1 void strided_vcpy(double *restrict y,
2                   double *restrict x,
3                   const int n,
4                   const int s)
5 {
6   for (int i = 0; i < s*n; i += s)
7     y[i] = x[i];
8 }
```

Benchmark driver:

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <time.h>
4
5 void strided_vcpy(double *restrict, double *restrict,
6                   const int, const int);
7
8 int main() {
9   int n    = 1 << 20;    int smax  = 32;
10  int nmax  = n * smax;   int nrep  = 1e3;
11  double *x = (double *) calloc(nmax, sizeof(double));
12  double *y = (double *) calloc(nmax, sizeof(double));
13
14  for (int s = 1; s < smax; s++) {
15    struct timespec t0, t1;
16    clock_gettime(CLOCK_PROCESS_CPUTIME_ID , &t0);
17
18    for (int i = 0; i < nrep; i++) strided_vcpy(y, x, n, s);
19
20    clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &t1);
21    double t  = (t1.tv_sec  - t0.tv_sec ) +
22                (t1.tv_nsec - t0.tv_nsec) * 1e-9;
23    double bw = 2 * nrep * n * sizeof(double) / t;
24    printf("%-4d %.4e\n", s, bw);
25  }
26 }
```

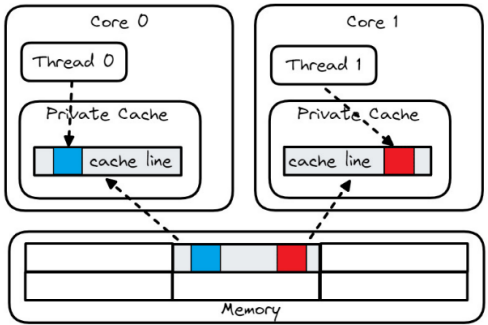
Strided Vector Copy: Results Kunpeng 920



- Until $s = 8$ one observes $b_{\mathrm{mem}}(s) \simeq b_{\mathrm{mem}}(1)/s$

False Sharing (1/2)

- **False sharing** = Two threads running on different cores modify different variables that reside on the same cache line



False Sharing (2/2)



Question: Why is false sharing problem?

False Sharing (2/2)



Question: Why is false sharing problem?

- Cores typically have private L1 data caches
- Multiple cores writing to the same cache will make the cache coherence protocol constantly moving a copy of the cache line for exclusive acces between the L1 caches

False Sharing (2/2)



Question: Why is false sharing problem?

- Cores typically have private L1 data caches
- Multiple cores writing to the same cache will make the cache coherence protocol constantly moving a copy of the cache line for exclusive acces between the L1 caches

- False versus true sharing
  - **False sharing** has only a performance impact
  - **True sharing** results in wrong results

False Sharing Example



```
struct S {
  int sumA; // sumA and sumB are likely to
  int sumB; // reside in the same cache line
};
S s;

{ // code executed by thread A
  for (int i = 0; i < N; i++)
    s.sumA += a[i];
}
```

```
{ // code executed by thread B
  for (int i = 0; i < N; i++)
    s.sumB += b[i];
}
```

Content



1. OpenMP Introduction
2. OpenMP Syntax
3. OpenMP Thread Scheduling
4. OpenMP Parallel Loops
5. Cache Line Optimization
6. **Final**

Summary Lecture #11



- OpenMP basics and syntax
- Performance engineering related to OpenMP
  - Optimization of critical sections
  - Tuning work scheduling and thread placement
  - Ad-hoc checklist methodology for OpenMP parallelisation of loops
  - NUMA optimization
  - Reduction loops
  - Nested loops
- Cache line optimizations: poor data locality, false sharing
- Memory-level parallelism and means to improve it

Questions?



[Wiki Media]

These slides contain content and figures from

- D. Bakhvalov, Performance Analysis and Tuning on Modern CPUs (2nd Ed.). 2024. [Online]. Available: `https://github.com/dendibakh/perf-book/releases`



university of groningen

Performance Engineering: Lecture #12

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-10-08

Overview



1. Memory-level Parallelism
2. Memory Prefetching
3. Data Structures
4. Final

Content



1. Memory-level Parallelism

2. Memory Prefetching

3. Data Structures

4. Final

Memory-Level Parallelism: Introduction



- **Memory-level parallelism (MLP)** = Measure of the number of outstanding and overlapping memory requests a system can have at a given time
- **Recall**: Requests to the memory are generated by the cache controllers
  - **MLP measure**: Number cache misses simultaneously outstanding
- **Background**: The number of outstanding requests per cache is limited by the size of the **Miss Status Handling Registers (MSHR)**
  - Typical MSHR size is $O(10)$ (smaller for L1, larger for LLC)
- **Strategies** for enabling more MLP:
  - Larger number of cores (and caches)
  - Larger out-of-order depth to enable earlier start of load instructions
  - Larger MSHRs
  - Memory prefetching

MSHR Register



- **Role**: Keep track of
  - Outstanding cache misses
  - Pending load/store accesses that refer to the missing cache line
- **Data structure** of an MSHR
  - Valid bit
  - Cache line address (to match incoming accesses)
  - Control/status bits
  - For each pending load/store
    - Valid, type, data size, byte in cache line, destination register or store buffer
- **Actions** involving MSHR
  - On a cache miss: Search MSHR for a pending access to the same cache line
    - Found: Allocate a load/store entry in the same MSHR entry
    - Not found: Allocate a new MSHR
    - No free entry: stall
  - When a cache line returns from the next level in memory: Search and update MSHR

Memory-Level Parallelism: Little's Law



- **Reminder** Little's Law: $N = X \cdot R$
  - N: Average number of requests in the system
  - X: Throughput
  - R: Residence time
- **Consider** the following case

$$\begin{aligned} N &= N_{\text{core}} \cdot N_{\text{MSHR}} \\ N_{\text{MSHR}} &= 16 \\ R &= 100\,\text{ns} \end{aligned}$$

- **Compute** the number of cores needed to saturate a memory bandwidth of $500\,\text{GB/s}$ assuming a cache line size of $64\,\text{B}$

$$N_{\text{core}} = \frac{(500\,\text{GB/s}) \cdot (100\,\text{ns})}{(64\,\text{B}) \cdot 16} \simeq 49$$

Increasing Memory-Level Parallelism



- Performance engineering means to improve memory-level parallelism
  - Increase the number of independent load instructions
  - Use more cores (and caches), e.g. by multi-threading
  - Improve memory prefetching
    - Use memory patterns that are more suitable for prefetcher
    - Consider software memory prefetching

Content



1. Memory-level Parallelism

2. Memory Prefetching

3. Data Structures

4. Final

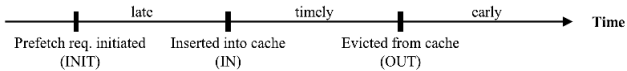
Memory Prefetchers



- **Idea**: Load data before program issues the instruction to load this data
- **Memory prefetcher** = Mechanism to fetch data from memory (or a lower cache level) to a (higher-level) cache
- **Software memory prefetching** relies on insertion of prefetch instructions into the program
- **Hardware memory prefetching** requires the following to be implemented
  - **Monitor**: Observe (and possibly record) memory access pattern
  - **Predictor**: Predict memory addresses to be accessed in the future
  - **Fetcher**: Issue a cache line fetch request
- Modern processors implement multiple prefetchers
  - Separate data and instruction prefetchers
  - Memory to cache prefetchers
  - Cache to cache prefetchers

Memory Prefetcher: Metrics

- **Metrics** to assess prefetcher implementations
  - **Coverage**: Number of misses that can be prefetched
  - **Timeliness**: Distance between prefetch and original miss
  - **Accuracy**: Probability that a prefetched line is used before being replaced



[Lee et al., 2012]

- Metric assessment is important to **minimise risk of negative performance impact**
  - Memory bus contention
  - Cache pollution

Monitoring of Memory Prefetchers

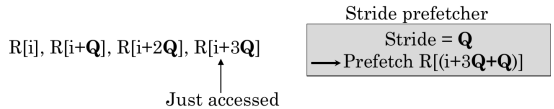


- Support for monitoring hardware prefetching is **not standardised**
- Some processors provide suitable **hardware counters**
- Example(s):
  - Kunpeng 920: The `hit_on_prf` counts the number of hits on prefetched data

Hardware Memory Prefetcher (1/2)

- Popular **memory prefetcher algorithms**
  - **Tagged prefetcher** [A.J. Smith; 1982]
    - Associates a tag bit with every memory block
    - Use tag to detect when a block is demand-fetched or a prefetched block is referenced for the first time
    - In either of these cases, the next sequential block is fetched
  - **Stride prefetcher**
    - Monitor sequence of memory addresses to update a Reference Prediction Table (RPT)
    - Use RPT to identify strided memory access patterns to predict addresses used in the future

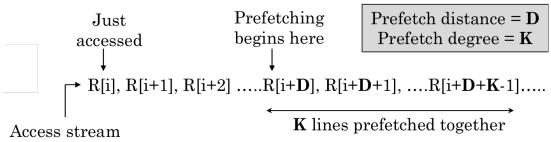
[Mittal; 2016]



Hardware Memory Prefetcher (2/2)

- **Key parameters**
  - Prefetch distance
  - Prefetch degree

[Mittal; 2016]



Software Memory Prefetching Instructions (1/2)



- Some processors support **prefetching instructions or prefetch routines**
- Options to leverage such instructions
  - Let the compiler generate instructions or calls
  - Leverage compiler-specific support
    - Example GCC: `__builtin_prefetch(const void *addr, ...)`
      - Optional argument `rw` $= 0, 1, 2$: '1' means prepare for write
      - Optional argument `locality` $= 0, 1, 2, 3$: degree of temporal locality

```
1 void vcpy(double *restrict a, double *restrict b, int n, int d)
2 {
3   for (int i = 0; i < n; i++) {
4     a[i] = a[i] + b[i];
5     __builtin_prefetch (&a[i+d], 1, 0);
6     __builtin_prefetch (&b[i+d], 0, 0);
7   }
8 }
```

Note: Data prefetch does not generate faults if the address is invalid

Software Memory Prefetching Instructions (2/2)



- Options to leverage such instructions (cont.)
  - Use of intrinsics
    - Example Intel: `_mm_prefetch(const void *, enum _mm_hint)`

Hint	Purpose
`_MM_HINT_T0`	Load data to L1D and L2 cache
`_MM_HINT_T1`	Load data to L2 cache only
`_MM_HINT_NTA`	Load data to L1D and L2 cache + set non-temporal hint

Memory Prefetcher Optimization



- Performance engineering means to leverage prefetcher techniques
  - **Hardware prefetchers**
    - Aim for sequential memory accesses that benefit from tagged prefetchers and stride prefetchers
    - Vendor information on prefetcher tends to be scarse
  - **Use software prefetching**
    - Warning: This may result in non-portable code

Content



1. Memory-level Parallelism

2. Memory Prefetching

3. Data Structures

4. Final

Data Packing



- **Consider**: Complex data structures with elements of different but small size
- **Optimization target**: Reduce size of data structures
  - Minimise memory footprint
  - Make better use of memory bandwidth
  - Optimize cache reuse
- **Implementation approach**: Minimize variable size and pack multiple variables in a single Byte

```
1 typedef struct {
2   int a;
3   int b;
4   int c;
5 } flags_up_t;
```

$\Rightarrow$

```
1 typedef struct {
2   unsigned char a:4;
3   unsigned char b:2;
4   unsigned char c:2;
5 } flags_pa_t;
```

Question: How does size change?

- **Caveat**: Computing may become more expensive due to bit operations

Data Packing



- **Consider**: Complex data structures with elements of different but small size
- **Optimization target**: Reduce size of data structures
  - Minimise memory footprint
  - Make better use of memory bandwidth
  - Optimize cache reuse
- **Implementation approach**: Minimize variable size and pack multiple variables in a single Byte

```
1 typedef struct {
2   int a;
3   int b;
4   int c;
5 } flags_up_t;
```

$\Rightarrow$

```
1 typedef struct {
2   unsigned char a:4;
3   unsigned char b:2;
4   unsigned char c:2;
5 } flags_pa_t;
```

Question: How does size change?

```
sizeof(flags_up_t) = 12
sizeof(flags_pa_t) = 1
```

- **Caveat**: Computing may become more expensive due to bit operations

Data Padding



- **Consider**: Complex data structures with elements of different size causing alignment issues
- **Optimization target**: Data structures with aligned memory addresses
- **Implementation strategy**: Add padding to improve data alignment
  - Allow for memory footprint to increase
- **Example**: Aim for 32-Byte alignment

```
1 typedef struct {
2   int    i;
3   double x;
4   char   flg;
5 } mydata_ua_t;
```

$\Rightarrow$

```
1 typedef struct {
2   int    i;
3   double x;
4   char   flg;
5   char   p[11];
6 } mydata_al_t;
```

Question: How does size change?

- **Note**:
  - Compiler performs itself padding
  - Structure size is architecture dependent

Data Padding



- **Consider**: Complex data structures with elements of different size causing alignment issues
- **Optimization target**: Data structures with aligned memory addresses
- **Implementation strategy**: Add padding to improve data alignment
  - Allow for memory footprint to increase
- **Example**: Aim for 32-Byte alignment

```
1 typedef struct {
2   int    i;
3   double x;
4   char   flg;
5 } mydata_ua_t;
```

⇒

```
1 typedef struct {
2   int    i;
3   double x;
4   char   flg;
5   char   p[11];
6 } mydata_al_t;
```

Question: How does size change?

```
sizeof(flags_ua_t) = 24
sizeof(flags_ul_t) = 32
```

- **Note**:
  - Compiler performs itself padding
  - Structure size is architecture dependent

Field Re-ordering



- **Consider**: Complex data structures with elements of different size (including small sizes)
- **Optimization target**: Reduce memory footprint
- **Example**: Aim for 32-Byte alignment

```
1 typedef struct {
2   char    flg1;
3   int     i;
4   double  x;
5   double  y;
6   char    flg2;
7 } mydata_uo_t;
```

$\Rightarrow$

```
1 typedef struct {
2   char    flg1;
3   char    flg2;
4   int     i;
5   double  x;
6   double  y;
7 } mydata_re_t;
```

Question: How does size change?

- **Note**:
  - Best to order structure elements by size
  - Structure size is architecture dependent

Field Re-ordering



- **Consider**: Complex data structures with elements of different size (including small sizes)
- **Optimization target**: Reduce memory footprint
- **Example**: Aim for 32-Byte alignment

```
1 typedef struct {
2   char    flg1;
3   int     i;
4   double  x;
5   double  y;
6   char    flg2;
7 } mydata_uo_t;
```

⇒

```
1 typedef struct {
2   char    flg1;
3   char    flg2;
4   int     i;
5   double  x;
6   double  y;
7 } mydata_re_t;
```

Question: How does size change?

```
sizeof(flags_uo_t) = 32
sizeof(flags_re_t) = 24
```

- **Note**:
  - Best to order structure elements by size
  - Structure size is architecture dependent

Containers



- There is a wide variety of ready-to-use data containers in almost any language
  - Examples for C++:
    - `std::array`
    - `std::vector`
    - `std::map`
- For containers defined in the language standard, good compiler support is likely
  - **Note**: No guarantee for optimality, but likely good compromise between performance and portability

Structure Splitting



- **Consider**: Complex data structures with elements with different usage frequency
- **Optimization target**: Keep more frequently used data in cache
- **Implementation strategy**: Split data structures
- **Example** that assumes that point coordinates are access more often:

```cpp
#include <vector>

struct Point {
  int x;
  int y;
  int z;
  /*many other fields*/
};

std::vector<Point> points;
```

$\Rightarrow$

```cpp
#include <vector>

struct PointCoords {
  int x;
  int y;
  int z;
};

struct PointInfo {
  /*many other fields*/
}

std::vector<PointCoords> pointCoords;
std::vector<PointInfo> pointInfos;
```

AoS versus SoA Data Layout (1/2)



AoS = In the **Array of Structure** data layout there is a single array with each array element holding several attributes

```cpp
#include <vector>

struct Point {
   int x;
   int y;
   int z;
};

std::vector<Point> points;
```

⇔

SoA = In the **Structure of Array** data layout there is for each attribute an array

```cpp
#include <vector>

struct Points {
   std::vector<int> x;
   std::vector<int> y;
   std::vector<int> z;
} points;
```

AoS versus SoA Data Layout (2/2)



- Benefit(s) of **SoA**
  - Helps to facilitate streaming data access
  - Good of architectures that are optimised for data stream processing (e.g., GPUs)
- Benefit(s) of **AoS**
  - Often results in higher temporal data locality
  - Good of architectures with large caches (e.g., CPUs)

Multidimensional Arrays



- **Consider**: Multidimensional arrays of simple data type (e.g., double)
- **Optimization targets**:
  - Good alignment properties
  - No memory fragmentation
  - Efficient access to single elements
- **Challenge**: C and C++ do provide only limited support for multidimensional arrays

Case Study: Multidimensional Arrays



- **Purpose** of this case study: Test different implementations of a 2-dimensional array
- **Specification** for `mdarray`: Support the following functionality
  - **Allocation** of a 2-dimensional array of doubles with size defined at run-time
  - **Accumulation**: Add to a given array the values of another array ($B \leftarrow B + A$)
  - **Free** the allocated memory

mdarray: Array of Pointers



Header file:

```
double **mda_alloc(int, int);
void mda_free(double **, int, int);
void mda_accum(double **, double **, int, int);
```

Benchmark driver:

```
#include "aop.hpp"

int main() {
  const int m = (1 << 18);
  const int n = 16;

  for (int i = 0; i < 1000; i++) {
    double **a = mda_alloc(m, n);
    double **b = mda_alloc(m, n);

    mda_accum(a, b, m, n);

    mda_free(a, m, n);
    mda_free(b, m, n);
  }
}
```

Benchmark:

```
#include <stdlib.h>
#include "aop.hpp"

double **mda_alloc(int m, int n) {
  double **a = (double **) calloc(m, sizeof(double *));
  for (int i = 0; i < m; i++)
    a[i] = (double *)calloc(n, sizeof(double));
  return a;
}

void mda_free(double **a, int m, int n) {
  for (int i = 0; i < m; i++)
    free(a[i]);
  free(a);
}

void mda_accum(double **a, double **b, int m, int n) {
  for (int i = 0; i < m; i++)
    for (int j = 0; j < n; j++)
      b[i][j] += a[i][j];
}
```

mdarray: Flattened Array



Header file:

```
1 double *mda_alloc(int, int);
2 void mda_free(double *);
3 void mda_accum(double *, double *, int, int);
```

Benchmark driver:

```
1 #include "flat.hpp"
2
3 int main() {
4   const int m = (1 << 18);
5   const int n = 16;
6
7   for (int i = 0; i < 1000; i++) {
8     double *a = mda_alloc(m, n);
9     double *b = mda_alloc(m, n);
10
11    mda_accum(a, b, m, n);
12
13    mda_free(a);
14    mda_free(b);
15  }
16 }
```

Benchmark:

```
1 #include <stdlib.h>
2 #include "flat.hpp"
3
4 double *mda_alloc(int m, int n) {
5   double *a = (double *) calloc(m*n, sizeof(double));
6   return a;
7 }
8
9 void mda_free(double *a) {
10  free(a);
11 }
12
13 void mda_accum(double *a, double *b, int m, int n) {
14   for (int i = 0; i < m; i++)
15     for (int j = 0; j < n; j++)
16       b[i*n + j] += a[i*n + j];
17 }
```

mdarray: Own C++ Class



Header file:

```
1  #include <stdlib.h>
2
3  class mda2_t {
4    private:
5      double *_a; int _m, _n;
6
7    public:
8      mda2_t(int m, int n) : _m(m), _n(n) {
9        _a = (double *) calloc(m*n, sizeof(double));
10     }
11
12     ~mda2_t() { free(_a); }
13
14     double& operator()(int i, int j) {
15       return _a[i*_n + j];
16     }
17
18     int m() const { return _m; }
19     int n() const { return _n; }
20 };
21
22 void mda_accum(mda2_t &, mda2_t &);
```

Benchmark:

```
1 #include <stdlib.h>
2 #include "class.hpp"
3
4 void mda_accum(mda2_t &b, mda2_t &a) {
5   for (int i = 0; i < b.m(); i++)
6     for (int j = 0; j < b.n(); j++)
7       b(i, j) += a(i, j);
8 }
```

Benchmark driver:

```
1  #include "class.hpp"
2
3  int main() {
4    const int m = (1 << 18);
5    const int n = 16;
6
7    for (int i = 0; i < 1000; i++) {
8      mda2_t a(m, n);
9      mda2_t b(m, n);
10
11     mda_accum(a, b);
12   }
13 }
```

`mdarray`: Kunpeng 920 Results

	N_{cycle}	N_{instr}
Array of pointers, -O3	$110.0 \cdot 10^9$	$26.7 \cdot 10^{10}$
Flattened array, -O3	$8.7 \cdot 10^9$	$1.8 \cdot 10^{10}$
C++ class, -O3	$9.1 \cdot 10^9$	$1.9 \cdot 10^{10}$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - The array-of-pointers version is about $10\times$ slower and requires about $15\times$ more instructions
  - The array-of-pointers requires a very large number of instructions
    - About 64 instructions per matrix element and iteration

Multidimensional Array Access



- For very high-dimensional arrays, computing element addresses can become expensive

Case Study: Tensor Contractions



- **Purpose** of this case study: Explore the overhead of index computations
- **Specification**: Implement the following tensor contraction

$$\rho_{t,p} \leftarrow \Sigma_{t,x,y,z,c} f^{(1)}_{t,x,y,z,c,c} \cdot f^{(2)}_{t,x,y,z,c,c} \cdot \phi_p$$

with

- $t = 0, \ldots, N_t - 1$
- $x, y, z = 0, \ldots, N_s - 1$
- $c = 0, \ldots, N_c - 1$
- $p = 0, \ldots, N_p - 1$

Tensor Contractions: Original



Benchmark:

```
1  #include <stdlib.h>
2
3  void contract(double *restrict f1,
4                double *restrict f2,
5                double *restrict p,
6                double *restrict r,
7                int nt, int ns, int nc, int np) {
8    for (int it = 0; it < nt; it++) {
9      for (int ip = 0; ip < np; ip++)
10       r[it*np+ip] = 0.0;
11     for (int ix = 0; ix < ns; ix++)
12       for (int iy = 0; iy < ns; iy++)
13         for (int iz = 0; iz < ns; iz++)
14           for (int ic = 0; ic < nc; ic++) {
15             int idx = it * ns * ns * ns * nc * nc +
16                       ix * ns * ns * nc * nc +
17                            iy * ns * nc * nc +
18                                 iz * nc * nc +
19                                      ic * nc +
20                                           ic;
21             for (int ip = 0; ip < np; ip++)
22               r[it*np+ip] += f1[idx] * f2[idx] * p[ip];
23           }
24   }
25 }
```

Benchmark driver:

```
1  #include <stdlib.h>
2
3  void contract(double *restrict,
4                double *restrict,
5                double *restrict,
6                double *restrict,
7                int, int, int, int);
8
9  int main() {
10   const int nt =  8;
11   const int ns =  4;
12   const int nc =  3;
13   const int np = 40;
14   const int sd = sizeof(double);
15
16   double *f1 = (double*) calloc(nt*ns*ns*ns*nc*nc, sd);
17   double *f2 = (double*) calloc(nt*ns*ns*ns*nc*nc, sd);
18   double *p  = (double*) calloc(np, sd);
19   double *r  = (double*) calloc(nt*np, sd);
20
21   for (int i = 0; i < 100000; i++)
22     contract(f1, f2, p, r, nt, ns, nc, np);
23
24   free(f1); free(f2); free(p); free(r);
25 }
```

Tensor Contractions: Improved



Benchmark:

```
1  #include <stdlib.h>
2
3  void contract(double *restrict f1,
4                double *restrict f2,
5                double *restrict p,
6                double *restrict r,
7                int nt, int ns, int nc, int np) {
8    for (int it = 0; it < nt; it++) {
9      for (int ip = 0; ip < np; ip++)
10       r[it*np+ip] = 0.0;
11     for (int ix = 0; ix < ns; ix++)
12       for (int iy = 0; iy < ns; iy++) {
13         int idx1 = it * ns * ns * ns * nc * nc +
14                          ix * ns * ns * nc * nc +
15                               iy * ns * nc * nc;
16         for (int iz = 0; iz < ns; iz++)
17           for (int ic = 0; ic < nc; ic++) {
18             int idx = idx1 + iz * nc * nc +
19                                   ic * nc +
20                                        ic;
21             for (int ip = 0; ip < np; ip++)
22               r[it*np+ip] += f1[idx] * f2[idx] * p[ip];
23           }
24       }
25   }
26 }
```

Benchmark driver:

```
1  #include <stdlib.h>
2
3  void contract(double *restrict,
4                double *restrict,
5                double *restrict,
6                double *restrict,
7                int, int, int, int);
8
9  int main() {
10   const int nt =  8;
11   const int ns =  4;
12   const int nc =  3;
13   const int np = 40;
14   const int sd = sizeof(double);
15
16   double *f1 = (double*) calloc(nt*ns*ns*ns*nc*nc, sd);
17   double *f2 = (double*) calloc(nt*ns*ns*ns*nc*nc, sd);
18   double *p  = (double*) calloc(np, sd);
19   double *r  = (double*) calloc(nt*np, sd);
20
21   for (int i = 0; i < 100000; i++)
22     contract(f1, f2, p, r, nt, ns, nc, np);
23
24   free(f1); free(f2); free(p); free(r);
25 }
```

Tensor Contractions: Kunpeng 920 Results

	N_{cycle}	N_{instr}
Original, -O1	$4.18 \cdot 10^9$	$12.0 \cdot 10^9$
Improved, -O1	$3.74 \cdot 10^9$	$12.0 \cdot 10^9$
Original, -O3	$2.12 \cdot 10^9$	$6.3 \cdot 10^9$
Improved, -O3	$2.08 \cdot 10^9$	$6.4 \cdot 10^9$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - At the O1 optimization level, the gain is about $11\,\%$
  - Significantly less gain for O3 (only about $1\,\%$)

mdspan



[D. Hollman et al.; 2019]

- **mdspan** is a multidimensional array view proposed for the C++ standard
  - First developed within Kokkos
- Pseudo-code example inspired by the tensor contraction case study:

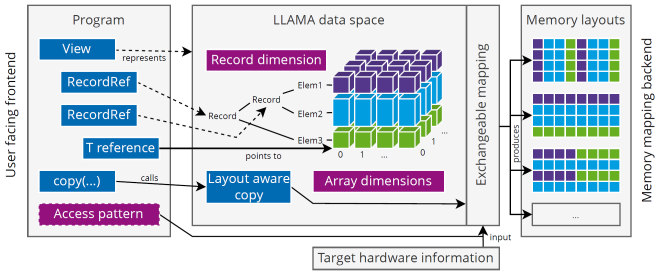
```
1  #include "mdspan.hpp"
2
3  int main() {
4    const int nt =  8;
5    const int ns =  4;
6    const int nc =  3;
7    const int np =  8;
8
9    double *_f1  = (double *) calloc(nt * ns * ns * ns * nc * nc, sizeof(double));
10
11   Kokkos::mdspan f1 { _f1, Kokkos::extents{nt, ns, ns, ns, nc, nc} };
12
13   f1(0,0,0,0,0,0) = 1.0;
14 }
```

- **Note**: Separation of **memory region** (here referenced by `_f1`) and **memory view**
- Also slicing is supported

Memory Abstractions

- **Goal**: Various efforts aim for a portable abstraction layer unifying memory optimization techniques
- Selected examples:
  - SIMD vectors: NSIMD, Highway
  - Multidimensional arrays: Kokkos
  - More general approach: LLAMA

[B. Gruber et al.; 2023]



Content



1. Memory-level Parallelism

2. Memory Prefetching

3. Data Structures

4. **Final**

Summary Lecture #12



- Memory-level parallelism and means to improve it
- Memory prefetching
- Data structures
  - Different optimization techniques
    - Data packing
    - Data padding
    - Field re-ordering
    - Use of containers
    - Structure splitting
  - Multi-dimensional arrays
  - AoS versus SoA
  - Path towards memory abstractions

Questions?



[Wiki Media]



university of groningen

Performance Engineering: Lecture #13

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-10-14

Overview



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

Content



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

Network Architectures: Introduction



- Key components relevant for communication performance
  - Link technology and speed
  - Link communication protocols
    - Protocol for communication of messages between nodes
  - Communication library protocol
    - Protocol for interacting with the communication library
  - Network topology

Message Communication Protocols



- **Eager protocol**:
  - Sender pushes message regardless of receiver state
  - Low overhead
  - Useful for small messages
- **Rendezvous protocol**:
  - Handshake happens between the sender and the receiver before sending data
  - High overhead due handshake involving control messages
  - Useful for large messages

Blocking versus Non-blocking Communication



- **Blocking call** = Call only returns once request has been served
  - Caller will wait, potentially missing the opportunity to do useful work
- **Non-blocking call** = Call returns before request has been served
  - Caller will have to check for completion

One- Versus Two-sided Communication



- **One-sided communication** = Sender (or receiver) pushes (or pulls) a message without active involvement of the receiver (or sender)
  - Get/put semantics
  - **Advantages(s)**
    - Typically lower overhead
  - **Disadvantage(s)**
    - Need for explicit synchronization, i.e. more complexity
- **Two-sided communication** = Sender and receiver cooperate for communicating a message
  - Send/receive semantics
  - **Advantages(s)**
    - More intuitive communication model
  - **Disadvantage(s)**
    - Higher latency, more overhead

Remote Direct Memory Access (RDMA)



- **Idea**: Allow one process to directly access the (private) memory space of another process running on another node without involving the CPU cores or operating system of the other node
- **Implementation** of an **RDMA "put"** on a NIC
  - Initial setup
  - For each communication:
    - Sender CPU places request into a queue
    - Sender NIC fetches request
    - Sender NIC gets data from local memory
    - Sender NIC sends message including destination addresses
    - Receiver NIC receives data and puts it into local memory
- Network technologies supporting RDMA
  - Infiniband
  - Ethernet supporting RoCE (RDMA over Converged Ethernet)

Network Topology



- **Network topology** refers to the way in which the network of computing nodes is connected
- **Physical topology** = Topology of the physical network
- **Logical topology** = Logical view on the way network is connected
- Simplest topology: point-to-point topology
  - Permanent link between two **endpoints**

Network Topology Evaluation Metric



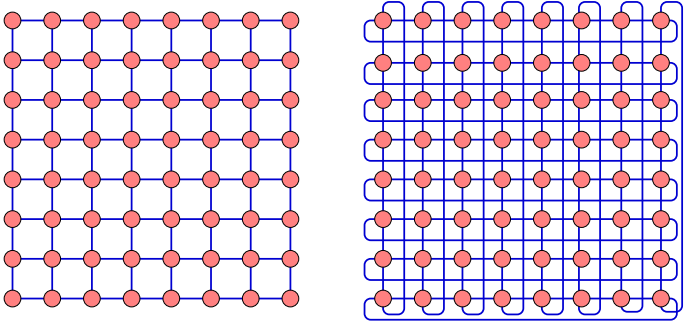
- Diameter
  - Maximum minimal distance between 2 nodes
  - Smaller is better
- Connectivity
  - The minimum number of arcs that must be removed to break it into two disconnected networks
  - Larger is better
- Bi-section width
  - The minimum number of arcs that must be removed to partition the network into two equal halves
  - Larger is better
- Bi-section bandwidth
  - Aggregate bandwidth of all links connecting two equal halves
  - Larger is better
- Costs
  - Smaller is better

Cartesian Network Topologies (1/2)

- 1-dimensional linear array and ring:



- 2-dimensional mesh and torus:



Cartesian Network Topologies (2/2)

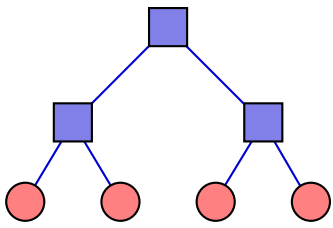


- Evaluation:

	Mesh	Torus
Diameter	$d(P^{1/d} - 1)$	$d(P^{1/d}/2)$
Connectivity	d	$2d$
Bi-section width	$P^{(d-1)/d}$	$2P^{(d-1)/d}$

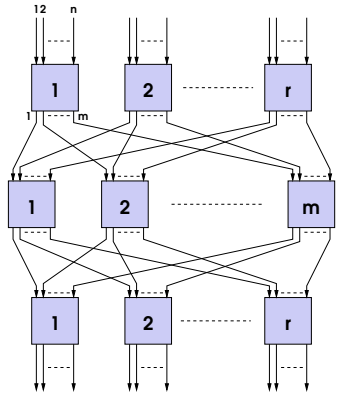
Network Topology: Tree

- Switch-based network
- Evaluation binary tree:
  - Diameter: $O(\log(P))$
  - Connectivity: 1
  - Bi-section width: 1
  - Costs: $O(P)$
- Number of links between upper levels smaller
  ☞ **Blocking network topology**



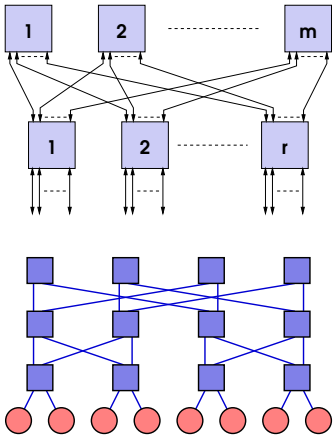
Network Topology: Clos Network

- N-stage switch-based network
  - Here: $N = 3$
- Network is characterised by
  - r: number of ingress stage switches
  - n: number of ingress ports at ingress stage
  - m: number of egress ports at ingress stage
- Evaluation:
  - Diameter: $N + 1$
  - Connectivity: m (at switch-level)
  - Bi-section width: $r\frac{m}{2}$
- A Clos network can be made **non-blocking**
  - Non-blocking for $m \geq n$ if one allows for rearrangements
  - Strictly non-blocking if $m \geq 2n - 1$



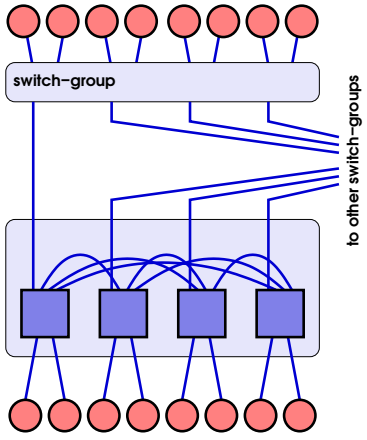
Network Topology: Folded Clos Network

- Topology obtained by folding ingress and egress stage
  - Example: folded 3-stage Clos
- Evaluation of Clos and Folded Clos network:
  - Diameter $= N + 1$
  - Connectivity $= m$
  - Bi-section width $= r\frac{m}{2}$
- Closely related to **fat-tree** topology
  - Here: similar to folded 5-stage Clos



Network Topology: Dragonfly

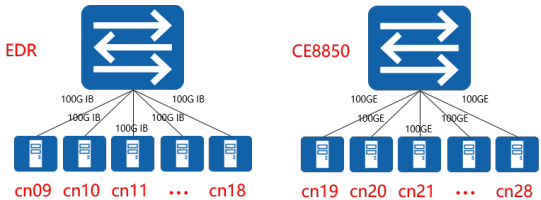
- Multi-layer network
  - Switch layer
  - Switch-group layer
  - System layer
- All-to-all connectivity within layer
- Network parameters
  - p: number of nodes per switch
  - a: number of switches per group
  - h: number of links per switch to other groups
- Evaluation:
  - Diameter: 5
  - Connectivity: $ah - b$ (at system layer; b = bundel width)
  - Bi-section width: $\left(\frac{a\,h-1}{2}\right)^2$



Example: HAICGU

- Fat-tree topology
- Two network technologies supported
  - EDR Infiniband (Mellanox MSB7890-ES2F)
  - 100GE (Huawei CloudEngine 8850)
- All nodes equipped with Mellanox ConnectX-5 adapters
- Select the Slurm partitions `cn-ib*` or `cn-eth`

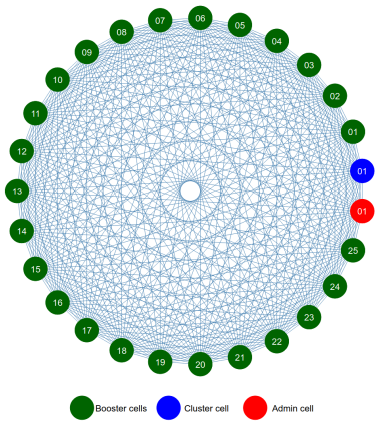
[L. Pichetti et al.; 2024]



Example: JUPITER (1/2)

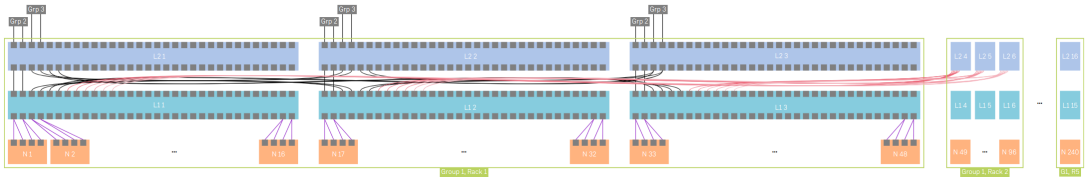
- JUPITER is the first exascale system in Europe
- Installation site: Jülich Supercomputing Centre
- Number of nodes:
  - ~6,000 GPU-accelerated nodes
  - ~1,300 CPU-only nodes
- Network topology: **Dragonfly+**
  - Fat-tree within a switch-group
  - All-to-all between switch-groups

[A. Herten; 2025]



Example: Jupiter (2/2)

[A. Herten; 2025]



- **Key benefit of dragonfly**: Moderate number of long-distance optical cables to interconnect switch-groups
  - Cables have to extend multiple containers

Content



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

MPI Introduction



- **MPI** = Message-Passing library Interface specification
  - Model: Data is moved from the address space of one process to that of another process through cooperative operations on each process
- Specification defined by MPI Forum
  - Members of forum: academic and commercial institutions
  - History:

May 1994:	Release of first version MPI-1.0
September 2009:	Release of version MPI-2.2
June 2015:	Release of version MPI-3.1
November 2023:	Release of version MPI-4.1
June 2025:	Release of version MPI-5.0

MPI Implementations



- MPI is only a **specification**, not an **implementation**
- Most popular implementations
  - OpenMPI (U Tennesse, LANL, Indiana U plus collaborators)
  - MPICH (ANL plus collaborators)
  - MVAPICH (Ohio State U plus collaborators)
- Different implementations have different advantages/disadvantages. Performance may differ on different platforms and depend on application.

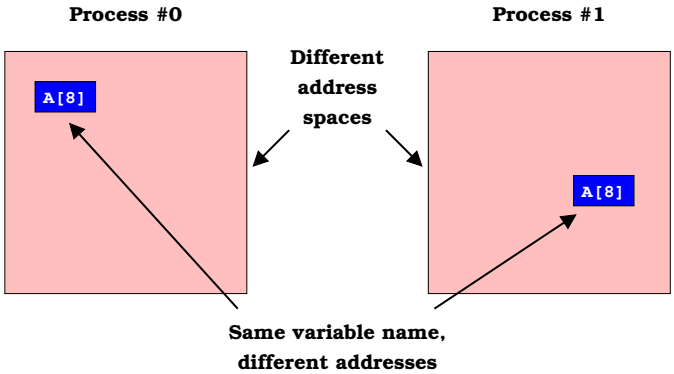
Message Passing Model (1/2)



- MPI-based parallel programs consist of a **set of separate processes**
  - A process is an active instance of a program with its own program counter and address space
  - The processes can run on the same node or on different nodes
- Typically, **all processes execute the same program**, but operate on different data
  - **SPMD** = Single Program Multiple Data
- Data has to be communicated explicitly between processes
  - Different communication patterns are supported:
    - Point-to-point communication operations
    - Collective communication operations
- Programmer has to take care of data distribution, communication and synchronisation

Message Passing Model (2/2)

Beware: Variables of the same program with the same name in different processes are different variables stored at different memory addresses



Why Using MPI? (1/2)



- **Observation**: MPI has become a standard parallel programming model and is supported on essentially all HPC systems
  - MPI replaced various other message passing solutions
- **Portability**: An application developed in a standard compliant manner on one system will run also on other systems that support an MPI implementation
  - In practice, some system specific optimizations may be required
- **Functionality**: Increasingly rich set of features
- **Wide uptake**: MPI is used for the vast majority of parallel applications leading to
  - Many training opportunities and widely available expertise
  - Broad eco-system

Why Using MPI? (2/2)



- **Availability**: MPI runs on all Linux and most other systems
  - Open source implementations are available and distributed with popular Linux distributions
- **Performance opportunities**: MPI is co-developed with HPC solution providers that perform significant efforts optimizing the software
  - Network technology providers provide optimized back-ends
  - HPC system providers provide there own implementations of MPI, e.g. Cray MPI, IBM Spectrum MPI

Content



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

First MPI Program: Hello World in C



```c
#include <stdio.h>
#include <mpi.h>

int main (int argc, char *argv[])
{
  int irank, isize;

  MPI_Init(&argc, &argv);

  MPI_Comm_rank(MPI_COMM_WORLD, &irank);
  MPI_Comm_size(MPI_COMM_WORLD, &isize);

  printf("Processor %d of %d: Hello World!\n", irank, isize);

  MPI_Finalize();

  return 0;
}
```

First MPI Program: Comments



- Need to include MPI header files or declare use of MPI module to make MPI functions, variable types and constants declared or defined
  - C or C++: `#include <mpi.h>`
  - Fortran 2008 or later: `USE mpi_f08`
  - Earlier Fortran: `USE mpi` or `INCLUDE 'mpif.h'`
- `MPI_Init` must be the first MPI function that is called (few exceptions)
- `MPI_Finalize` is required to clean-up all MPI states
- C-language binding
  - MPI used through function calls
  - All functions return an error code of type `int`

MPI Initialisation



- Single-threaded case

```
int MPI_Init(int *argc, char ***argv)
```

- Instead of the pointers `argc` and `argv` one may also provide `NULL`

```
int MPI_Init(NULL, NULL)
```

- Multi-threaded case

```
int MPI_Init_thread(int *argc, char ***argv,
                    int required, int *provided)
```

  with `required` taking the following values
  - `MPI_THREAD_SINGLE`: Only one thread will execute
  - `MPI_THREAD_FUNNELED`: Only main thread makes MPI calls
  - `MPI_THREAD_SERIALIZED`: Only one threat at a time makes MPI calls
  - `MPI_THREAD_MULTIPLE`: No restrictions

MPI Finalisation



- Routine to clean up all MPI state

```
int MPI_Finalize(void)
```

- Before calling for finalisation, a process must perform all MPI calls to complete its involvement in MPI communications

MPI Communicators



- MPI **communicators** (more precisely: intra-communicators) allow to group MPI processes
  - Processes within a communicator can communicate among each other
- MPI communicators define a communication context
  - Communications within different communicators will never be mixed up
- Predefined Communicators
  - `MPI_COMM_WORLD`: Group including all processes
  - `MPI_COMM_SELF`: Group containing only the current process

MPI Process Identification: Rank



- All processes within a communicator are assigned a unique identify called **rank**
  - Ranks are integer values starting from zero
- An MPI process may be part of multiple communicators and therefore have multiple ranks
  - Typically, rank refers to the identity of a process within the communicator `MPI_COMM_WORLD`

Compiling MPI Programs



- MPI programs should be compiled using the wrapper compilers provided by the different MPI implementations, e.g.
  - C:
    - mpicc, mpiicc
  - C++:
    - mpicxx, mpiCC, mpic++, mpiicpc
- Most wrappers allow to show what they are doing
  - Example OpenMPI: `mpicc --showme`
- May need to load modules, e.g. on HAICGU: `ml GCC OpenMPI`

Running MPI Programs



- Starting an MPI program requires a tool called `mpiexec`
  - Not standard compliant alternative: `mpirun`
- Typically, a single MPI is executed:

  ```
  mpiexec -n 8 ./hello_mpi.x
  ```

  - The option `-n` <numprocs> defines the number of processes that are started
  - Other options are MPI implementation specific
- MPI also allows to execute multiple programs, e.g.

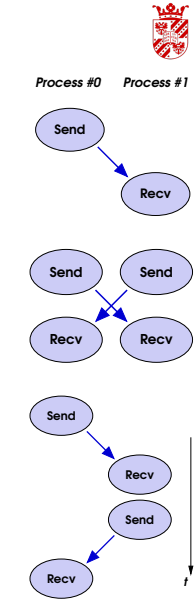
  ```
  mpiexec -n 1 ./master.x : -n 23 ./worker.x
  ```

Content



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

Digression: Communication Patterns



- Communication patterns involving a pair of processes
- **Ping**:
  - Process #0 sends a message to process #1 (Ping)
- **PingPing**:
  - Each process sends a message to the other process
- **PingPong**:
  - Process #0 sends a message to process #1 (Ping)
  - After receiving the message process #1 returns it to process #0 (Pong)

Ping in C using MPI



```c
#include <stdio.h>
#include <mpi.h>

int main(int argc ,char *argv[])
{
  int irank;
  int x, y;
  int src, dst;
  MPI_Status status;

  MPI_Init(&argc, &argv);
  MPI_Comm_rank(MPI_COMM_WORLD, &irank);

  x = 100 + irank; y = -1;
  src = 1; dst = 0;

  if (irank == src) {
    MPI_Send(&x, 1, MPI_INT, dst, 99,
             MPI_COMM_WORLD);
    printf("#%d: have sent x=%d\n",
           irank, x);
  }
  else if (irank == dst) {
    MPI_Recv(&y, 1, MPI_INT, src, 99,
             MPI_COMM_WORLD, &status);
    printf("#%d: have received y=%d\n",
           irank, y);
  }

  MPI_Finalize();

  return 0;
}
```

Standard `MPI_Send`



- Syntax of a blocking send operation:

```
int MPI_Send(
  const void* buf,         /* Pointer to send buffer */
  int count,               /* Number of elements     */
  MPI_Datatype datatype,   /* Data type              */
  int dest,                /* Rank of destination    */
  int tag,                 /* Message tag            */
  MPI_Comm comm            /* Communicator           */
);
```

- The **message buffer** is described by `buf`, `count` and `datatype`
- The **target** is identified by `dest` and `comm`
- Messages are sent with an accompanying user-defined integer **tag** that is used by receiver for identification
  - Allow send-receiver pairs to perform concurrent communications

MPI Data Types



- MPI is aware of intrinsic data types in the supported languages (C, Fortran, C++)
- Benefits
  - No need for buffer length in Bytes to be computed by programmer
  - Facilitate some support of type checking
  - Automatic data conversion, e.g.
    - Little versus big endian
    - Interfacing C and Fortran routines

MPI Predefined Data Types



MPI data type	C data type
`MPI_CHAR`	`char`
`MPI_INT`	`signed int`
`MPI_FLOAT`	`float`
`MPI_DOUBLE`	`double`

Standard `MPI_Recv`



- Syntax of a blocking receive operation:

```
1 int MPI_Receive(
2   const void* buf,        /* Pointer to receive buffer */
3   int count,              /* Number of elements        */
4   MPI_Datatype datatype, /* Data type                 */
5   int source,             /* Rank of source            */
6   int tag,                /* Message tag               */
7   MPI_Comm comm,          /* Communicator              */
8   MPI_Status *status      /* Status object             */
9 );
```

- Waits until a matching (on source and tag) message is received
  - After function returned, receive buffer can be used
- Alternative to a specific source or tag: `MPI_SOURCE_ANY` or `MPI_TAG_ANY`
- The `status` object on return contains further information
- Receiving fewer than `count` occurrences of `datatype` is OK, but receiving more is an error

Return Status Object



- Returned information
  - `MPI_SOURCE`: Source of received message
  - `MPI_TAG`: Tag of received message
  - `MPI_ERROR`: Error code
- Language binding
  - C: Structure with 3 elements, e.g. `status.MPI_SOURCE`

Error Codes



- The MPI specification leaves it to the implementations to define specific error codes, which must comply to the following rules
  - Predefined code 0 $=$ `MPI_SUCCESS`
  - Other codes must be in the range $[1, \texttt{MPI_ERR_LASTCODE}]$
- The standard defines a set of error classes and a function to translate error code to error class:

```
int MPI_Error_class(int errorcode, int *errorclass)
```

- Error class examples:

`MPI_ERR_BUFFER`	Invalid buffer pointer
`MPI_ERR_RANK`	Invalid rank

Standard `MPI_Sendrecv`



- If all processes send and receive messages (PingPong pattern) send and receive can be initiated using a single MPI call:

```
int MPI_Sendrecv(
  const void *sendbuf,    /* Send buffer          */
  int sendcount,          /* Number of elements   */
  MPI_Datatype sendtype,  /* Data type            */
  int dest,               /* Rank of destination  */
  int sendtag,            /* Send tag             */
  void *recvbuf,          /* Receive buffer       */
  int recvcount,          /* Number of elements   */
  MPI_Datatype recvtype,  /* Data type            */
  int source,             /* Rank of source       */
  int recvtag,            /* Receive tag          */
  MPI_Comm comm,          /* Communicator         */
  MPI_Status *status      /* Status object        */
);
```

Timing Measurement



- MPI defines a timer that returns time in units of seconds:

  ```
  double MPI_Wtime(void)
  ```

- Example program:

```
1 double tstart = MPI_Wtime();
2 /* stuff to be timed */
3 double tend = MPI_Wtime();
4 printf("Execution time: %.2e seconds\n",tend-tstart);
```

- Time is measured as difference to some time in the past that is guaranteed not to change during process execution
- The times returned are local to the node that called them
- Time resolution can be checked using `MPI_Wtick()`

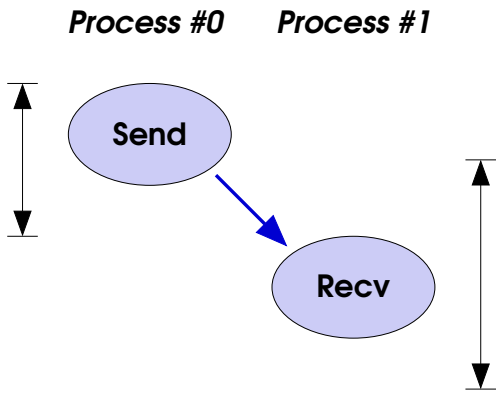
Time Measurement: Pitfall (1/3)



- Will the modified version of the MPI Ping program correctly measure the time to communicate one integer?

```
1   src = 1; dst = 0;
2
3   double tstart = MPI_Wtime();
4   if (id == src) {
5     MPI_Send(&x, 1, MPI_INT, dst, 99, MPI_COMM_WORLD);
6     printf("#%d: have sent x=%d\n", id, x);
7   }
8   else if (id == dst) {
9     MPI_Recv(&y, 1, MPI_INT, src, 99, MPI_COMM_WORLD, &status);
10    printf("#%d: have received y=%d\n", id, y);
11  }
12  double tend = MPI_Wtime();
13  printf("Time for communication: %.2es\n", tend - tstart);
```

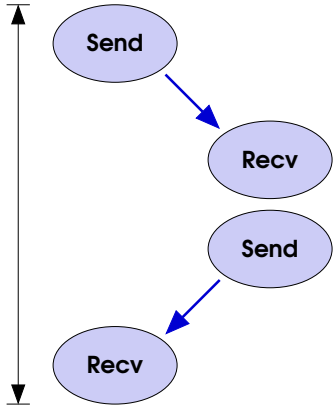
Time Measurement: Pitfall (2/3)



- **Answer**: No
- **Process #0**: measures the time it takes to complete the send operation (plus printing a string)
- **Process #1**: measures the time from an arbitrary starting point until the receive operation completed

Time Measurement: Pitfall (3/3)

- **Alternative**: Implement a PingPong communication pattern to measure **round-trip time**



Content



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

Summary Lecture #13



- Network architecture
  - Link and communication library protocols
  - Network topologies
- MPI
  - Basics
  - Blocking point-to-point communication

Questions?



[Wiki Media]



university of groningen

Performance Engineering: Lecture #14

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-10-15

Overview



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. Final

Content



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. Final

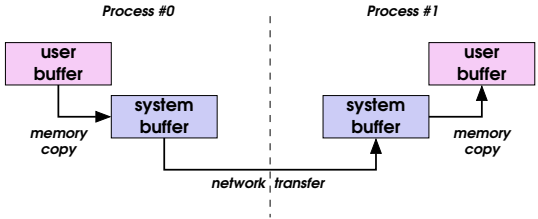
Communication Buffering (1/3)



- MPI communication involves **multiple transactions**
- **Challenges**:
  - Intermediate buffer space required
  - Buffered data must remain unchanged until buffer is completely read

Communication Buffering (2/3)

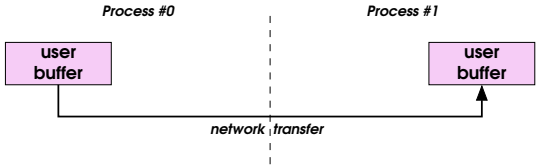
- **Design option #1**: **Intermediate memory buffer**



  - **Advantages**:
    - MPI send completes after fast in-memory copy
  - **Disadvantages**:
    - Need additional space in memory
    - Performing memory copies increases pressure on memory bus

Communication Buffering (3/3)

- **Design option #2**: **Zero-copy communication**



  - **Advantages**:
    - No additional memory space and less memory traffic
  - **Disadvantages**:
    - MPI send only completes once data has been transferred over network

MPI Buffered Mode



- User can explicitly request communication to be buffered (**buffered mode**):

```
1 int MPI_Bsend(
2   const void* buf,        /* Pointer to send buffer */
3   int count,              /* Number of elements     */
4   MPI_Datatype datatype,  /* Data type              */
5   int dest,               /* Destination rank       */
6   int tag,                /* Communication tag      */
7   MPI_Comm comm           /* Communicator           */
8 );
```

- A buffered mode send operation can be started whether or not a matching receive has been posted

MPI Buffered Mode: User-provided Buffer



- A user may specify a buffer to be used for buffering messages sent in buffered mode:

```
1 int MPI_Buffer_attach(
2   void* sysbuf,   /* Pointer to user allocated system buffer */
3   int size        /* Size of buffer in Bytes                 */
4 );
5
6 int MPI_Buffer_detach(
7   void* sysbuf,  /* Returned pointer to system buffer       */
8   int* size      /* Returned size of buffer                 */
9 );
```

Blocking versus Non-Blocking Communication



- **Blocking**
  - Function returns only after completion of the associated operation
  - Examples: `MPI_Send`, `MPI_Recv`
- **Non-blocking**
  - Function may return before the associated operation has completed
  - Examples:
    - Immediate send: `MPI_Isend`
    - Immediate receive: `MPI_Irecv`
  - Resources (e.g. message buffers) passed to function must not be reused until the operation has completed
    - Non-blocking functions return a handle that allow to query status of the initiated operation

MPI Immediate Send and Receive



- Syntax of immediate send operation

```
int MPI_Isend(
  const void* buf,          /* Send buffer                */
  int count,                /* Number of elements         */
  MPI_Datatype datatype,    /* Data type                  */
  int dest,                 /* Destination rank           */
  int tag,                  /* Communication tag          */
  MPI_Comm comm,            /* Communicator               */
  MPI_Request *request      /* Request handle             */
);
```

- Syntax of immediate receive operation

```
int MPI_Irecv(
  const void* buf,          /* Pointer to receive buffer */
  int count,                /* Number of elements        */
  MPI_Datatype datatype,    /* Data type                 */
  int source,               /* Rank of source            */
  int tag,                  /* Message tag               */
  MPI_Comm comm,            /* Communicator              */
  MPI_Request *request      /* Request handle            */
);
```

Waiting for Completion (1/2)



- To wait for a communication identified by a given request handler to complete, the blocking function `MPI_Wait` can be used:

```
1 int MPI_Wait(
2   MPI_Request *request,   /* Request handle (in/out)   */
3   MPI_Status *status      /* Status object (out)       */
4 );
```

  - On return the status object is updated

- `MPI_Test` is available to perform checks without block:

```
1 int MPI_Test(
2   MPI_Request *request,    /* Request handle (in/out) */
3   int *flag,               /* Completion flag (out)   */
4   MPI_Status *status       /* Status object (out)     */
5 );
```

  - The returned `flag` is set to true when operation completed

Waiting for Completion (2/2)



- Use `MPI_Waitall` when waiting for a set of requests to complete:

```
1 int MPI_Waitall(
2   int count,               /* Array size              */
3   MPI_Request request[],   /* Array of requests       */
4   MPI_Status status[]      /* Array of status objects */
5 );
```

MPI Modes Overview (1/2)



- **Standard mode:**
  - A send operation is started whether or not a matching receive has been posted
  - It may complete before a matching receive is posted
- **Synchronous mode:**
  - A send operation can be started whether or not a matching receive has been posted
  - The send operation send will complete successfully only if a matching receive is posted
- **Ready mode:**
  - A send operation may be started only if the matching receive is already posted
- **Buffered mode:**
  - A send operation can be started whether or not a matching receive has been posted
  - It may complete before a matching receive is posted

MPI Modes Overview (2/2)



The following variants of MPI send are available:

Mode	Blocking variant	Non-blocking variant
Standard	`MPI_Send`	`MPI_Isend`
Synchronous	`MPI_SSend`	`MPI_Issend`
Ready	`MPI_RSend`	`MPI_Irsend`
Buffered	`MPI_BSend`	`MPI_Ibsend`

Content



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. Final

MPI One-Sided Communication



- Send-receive communication assumes that both end-points of the communication take action (**two-sided communication**)
- MPI since version 2.0 supports also communication based on a get/put semantics with only one end-point being active (**one-sided communication**)
  - Also called **remote memory access (RMA)**
- Challenges:
  - Need for a common address space
  - Suitable synchronisation to ensure RAW and WAR dependencies to be respected

One-Sided Communication: Basic Operations



- Create a memory window for remote access
- Communicate using, e.g.,
  - `MPI_Put`: Write to remote memory
  - `MPI_Get`: Read from remote memory
- Synchronise using `MPI_Win_fence`

Memory Window Creation



- Specify a window of existing memory that processes exposes to RMA accesses:

```c
int MPI_Win_create(
  void *base,      /* Initial address of window                    */
  MPI_Aint size,   /* Size of window [Bytes]                       */
  int disp_unit,   /* Local unit size for displacements [Bytes]    */
  MPI_Info info,   /* Info object                                  */
  MPI_Comm comm,   /* Communicator                                 */
  MPI_Win *win     /* Window handle (out)                          */
);
```

- Operation is a collective call, all processes must participate
- All address computations are scaled by the `disp_unit`
  - Choose `disp_unit` $= 1$ for no scaling
- The `MPI_Info` object allows to provide optimization hints (not covered here)
  - `MPI_Info` allows to store an unordered set of (key,value) pairs
  - May use `info` $=$ `MPI_INFO_NULL`

Synchronisation



- All RMA calls related to a particular window can be synchronised using

```
int MPI_Win_fence(
  int assert,   /* Program assertion */
  MPI_Win win   /* Window handle     */
);
```

- The application may make assertions, which may allow for optimizations but `assert` $= 0$ is always valid
- `MPI_Win_fence` separates different **access epochs**
  - Epoch starts with an RMA synchronisation call on `win`
  - Next, a number of communication calls may happen
  - Epoch stops with another synchronisation call
- Other synchronisation mechanisms available (not considered here)

MPI_Put (1/2)



- The analogue of a send operation is the following operation that puts data into remote memory:

```
1  int MPI_Put(
2    void *oaddr,              /* Address of origin buffer                    */
3    int ocount,               /* Number of entries in origin buffer          */
4    MPI_Datatype odatatype,   /* Data type of entries in origin buffer       */
5    int trank,                /* Target rank                                 */
6    MPI_Aint tdisp,           /* Displacement from window start to target buffer */
7    int tcount,               /* Number of entries in target buffer          */
8    MPI_Datatype tdatatype,   /* Data type of entries in origin buffer       */
9    MPI_Win win               /* Window handle                               */
10 );
```

- The data is
  - Read from the local memory at address `oaddr`
  - written to the remote memory at address
    `taddr` $=$ `window_base` $+$ `tdisp` $\times$ `disp_unit`

MPI_Put (2/2)



- Need synchronisation to avoid
  - Remote data is overwritten before read of old data (WAR)
  - Remote data is used before new data is written (RAW)

MPI_Get



- To copy data from the target memory to the origin use

```
1  int MPI_Get(
2    void *oaddr,              /* Address of origin buffer                    */
3    int ocount,               /* Number of entries in origin buffer          */
4    MPI_Datatype odatatype,   /* Data type of entries in origin buffer       */
5    int trank,                /* Target rank                                 */
6    MPI_Aint tdisp,           /* Displacement from window start to target buffer */
7    int tcount,               /* Number of entries in target buffer          */
8    MPI_Datatype tdatatype,   /* Data type of entries in origin buffer       */
9    MPI_Win win               /* Window handle                               */
10 );
```

- Need synchronisation to avoid remote data is
  - **Updated** before get has started (RAW)
  - **Not updated** before get completed (WAR)

One-Sided Communication: Ping (1/2)



```
1  int main(int argc ,char *argv[]) {
2    MPI_Init(NULL, NULL);
3
4    int irank;
5    MPI_Comm_rank(MPI_COMM_WORLD, &irank);
6
7    int buf = -1; MPI_Win win;
8    MPI_Win_create(&buf, sizeof(int), sizeof(int), MPI_INFO_NULL, MPI_COMM_WORLD, &win);
9
10   int src = 1; int dst = 0;
11   MPI_Win_fence(0, win);
12   if (irank == src) {
13     int x = 100 + irank;
14     MPI_Put(&x, 1, MPI_INT, dst, 0, 1, MPI_INT, win);
15     printf("#%d: have put x=%d\n", irank, x);
16   }
17   MPI_Win_fence(0, win);
18
19   printf("#%d: buf=%d\n", irank, buf);
20
21   MPI_Finalize();
22   return 0;
23 }
```

One-Sided Communication: Ping (2/2)



Question: What is the expected output?

One-Sided Communication: Ping (2/2)



Question: What is the expected output?

When using 4 processes (order is not deterministic):

```
#1: have put x=101
#1: buf=-1
#3: buf=-1
#0: buf=101
#2: buf=-1
```

Content



1. Non-Blocking Communication
2. One-Sided Communication
3. **Collective Communication**
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. Final

MPI Collective Operations: Overview



- MPI defines 9 types of collective operations:
  - Barrier
  - Broadcast
  - Gather
  - All-Gather
  - Scatter
  - All-to-all
  - Reduce
  - All-Reduce
  - Reduce-Scatter
  - Scan
- Collective communication is over all of the processes in particular group identified by a communicator
- Standard meanwhile introduced blocking and non-blocking variants

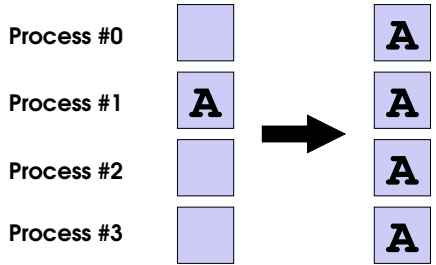
MPI Barrier



- Function blocks the caller until all group members (identified by the communicator `comm`) have called it
  - A deadlock occurs if one group member does not call the function
- Syntax:

  ```
  int MPI_Barrier(MPI_Comm comm)
  ```
- Practical advice
  - In a program with send-receive and other collective communications, an MPI barrier is rarely needed
  - MPI makes no guarantees on how long it will take other processes to leave the barrier
    - MPI barriers are not appropriate for highly accurate time measurements

MPI Broadcast

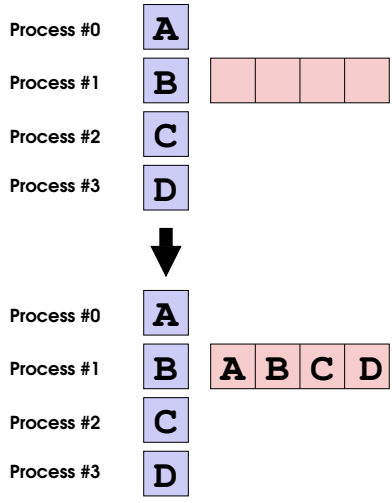


- Function copies data from process identified by `root` and `comm` to all other processes of the group
- Example with 4 processes and `root=1`:
- Syntax:

```
1 int MPI_Bcast(
2   void* buffer,           /* Pointer to output/input buffer */
3   int count,              /* Number of elements             */
4   MPI_Datatype datatype,  /* Data type                      */
5   int root,               /* Rank of root process           */
6   MPI_Comm comm           /* Communicator                   */
7 );
```

MPI Gather (1/2)

- Function collects data on process identified by `root` and `comm` sent by all processes of the group (in rank order)



MPI Gather (2/2)

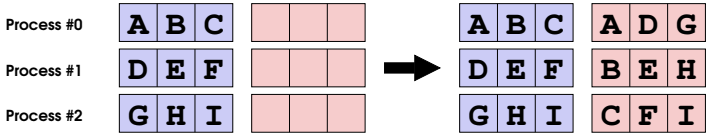


- Syntax:

```
int MPI_Gather(
  const void* sendbuf,    /* Pointer to output buffer  */
  int sendcount,          /* Number of elements        */
  MPI_Datatype sendtype,  /* Data type                 */
  void* recvbuf,          /* Pointer to input buffer   */
  int recvcount,          /* Number of elements        */
  MPI_Datatype recvtype,  /* Data type                 */
  int root,               /* Rank of receiving process */
  MPI_Comm comm           /* Communicator              */
);
```

MPI All-to-all

- Each process sends distinct data to each of the other processes within the group

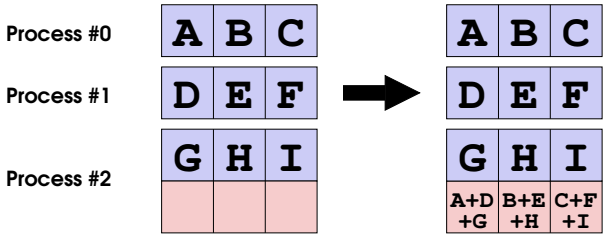


- Syntax:

```
1 int MPI_Alltoall(
2   const void* sendbuf,     /* Pointer to output buffer */
3   int sendcount,           /* Number of elements       */
4   MPI_Datatype sendtype,   /* Data type                */
5   void* recvbuf,           /* Pointer to input buffer  */
6   int recvcount,           /* Number of elements       */
7   MPI_Datatype recvtype,   /* Data type                */
8   MPI_Comm comm            /* Communicator             */
9 );
```

MPI Reduce (1/3)

- Function combines the elements provided in the input buffer of each process in the group, using the operation op, and returns the combined value in the output buffer of the process with rank `root`



MPI Reduce (2/3)



- Syntax:

```
int MPI_Reduce(
 const void* sendbuf,    /* Pointer to output buffer */
 void* recvbuf,          /* Pointer to input buffer */
 int count,              /* Number of elements */
 MPI_Datatype datatype, /* Data type */
 MPI_Op op,              /* Reduction operator */
 int root,               /* Rank of root process */
 MPI_Comm comm           /* Communicator */
);
```

MPI Reduce (3/3)



- MPI's predefined reduction operations:

Name	Meaning
MPI_MAX	maximum
MPI_MIN	minimum
MPI_SUM	sum
MPI_PROD	product
MPI_LAND	logical and
MPI_BAND	bit-wise and

Name	Meaning
MPI_LOR	logical or
MPI_BOR	bit-wise or
MPI_LXOR	logical exclusive or (xor)
MPI_BXOR	bit-wise exclusive or (xor)
MPI_MAXLOC	max value and location
MPI_MINLOC	min value and location

MPI Reduce Example: Global Sum



```c
int main() {
    const int N=1048576;
    int isize, irank;

    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &irank);
    MPI_Comm_size(MPI_COMM_WORLD, &isize);

    int n = N / isize;  /* Assume N to be multiple of isize */
    int *v = (int *) calloc(n, sizeof(int));

    for (int i = 0; i < n; i++) v[i] = irank * n + i + 1;

    double s = 0;       /* Local sum */
    for (int i = 0; i < n; i++) s += v[i];

    double gs = -1;     /* Global sum */
    MPI_Reduce(&s, &gs, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (irank == 0) printf("N=%d, gs=%.2f\n", N, gs);

    free(v);
    MPI_Finalize();

    return 0;
}
```

Non-Blocking Collectives



- Collective operations are typically also available in a non-blocking variant
  - Examples
    - `MPI_Ibcast`
    - `MPI_Ireduce`
- These functions return a request handle such that `MPI_Wait` or `MPI_test` can be used to check for completion

Content



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. Final

P2P Communication Optimization: Introduction



- Optimization strategies:
  - Latency hiding
  - Bandwidth maximisation
    - Reduce communication overhead by one-sided communication (not further considered here)
    - Manual data packing
    - Use of MPI's derived data types
  - Optimized mapping to physical network topology

Digression: Software Pipelining



[V.H. Allan et al.; 1995]

- **Software pipelining** = Optimization technique where tasks from different loop iterations are overlapped
- **Goal**: Perform tasks (or instructions) on which other tasks (instructions) depend as early as possible
  - Example: Start send well before data needs to be received

Pipelining: Blocking Communication



- **Assumption**: Routine comprising a routine where in each iteration
  - Data is send to another node
  - Data is received from another node
  - Received data is processed

- **Example algorithm**:

  send $a^{(0)}$;
  for $i = 0$; $i < N - 1$; $i++$ **do**
    send $a^{(i+1)}$;
    receive $a^{(i)}$;
    process $a^{(i)}$;
  end
  receive $a^{(N-1)}$;
  process $a^{(N-1)}$;

Pipelining: Non-blocking Communication



- **Same assumption** but now using non-blocking communication

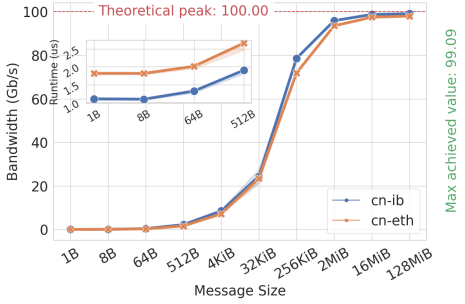
- **Example algorithm**:

$h_0 =$ immediate send $a^{(0)}$;
for $i = 0$; $i < N-1$; $i++$ **do**
  if $i < N-2$ **then**
    $h_{(i+1)\%2} =$ immediate send $a^{(i+1)}$;
  end
  wait $h_{i\%2}$;
  receive $a^{(i)}$;
  process $a^{(i)}$;
end
wait $h_{(N-1)\%2}$;
receive $a^{(N-1)}$;
process $a^{(N-1)}$;

Bandwidth versus Message Size

- The effective bandwidth **depends on the message size**
- Root causes:
  - Limited message rate
  - Relative impact of communication overhead (e.g. message header) higher for small messages

Example: HAICGU cluster



[L. Pichetti et al.; 2024]

Optimization: (Manual) Data Packing



- **Idea**:
  - Copy **scattered** data items **continguously** into a communication buffer
  - Send a set of data in a single message
- **Advantage(s)**:
  - Less and larger messages
- **Disadvantage(s)**:
  - Memory copy operations potentially expensive
  - Data copy cannot easily be overlapped with data processing

Derived Data Types



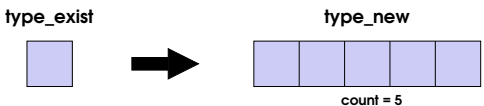
- A **derived data** type (also called general data type) is an opaque object that specifies two things:
  - A sequence of basic data types
  - A sequence of integer (byte) displacements
- Creating derived data types involves the following steps:
  - Definition of derived data types through type constructor functions
  - Commit of the new type
  - Release of all types

Derived Data Types: Contiguous Type



- Constructor allows replication of a data type into contiguous locations:

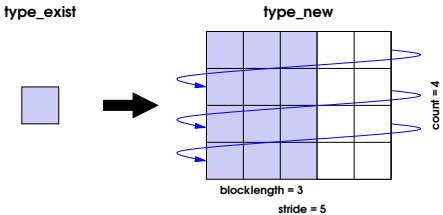
```
1 int MPI_Type_contiguous(
2   int count,                  /* Replication count  */
3   MPI_Datatype type_exist,    /* Existing data type */
4   MPI_Datatype *type_new      /* New data type      */
5 );
```



Derived Data Types: Vector Type



- `MPI_Type_vector` is a more general constructor that allows replication of a data type into locations that consist of equally spaced blocks:




```
1  int MPI_Type_vector(
2    int count,          /* Number of blocks          */
3    int blocklength,    /* Number of elements in
4                           each block                */
5    int stride,         /* Number of elements
6                           between start of
7                           each block                */
8
9    MPI_Datatype type_exist,/* Existing data type */
10   MPI_Datatype *type_new  /* New data type      */
11 );
```

Process Topologies



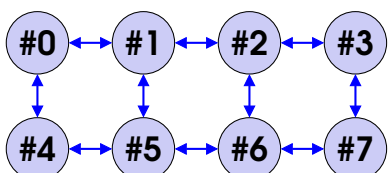
- Communicators can be used to define a **virtual network topology** that defines the neighbourship relations between the processes within the given group
- Benefits of such virtual topologies:
  - Allow to express communication patterns in a more natural way
  - Provide middleware with hints about target communication patterns
    - MPI may use this information to select a better mapping of processes to underlying **physical network topology** (if user allows for rank re-ordering)

Process Topologies: Cartesian (1/2)

- With a Cartesian communicator processes are assumed to be arranged on a one or higher-dimensional mesh or torus

```
1 int MPI_Cart_create(
2   MPI_Comm comm,          /* Existing communicator          */
3   int ndims,              /* Number of dimensions of grid   */
4   const int dims[],       /* Grid size                      */
5   const int periods[],    /* True (false) if grid is (not)
6                              periodic                       */
7   int reorder,            /* Allow to reorder or not        */
8   MPI_Comm *comm_cart     /* New communicator               */
9 );
```

- Example with 8 processes and
  - `ndims` $= 2$
  - `dims` $= [4, 2]$
  - `periods` $= [0, 0]$
  - `reorder` $= 0$



Process Topologies: Cartesian (2/2)



- MPI provides a (convenience) function to compute source and destination to communication with the grid:

```
1 int MPI_Cart_shift(
2   MPI_Comm comm_cart, /* Cartesian communicator                    */
3   int direction,      /* Communication direction                   */
4   int disp,           /* Displacement (sign determins direction)   */
5   int *source,        /* Returned source rank                      */
6   int *dest           /* Returned destination rank                 */
7 );
```

  - The returned `source` and `dest` ranks can be directly used as input for `MPI_Sendrecv`

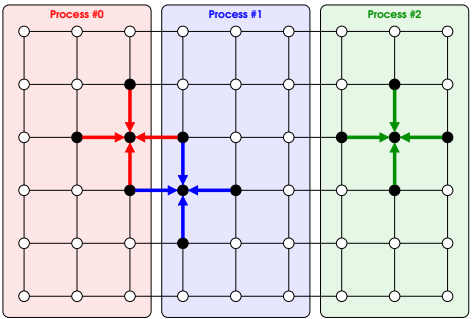
Content



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. **Case Study: Halo Update (1)**
6. Final

Solving the 2D Poisson in Parallel

- Graphical representation of a parallel version of the discrete 2-dimensional Poisson equation:

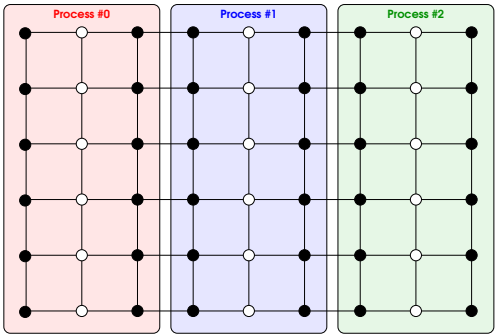


- Local update may require data from remote process

2D Poisson: Halo versus Bulk

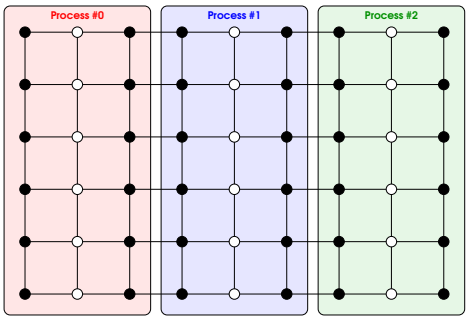
- Classification of grid points:
  - Inner points (**bulk**): No data from other processes needed
  - Boundary points (**halo**): Data from other processes needed

- Parallel update requires **halo exchange**

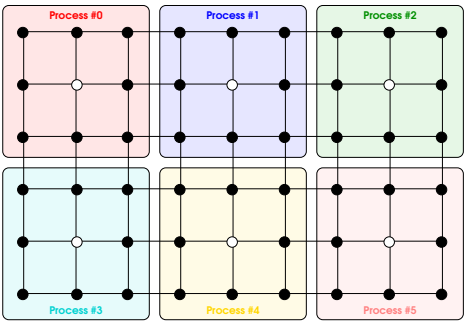


2D Poisson Equation: Data Decomposition

Data decomposition in 1 dimension:



Data decomposition in 2 dimensions:



2D Poisson Equation: Information Exchange



- **Assume** a square lattice of size L^2 being distributed over P processes and use of double-precision numbers
- Update of each site requires 8 Flop:

$$I_{\mathrm{fp}}(L,P) = (8 \cdot L^2/P)\,\mathrm{Flop}$$

- In each iteration, the field v is updated and the local halo needs to be communicated
  - Data decomposition in 1 dimension:

$$I_{\mathrm{net}}^{(1\mathrm{d})}(L,P) = (2 \cdot L \cdot 8)\,\mathrm{B}$$

  - Data decomposition in 2 dimensions:

$$I_{\mathrm{net}}^{(2\mathrm{d})}(L,P) \simeq \left(4 \cdot (L/\sqrt{P} - 1) \cdot 8\right)\,\mathrm{B}$$

2D Poisson Equation: Data Decomposition Analysis



- For fixed P we find

$$\frac{I_{\mathrm{fp}}}{I_{\mathrm{net}}} \propto L$$

  - Increasing L the amount of computation relative to network communication increases
- For fixed L we find

$$\frac{I_{\mathrm{net}}^{(1\mathrm{d})}}{I_{\mathrm{fp}}} \propto P, \qquad \frac{I_{\mathrm{net}}^{(2\mathrm{d})}}{I_{\mathrm{fp}}} \propto \sqrt{P}$$

  - 2-dimensional data decomposition requires less data to be communicated (for sufficiently large L)
  - But: In higher dimensions halo becomes fragmented in memory

Content



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. **Final**

Summary Lecture #14



- Finished introduction to MPI: blocking two-sided, one-sided communication, collective communication
- P2P communication optimization
  - Latency hiding
  - Bandwidth maximisation
  - Optimized mapping to physical network topology: data packing, use of MPI derived data types
- Case study on halo update

Questions?



[Wiki Media]



university of groningen

Performance Engineering: Lecture #15

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-10-19

Overview



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

2D Poisson Equation: Optimizations (1/2)



- **Optimization strategy #1**: **Overlap communication and computation**
  - Steps:
    1. Initiate communication of boundary points
    2. Update interior points
    3. Wait for communications to complete
    4. Update boundary points
  - Assumptions: $\Delta t(L) = \max\left[\Delta t_{\mathrm{fp}}(L), \Delta t_{\mathrm{net}}(L)\right]$ and latency-bandwidth model holds
    - Problem becomes local performance bound for large L as

$$\Delta t \to \Delta t_{\mathrm{fp}}(L) \quad \text{for} \quad L \to \infty$$

2D Poisson Equation: Optimizations (2/2)



- **Optimization strategy #2**: **Minimise number of send-receive operations**
  - Copy halo into single buffer before communication (manual data packing)
  - Alternative solution: Use derived data types
- Performance benefits difficult to predict as message rates improved
  - But: Higher network bandwidth demands due to additional overhead (message headers, minimal message length)
- **Optimization strategy #3**: **Use Cartesian process topology**

Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

Introduction



- Optimization and performance modelling strategies
  - Assess complexity of collective communication operations
  - Reduce number of collective communication operations
  - Avoid single processes being delayed

Complexity of Selected Collectives



- **Barrier** and **reduce**
  - Typical implementation: Binary tree
  - **Complexity**: $\log_2 P$
- **Gather** or **scatter**
  - One process receives (sends) data from (to) $P - 1$ other nodes
  - **Complexity**: P

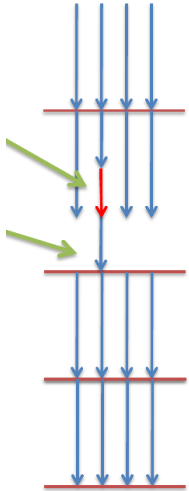
☞ **Collectives limit scalability**

- **Possible mitigation strategy**: Reduce number of collectives
  - Synchronisation only where strictly necessary
  - Data packing

OS Noise: Barriers

- **Problem**: A single delayed process makes all other processes wait
  - Example: barrier
- **Possible root cause**: OS delays execution of individual processes
  - Example: Daemons, interrupts, etc.

☞ **OS noise**

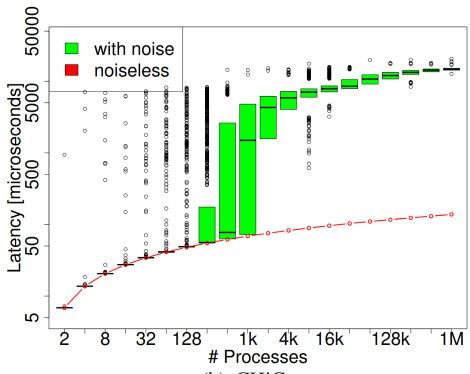


[Hoefler et al.; 2012]

OS Noise: Allreduce

- Analysis of `MPI_Allreduce` on a cluster lacking HPC optimizations at OS-level

[Hoefler et al.; 2010]



(b) CHiC

Content



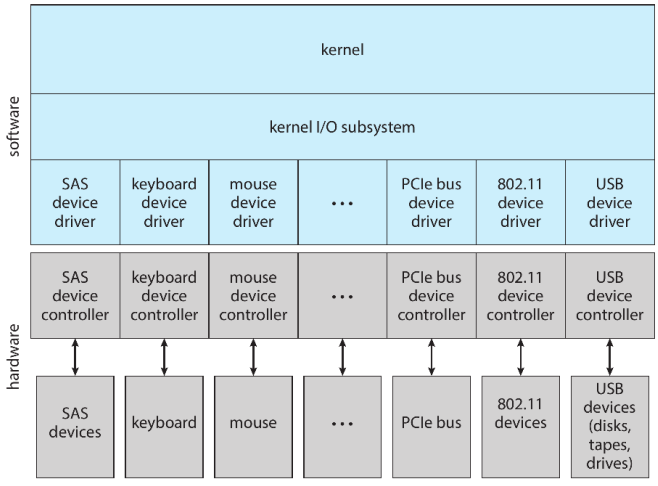
1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. **Storage Software: OS**
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

I/O Interface: OS Perspective



- **OS perspective**:
  - The operating system (OS) provides an **interface layer** between **applications** and underlying **I/O hardware**
  - I/O performance characteristics depend crucially on the OS
- **Approach** to interface layer design
  - **Generic classes** of I/O system calls to access devices with common behaviours
  - **Layered design** hides differences among I/O controllers from higher layers in the kernel
- **Challenges**
  - Each OS has its own I/O subsystem structures and device driver framework
    - Focus here: Linux

A Kernel I/O Structure



I/O Devices Types



- Character devices
- **Block devices** ⇐ **Most relevant for storage**
- Network devices

Block Devices



- **Block device** = An I/O device that is **randomly accessible** with **block-size chunks** being the smallest I/O unit
- By using block sizes $\gg 1\,\mathrm{B}$ makes this I/O device suitable if larger amounts of data are involved
- **Examples**: hard disk drives (HDD), solid-state drives (SSD)
- **Interface semantics**: read, write, seek

Kernel I/O Scheduling



- I/O requests are put in a **wait queue**
- The kernel is responsible for **request scheduling**
  - Scheduling means determining a **good request order** for **I/O request execution**
- Scheduling criteria
  - Improve overall system performance
  - Ensure fairness
  - Reduce average waiting times
  - Enforce quality of service (QoS)
- Tuning the scheduler for performance is beyond the scope of this lecture

I/O Buffering



- **Buffer** = Memory area used to store data being transferred
  - Transfer between 2 devices or between application and device
- What is the benefit of buffering? Why is buffering required?
  - Cope with **speed-mismatch** between data producer and consumer
  - Support adaptions required in case of **data transfer sizes discrepancies**
  - Optimise **blocking calls**
    - Example: Copy application data first to memory, return, and write data asynchronously to disk
- The OS typically provides means for I/O buffering
- **Buffering can be part of an performance optimization strategy**

I/O Caching



- **Cache** = Hardware or software component that transparently manages a copy of the data for faster access
  - **Transparent** means
    - The accessor observes the same functional behaviour as for accessing the original data
    - The cache manages coherence between original data and copies in the cache
- Important features for caches being beneficial
  - Access to the cache is much faster
  - Spatial and temporal localities make reuse of the cached data likely
- **Caching can be part of an performance optimization strategy**

Content

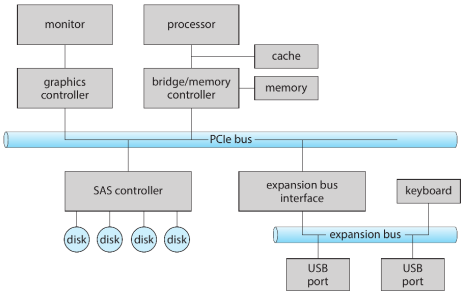


1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. **Storage Hardware**
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

I/O System Example



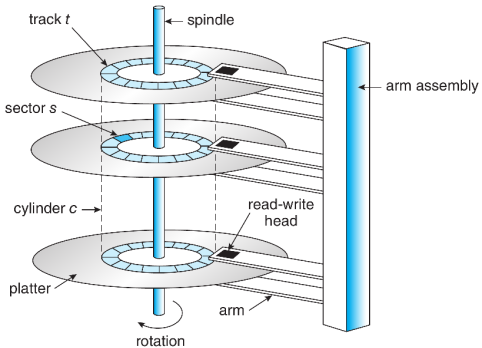
- Computer system with I/O devices connected via an **PCIe bus**
  - **PCI Express (Peripheral Component Interconnect Express)** is a high-speed serial computer expansion bus standard
  - Current generation **PCIe 5.0** supports up to **32 GT/s per lane**
  - Note: At the physical level, PCIe is based on point-to-point connections
- **Storage devices** can be attached through some intermediate controllers
  - Example: SAS controller



Storage Devices: Hard Disk Drives (HDD) (1/2)



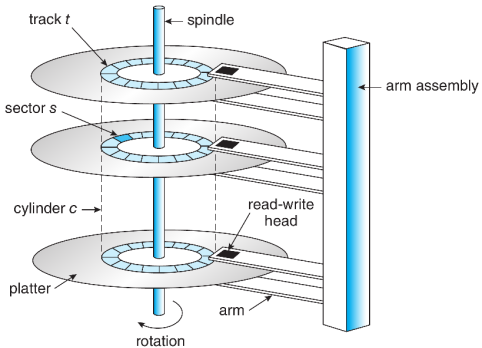
- Hardware organization
  - **Disk platter**: Flat circular shape covered with a magnetic material
  - **Heads** attached to a reading arm
  - Organization of the platters: Circular **tracks** subdivided in **sectors**
  - **Cylinder**: Set of tracks at the same position but different platters



Storage Devices: Hard Disk Drives (HDD) (2/2)



- Typical performance features
  - **Seek time**: Time to position head
    - Up to $10\,\text{ms}$
  - **Rotational latency**: Time for target sector to appear at the head
    - About $2\,\text{ms}$ to $5\,\text{ms}$
  - **Data rate**: Data transfer rate
    - $O(100)\,\text{MB}\,\text{s}^{-1}$



Storage Devices: Non-Volatile Memory (NVM)

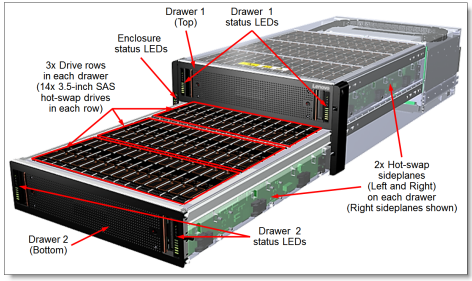


- Storage devices based on non-volatile memory technologies
  - 3D XPoint (discontinued)
  - NAND Flash
    - NAND Flash is organised in **blocks** comprising multiple **pages**
    - Pages cannot be overwritten, have to first erase blocks
      - To avoid immediate erasure, pages are marked as invalid
- **Advantages**:
  - Very high sequential access bandwidth ($> 5\,\mathrm{GB\,s^{-1}}$ for read)
  - Low access latencies and high bandwidth for random access
  - Low power at rest
- **Disadvantages**:
  - Less capacity per drive
  - Limited endurance

Storage Devices: JBOD

- **JBOD** = Just a Bunch of Disks is a collection of disks in a computer system combined as one logical volume
- Design goals
  - High storage density
  - Reasonable large aggregate bandwidth

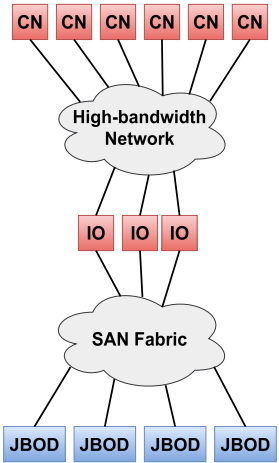
Example: Expansion enclosure, up to 84 drives, $3\times$ 12 Gb SAS $\times 4$



[Lenovo; 2025]

Network-attached, Parallel Storage

- Storage is often **network-attached**
- Typical HPC design: Compute cluster + storage cluster
  - Set of compute nodes connected to a set of storage nodes
  - Storage nodes connected to a set of JBODs
  - Optional: Tape backend



Storage Technologies: RAID



- **RAID** = Redundant Array of Independent Disks
  - Multiple physical disks are combined into a single unit to form one logical disk
  - Interface typically implemented by a controller in hardware
- **Advantages** of using RAID
  - **Protection against data-loss** due to redundant copy of data
  - **Performance increase** by spreading data over multiple disks (**striping**)
- Different RAID schemes ("levels"): RAID0, ..., RAID6

Level	$\min(n)$	Space efficiency	Fault tolerance
RAID1	2	$1/n$	$n-1$
RAID5	3	$1-1/n$	1
RAID6	4	$1-2/n$	2

n ... number of disks

Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. **Storage Software: File Systems (and others)**
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

File Concept



- **File** = A named collection of related information
  - An **abstraction** that provides a way for storing and retrieving information
  - The size of a file is not assumed to be fixed
- User perspective: **Smallest allotment of storage**
  - Files are typically stored on a storage device
  - Files may be created in memory

File System (1/2)



- **File system** = A file system is a technology that an operating system uses to manage files and the available storage resources
- A file system bridges the **logical structure** exposed to the user and the **physical structure** of the hardware
- File systems have been primarily designed to enable the use of non-volatile storage devices
- Key hardware features of such devices
  - Data can be (logically) **replaced in place**
  - **Random accessibility**
  - Transfer between storage and memory in units of **blocks**
    - Block sizes typically in the range between $512\,\mathrm{B}$ to $4096\,\mathrm{B}$

File System (2/2)



- Performance-critical aspects
  - **Metadata**: How to store information about the file
  - **Efficiency**: How to
    - Use the available storage space efficiently
    - Ensure that reading and writing data is as fast as possible
- File systems have been optimized wrt the performance characteristics of available hardware

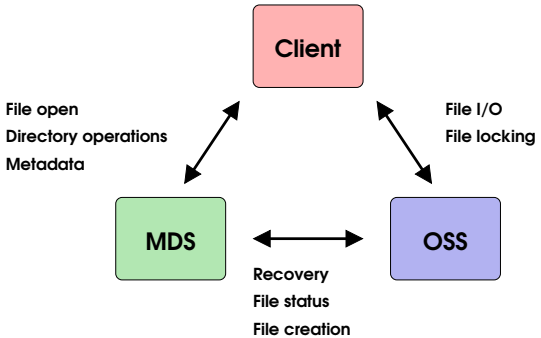
Parallel File Systems: Lustre (1/2)



- To reach very high performance and large capacity, parallel storage architectures based on various network-attached I/O servers are needed
  - Widely used example: **Lustre**
- Lustre features
  - POSIX compliant
  - Scales to a large number of clients
    - Current practical range: 100-100000
  - File system scalability
    - Current practical range: up to 512 PBytes, 4 billion files
  - Claims to have no single point of failure
  - Support of multiple network protocols

Parallel File Systems: Lustre (2/2)

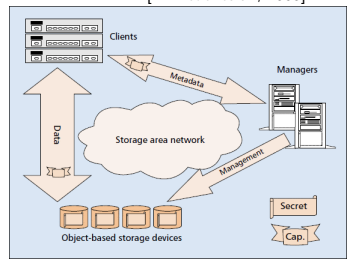
- Key components:
  - **Metadata Server (MDS)**: Server which manages the names and directories in the file system
  - **Object Storage Servers (OSS)**: Server which provides access to the storage
  - **Client**: Software running on compute nodes that mount the Lustre file system



Object Stores

- **Storage abstraction**: Data is exposed and managed as objects
  - Data is not managed as files or blocks
  - Lower layers of the storage architecture likely still manage data as blocks
  - A file system may be deployed on top of an object store
- Objects are typically organised in buckets
  - Unique bucket identifier (if applicable)
  - Object identifier unique within a bucket or across the full system
- **Assumed usage**: Single client writes object once, many clients cause many reads
- Popular **object store solution**: **Ceph**

[E. Riedel et al., 2003]



Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

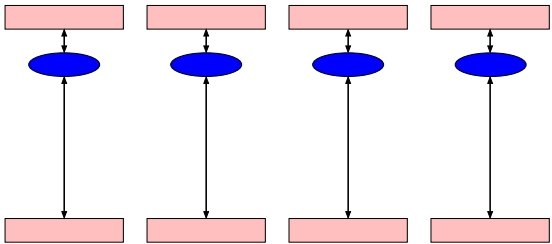
POSIX



- **POSIX file system** = A file system that provides the standard set of file operations, metadata, and behavioral semantics defined by the POSIX standard
  - POSIX = Portable Operating System Interface
  - First version of the **POSIX standard** released by IEEE in 1988
- **Key features**
  - Streaming I/O semantics (write to or read from data streams)
  - Synchronous I/O
- **Shortcomings** from a performance engineering perspective
  - Strict consistency model affects performance
  - High abstraction level makes it agnostic of the underlying hardware
  - Lacking support of asynchronous and parallel I/O
  - Metadata handling can become challenging

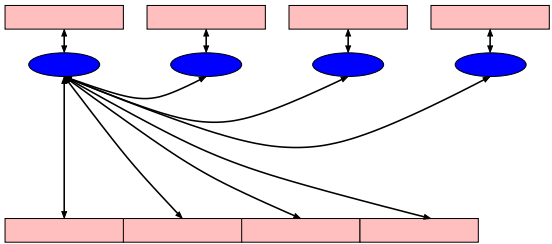
Digression: Parallel Case / Independent I/O

- **Advantage(s)**: Highly parallel without dependencies
- **Disadvantage(s)**: Many small files
  - Metadata performance bottleneck, inode limits in file system



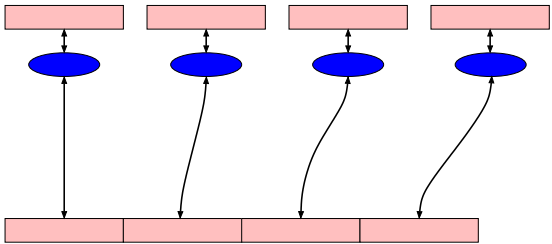
Digression: Parallel Case / Single Rank

- **Advantage(s)**:
  - Easy to implement
  - Single file
- **Disadvantage(s)**:
  - All I/O routed through a single node
  - Non-scalable approach



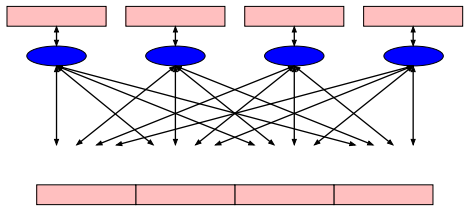
Digression: Parallel Case / Single File

- **Advantage(s)**:
  - Highly parallel
  - Single file
- **Disadvantage(s)**: High risk of concurrent storage block update
  - Possibly high costs of file block locking due to lock contention



MPI I/O

- MPI provides useful features for optimising parallel I/O
  - Non-blocking communication operations
  - Collective communication operations
  - Communicators to separate application-level message passing from I/O-related data transport
  - Support for defining application-specific data types
- **MPI I/O** uses MPI as a data transport engine to move data efficiently from or to storage
- **Performance engineering opportunity**: Cooperative I/O



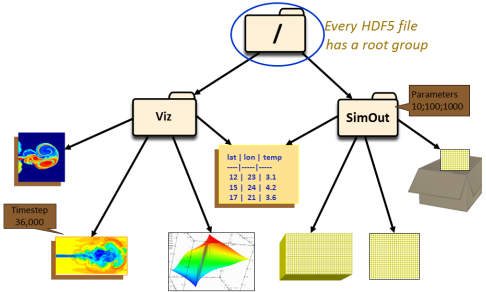
Digression: Container File Format



- **Container file format** = File format that supports combining multiple digital resources into a single digital resource
- Popular container file formats
  - Zip
  - tar
  - cpio
- Design approaches
  - Sequence of records with header
  - Optional: central directory
- **Performance engineering opportunity**: Reduce metadata operations

HDF5

- Example container file format: **HDF**
  - HDF = Hierarchical Data Format
  - A technology suite for managing (large) data collections
- HDF5 components:
  - Data model
  - API + software library
  - File format
- Features:
  - Various language bindings:
    C, Fortran, Java, C++, Python
  - Parallel I/O support based on MPI
  - Integrated support for compression



[HD5 Group]

Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

IO500 List



- The **IO500 List** shows HPC storage systems ranked according to their IO500 benchmark results
  - `https://io500.org/`
  - List is published $2\times$ per year
- The **IO500 Benchmark** is a set of workloads based on following benchmarks and tools:
  - IOR
  - mdtest
  - find

IOR Benchmark



- Benchmark designed to mimic I/O data patterns of HPC applications [Shan, Shalf; 2007]
  - Documentation: `https://ior.readthedocs.io/en/latest/`
- Features
  - Flexible support of various, regular, parallel I/O patterns
  - Support of various I/O interfaces: POSIX, MPI I/O, HDF5
  - Support of file system specific configurations (e.g. Lustre)

mdtest Benchmark

- Benchmark designed to stress file system's metadata operations
  - The benchmark has been merged with IOR

```
-- started at 10/19/2025 11:05:51 --

mdtest-1.9.3 was launched with 1 total task(s) on 1 node(s)
Command line used: ../git/mdtest -i 100 -n 1000
Path: /home/pleiter/hpc/prj/io/benchmarks/mdtest
FS: 787.1 GiB   Used FS: 86.2%   Inodes: 50.1 Mi   Used Inodes: 6.1%

1 tasks, 1000 files/directories

SUMMARY: (of 100 iterations)
   Operation                     Max            Min           Mean        Std Dev
   ---------                     ---            ---           ----        -------
   Directory creation:      63696.888      13284.010      43860.525      10303.765
   Directory stat    :     804836.747     377843.413     659962.999      66531.147
   Directory removal :     117406.927      59016.419      96714.344       9697.831
   File creation     :     109980.995      61351.097      90665.627       8066.919
   File stat         :     788692.671     377118.842     653536.902      70682.924
   File read         :     595248.016     327637.787     486837.878      47124.698
   File removal      :     210986.177      98456.827     171803.624      20928.307
   Tree creation     :      84210.526      15029.684      49078.978      20963.585
   Tree removal      :       2052.301        633.642       1533.078        235.798

-- finished at 10/19/2025 11:05:57 --
```

IO500: IOEasy



- IOR benchmark using well optimized I/O patterns
  - Sequential read/write
  - Each process creates it's own file
  - Use of large records: `transferSize`$=2$ MiB
  - Large file size: `blockSize`$=9\,920\,000$ MiB
- Select results for EuroHPC pre-exascale system **Leonardo** [IO500; 2025]
  - Lustre based on DDN's Exascaler

Workload	Result
Easy write	1552 GiB/s
Easy read	1886 GiB/s

IO500: IOHard



- IOR benchmark mimicking applications that require a random workload
  - Random read/write
  - Shared file
  - Use of smaller records: `transferSize`$=47\,008\,\mathrm{B}$

Question: What could be the motivation for this choice of `transferSize`?

IO500: IOHard



- IOR benchmark mimicking applications that require a random workload
  - Random read/write
  - Shared file
  - Use of smaller records: `transferSize`$=47\,008\,\mathrm{B}$

Question: What could be the motivation for this choice of `transferSize`?

This value does not match typical file system and hardware block sizes

IO500: IOHard



- IOR benchmark mimicking applications that require a random workload
  - Random read/write
  - Shared file
  - Use of smaller records: `transferSize`$=47\,008\,\mathrm{B}$

Question: What could be the motivation for this choice of `transferSize`?

This value does not match typical file system and hardware block sizes

- Select results for EuroHPC pre-exascale system **Leonardo** [IO500; 2025]
  - Lustre based on DDN's Exascaler

Workload	Result
Hard write	$155\,\mathrm{GiB/s}$
Hard read	$933\,\mathrm{GiB/s}$

  - Note: $10\times$ and $2\times$ worse compared to IOEasy

IO500: MDEasy



- mdtest benchmark operating on small objects
- Select results for EuroHPC pre-exascale system **Leonardo** [IO500; 2025]
  - Lustre based on DDN's Exascaler

Workload	Result
Easy write	$397\,\mathrm{kIOP/s}$
Easy stat	$807\,\mathrm{kIOP/s}$
Easy delete	$335\,\mathrm{kIOP/s}$

Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

iostat

- Standard Linux tool that provides input/output statistics for devices and partitions
- Good for monitoring while I/O is ongoing

```
1  Linux 6.8.0-85-generic (zam344)          19/10/25        _x86_64_        (8 CPU)
2
3  avg-cpu:  %user   %nice %system %iowait  %steal   %idle
4             6.37    0.06   10.65    0.29    0.00   82.63
5
6  Device            r/s     rkB/s   rrqm/s  %rrqm r_await rareq-sz     w/s     wkB/s   wrqm/s  %wrqm w_await wareq-sz     d/s     dkB/s   drqm/s  %drqm d_await dareq-sz     f/s f_await  aqu-sz  %util
7  nvme0n1          7.38    191.84     3.02  29.00    0.21    25.99    7.29    248.45     9.98  57.81    2.94    34.10    0.00     0.00     0.00   0.00    0.00     0.00    0.59    0.72    0.02   0.52
8
9
10 avg-cpu:  %user   %nice %system %iowait  %steal   %idle
11            3.77    0.00   23.30    1.01    0.00   71.92
12
13 Device            r/s     rkB/s   rrqm/s  %rrqm r_await rareq-sz     w/s     wkB/s   wrqm/s  %wrqm w_await wareq-sz     d/s     dkB/s   drqm/s  %drqm d_await dareq-sz     f/s f_await  aqu-sz  %util
14 nvme0n1         67.50    270.00     0.00   0.00    0.49     4.00 3003.50 1254186.00    60.00   1.96    3.74  417.57    0.00     0.00     0.00   0.00    0.00     0.00    0.00    0.00   11.28  35.60
```

- Similar other tools: `iotop`, `sysstat`

Linux perf



- Trace all I/O-related system calls for a specific command

```
sudo perf trace -e read,write dd if=/dev/zero of=/dev/null count=5
```

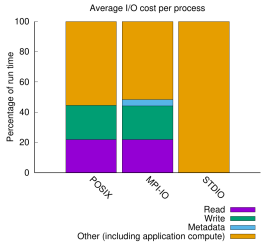
- Collect aggregate information using the Linux Kernel Tracepoint API:

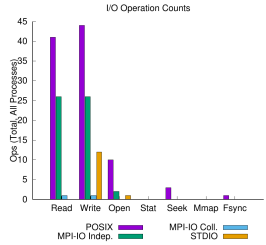
```
sudo perf trace --duration 5 \
     -e block:block_rq_issue,block:block_rq_complete \
     dd if=/dev/zero of=/dev/null count=5
```

  - `block_rq_issue`: traces I/O requests that have been issued
  - `block_rq_complete`: traces I/O requests that have completed

Darshan

- Darshan is an HPC I/O characterisation tool
  - Documentation: `https://www.mcs.anl.gov/research/projects/darshan/`
- Profiling approach: Intercept I/O calls by pre-loading a shared library
  - No recompilation is required
  - Intercepts the most relevant POSIX and MPI I/O functions
- Various tools available to process profiling files





Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. **Final**

Summary Lecture #15



- Collective communication optimization
  - Data packing, use of MPI derived data types
  - Impact of single process delays
- Background on storage architectures
  - OS-level software
  - Hardware
  - I/O APIs
- I/O benchmarks and monitoring tools
- I/O performance challenges
  - Expensive metadata operations
  - Low I/O bandwidth

Questions?



[Wiki Media]



university of groningen

Performance Engineering: Lecture #16

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-10-21

Overview



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

Challenge: Expensive Metadata Operations



- **Performance challenge**: Metadata operations can be expensive
  - This is particular relevant for parallel file systems
- **Optimization strategy**: Reduce metadata operations
  - Reduce the number of created files (e.g. use container file formats like **HDF5**)
  - Avoid check frequent file status checks
  - In case of parallel I/O: Use a shared file for multiple MPI ranks
  - In case the number of metadata operations cannot be reduced: Use file systems that are optimized for metadata handling

Challenge: Low I/O Bandwidth



- **Performance challenge**: I/O bandwidth is low compared to hardware capabilities
  - Note: Due to the complexity of storage subsystems, determining attainable I/O bandwidth can be difficult
  - Consider using benchmarks like IOR to determine attainable performance
- **Optimization strategies**:
  - Improve I/O patterns
  - In case of data locality: Use I/O buffers
  - Hide latency by means of I/O buffers or asynchronous I/O
  - In case of parallel I/O: Use cooperative I/O mechanisms (e.g. by using **MPI I/O**)

Improving I/O Patterns

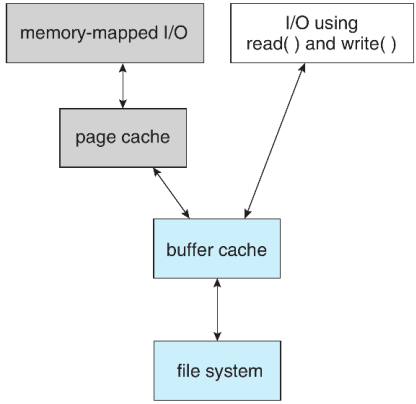


- **Sequential I/O**
  - Likely higher utilization of each block read from/written to storage
  - Benefit from lower-level optimization techniques like **read-ahead**
- **Large I/O records**
  - Records should be large with respect to file or storage hardware block size
  - Obtain required information: `stat -f <path>`
  - Example HAICGU:

```
1 [dpleiter@dev ~]$ stat -f /home/.
2   File: "/home/."
3     ID: 0         Namelen: 255     Type: nfs
4 Block size: 131072     Fundamental block size: 131072
5 Blocks: Total: 61022272   Free: 13962150   Available: 13962150
6 Inodes: Total: 781293760  Free: 769161482
```

Buffer Cache (1/2)

- **Strategy**: Maintain a buffer with copies of file system blocks in memory
  - Rationale:
    - Memory accesses are faster than storage device accesses
    - Much smaller penalties for small read-write accesses
- If a read or write request occurs the OS checks whether block is in the buffer cache
  - **Yes**: Execute I/O operation
  - **No**: Load block first before executing the I/O operation
- Need for a **replacement algorithm**
  - Typical choice: **Least Recently Used (LRU)**



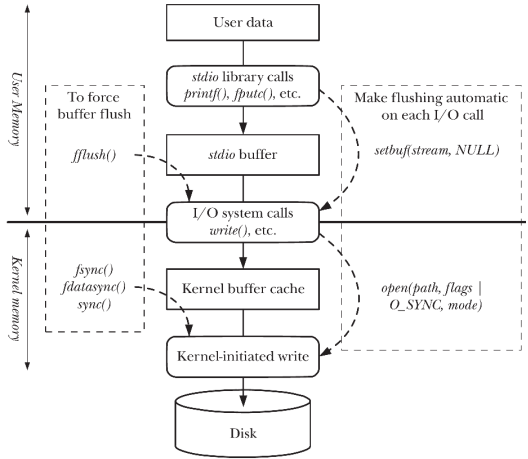
Buffer Cache (2/2)



- **Challenge**: The buffer cache and the virtual memory system compete for physical memory
  - This affects the choice of the buffer cache size
  - In case **buffered I/O** (usually the default) is not wanted, use **direct I/O**
    - Provide the `O_DIRECT` to `open()`
    - Databases often use direct I/O because they manage their own buffers
- **Optimizations** based on buffer caches: **Read-ahead**
  - If the OS detects a streaming read access to storage then it can trigger early read of blocks into the buffer cache
    - This applies to a case where the program reads from block $k, k+1, k+2, \ldots$
  - **Performance engineering opportunity**: Make sequential I/O easy detectable

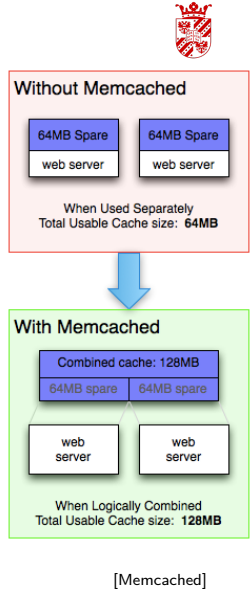
Linux I/O Buffering

- I/O can be buffered at **multiple levels**
  - Kernel level: buffer cache
  - **User level: POSIX stream buffers**
    - Used by functions like `fprintf()`, `fputs()`, `fscanf()`, `fgets()`
- Functions to manage POSIX stream buffers
  - `setvbuf()`: Assign buffering to a stream and set the type of buffering
  - `fflush()`: Flush a stream



[M. Kerrisk; 2010]

In-Memory Caches



- **Idea**: Keep frequently used data objects in an (distributed) in-memory cache
- **Example**: memcached [Brad Fitzpatrick; 2004]
  - In-memory key-value store with a distributed hash table

[Memcached]

Asynchronous I/O



- **Idea**:
  - Submit I/O read or write requests
  - Non-blocking function for submitting request
  - Blocking function to check to request completion
- **Example**: Linux-native asynchronous I/O facility `libaio`
  - Documentation: `https://pagure.io/libaio`

Question: Why is asynchronous I/O more important for read operations assuming the existence of a cache buffer?

Asynchronous I/O



- **Idea**:
  - Submit I/O read or write requests
  - Non-blocking function for submitting request
  - Blocking function to check to request completion
- **Example**: Linux-native asynchronous I/O facility `libaio`
  - Documentation: `https://pagure.io/libaio`

Question: Why is asynchronous I/O more important for read operations assuming the existence of a cache buffer?

Synchronous, blocking write operations typically return after data has been placed in the buffer cache. Synchronous, blocking read operations can likeley only complete once data arrived from disk.

libaio Example



```c
#include <libaio.h>
#include <stdio.h>
#include <fcntl.h>
#include <string.h>
#include <unistd.h>

int main() {
    io_context_t ctx = 0;                struct iocb cb;          struct iocb *cbs[1];
    struct io_event events[1];           int fd;                  int ret;
    struct timespec timeout = {0, 0};    char buf[128] = {0};

    io_setup(10, &ctx);                                  // 10 is the number of concurrent operations
    fd = open("slide.tex", O_RDONLY, 0644);

    io_prep_pread(&cb, fd, buf, 127, 0);                 // Initiate read
    cbs[0] = &cb;
    io_submit(ctx, 1, cbs);

    // DO SOMETHING ELSE

    io_getevents(ctx, 1, 1, events, &timeout);           // Wait for read to complete
    puts(buf);

    // Clean-up
    close(fd); io_destroy(ctx);

    return 0;
}
```

Content



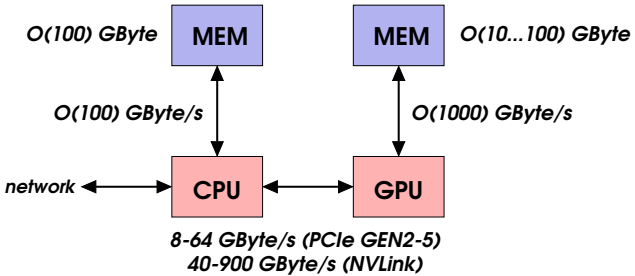
1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

GPU Computing



- GPU architecture is optimized for a **high degree of regular parallelism**
- **Relevant features**
  - High memory bandwidth
  - Highly multithreaded
  - Hardware thread scheduling
- GPUs enabled for HPC
  - NVIDIA: Hopper, Blackwell
  - AMD: Instinct MI250, Instinct MI300A
  - Intel: Ponte Vecchio
- GPUs are (typically) **not stand-alone devices** ☞ A **host CPU is required**

GPU Node Architecture



- Different memory devices
  - **Host memory** attached to CPU
  - **Device memory** attached to GPU
- Capacity and performance differ by $O(2 \dots 10)$

NVIDIA H100 Architecture

- NVIDIA Hopper architecture was introduced in 2022
- Up to 132 **Streaming Multiprocessors (SM)** with a shared L2 cache

- **Memory** (HBM3/HBM3e)
- **Host interface** (PCIe GEN5/NVLINK-C2C)
- **GPU-GPU interconnect** (NVLink)
- **Thread block scheduler** (GigaThread Engine)

[NVIDIA; 2022]



NVIDIA H100 SM

[NVIDIA; 2022]

- 4 processing blocks
- Processing per block
  - 16 FP64 units
  - 32 FP32 units
  - 1 mixed-precision Tensor Core
  - 8 load/store units
- Storage per SM
  - $4 \times 16,384$ 32-bit registers
  - 256 kByte shared memory/L1D cache
- Instructions
  - 1 shared L1I and 4 L0D caches
  - 4 warp schedulers
  - 4 dispatch units



NVIDIA H100 FP64/FP32 Performance



- IEEE 754-2008 double-/single-precision FMA
- Performance without tensor cores:
  - 2 Flop/cycle per floating-point unit
  - Double precision performance:
    $(132\,\text{SM}) \cdot (4 \cdot 16\,\text{FP64/SM}) \cdot (2\,\text{Flop/FP64/cycle}) = 16.896\,\text{Flop/cycle}$
    ☞ **Concurrency level required for high performance**
  - Assume $f = 1.98$ GHz (boost clock) $\Rightarrow$ $B_{\text{FP64}} = 33.5$ TFP64/s
  - FP32 operations throughput is $2\times$ larger

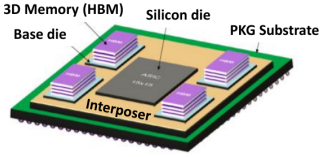
NVIDIA H100 Tensor Cores



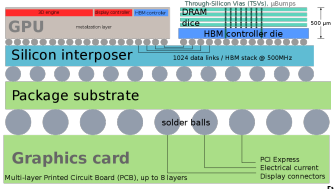
- Each Tensor Cores performs **matrix-multiply-accumulation**: $C \leftarrow A \times B + C$
- **Case of FP32**:
  - A, B, and C are 8×8 matrices
  - $B_{\mathrm{FP32,tensor}} = 8^3\,\mathrm{FP32/cycle/TC} = 270.336\,\mathrm{FP32/cycle/GPU}$
  - Assume $f = 1.83\,\mathrm{GHz}$ (boost clock) $\Rightarrow$ $B_{\mathrm{FP32}} = 494.7\,\mathrm{TFlop/s}$
- **Other data formats**
  - FP64
  - FP16, BFLOAT16
  - FP8 (different variants)
  - INT8

NVIDIA H100 Memory

- In-package HBM3 memory
- 5 memory stacks, $16\,\text{GByte/stack}$
  $\Rightarrow$ $C_{\text{mem}} = 80\,\text{GiB}$ **per GPU**
- A very wide bus of $5 \cdot 2 \cdot 512\,\text{bit}$ and a data rate of about $5\,\text{GT/s}$
  $\Rightarrow$ $B_{\text{mem}} = 3.2\,\text{TB/s}$ **per GPU**



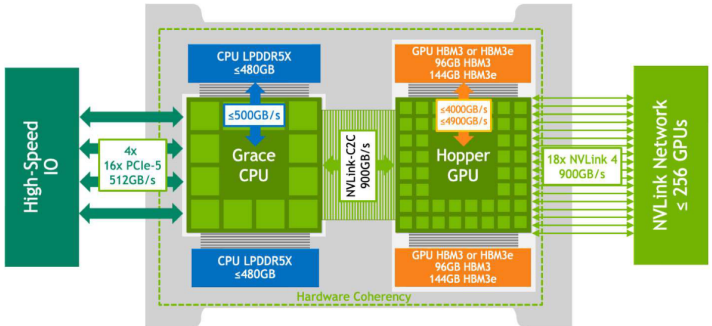
[H. Jun (SK Hynix), 2013]



[Wikipedia]

NVIDIA Grace-Hopper Superchip GH200

[NVIDIA; 2025]

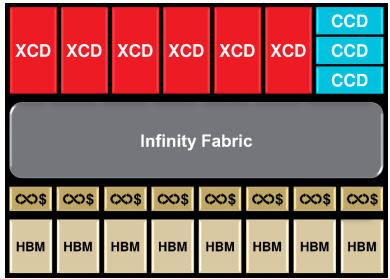


- NVIDIA started to integrate a CPU and GPU on a single daughterboard
- CPU-GPU Interconnect: NVLINK-C2C

AMD Instinct MI300a

- **Accelerated processing unit (APU) design**
  - CPU and GPU in a single package (multiple chiplets on an interposer)
  - Both access the same memory
  - $B_{\mathrm{FP64,vector}} = 61.3\,\mathrm{TFlop/s}$
  - $B_{\mathrm{FP64,matrix}} = 122.6\,\mathrm{TFlop/s}$
- $6\times$ accelerated compute dies (XCD)
- $3\times$ core complex dies (CCD)
  - $8\times$ Zen-4 cores per CCD
- $8\times$ HBM stacks
  - $B_{\mathrm{mem}} = 5.3\,\mathrm{TB/s}$
  - $C_{\mathrm{mem}} = 128\,\mathrm{GiB}$

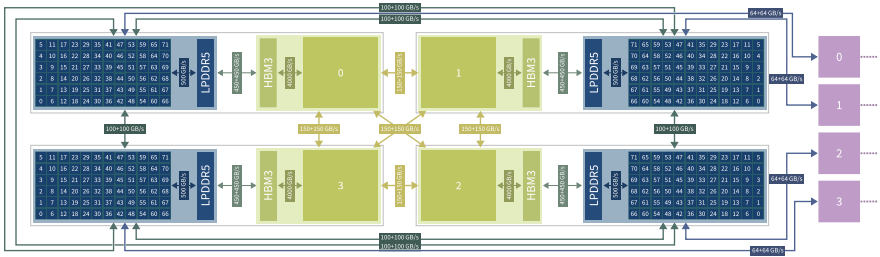
[J. Potyka et al. (AMD); 2025]



Multi-GPU Node

JUPITER node design with $4\times$ GH200 superchips:

[JSC; 2025]



- NVLink-C2C CPU-to-GPU links
- NVLink GPU-to-GPU links
- PCIe GEN5 links

Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. **GPU Programming Models**
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

GPU Programming



- **Accelerator model**
  - Main application running on CPU
  - Kernel offload on GPU
  - Overall performance may suffer from Amdahl's law
- Typical kernel execution flow:
  - Allocate memory on GPU
  - Initialise memory on GPU or transfer data from host to GPU
  - Launch kernel
  - Transfer results from GPU to host
  - Deallocate memory on GPU

OpenMP



- As of version 4.0, the OpenMP API supports accelerators/coprocessors
- **Device model**
  - Host CPU (multi-threading)
  - One or more accelerator devices
- New construct: `target`

Example: $\vec{y} \leftarrow a \cdot \vec{x} + \vec{y}$

```
1  void saxpy() {
2    float a, x[SZ], y[SZ];
3    double t = 0.0;
4    double tb, te;
5
6    tb = omp_get_wtime();
7
8    #pragma omp target
9    for (int i = 0; i < SZ; i++) {
10     y[i] = a * x[i] + y[i];
11   }
12
13   te = omp_get_wtime();
14   t = te - tb;
15   printf("Time of kernel: %lf\n", t);
16 }
```

CUDA



- **CUDA** = Compute Unified Device Architecture
- C-like parallel programming language
- First SDK released in 2007
- Current version: 13.0.2 (October 2025)
- Language developed and controlled by NVIDIA
  - Only supported GPU devices are from NVIDIA
  - Available for x86, POWER, and Arm host CPUs

CUDA Kernel



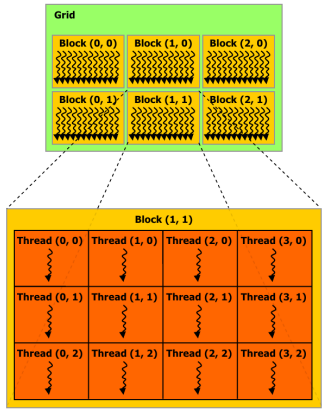
- **Kernel** = C function which is executed in parallel by $N \geq 1$ different threads
- Declaration specifiers:

`__global__`	Function on device, callable from host
`__device__`	Function on device, callable from device
`__host__`	Function on host, callable from host (default)

- Kernel launch syntax: `mykernel<<<Dg,Db>>>(...)`
  - `Dg`: Size of the grid
  - `Db`: Size of each block
- Each thread is (within a block) given a unique ID: `threadIdx`
  - `threadIdx` is structure with 3 components: `x`, `y`, `z`

CUDA Thread Hierarchy

- **Block** = Set of threads
  - 1-, 2- or 3-dimensional index
  - Synchronization within threads possible
  - Maximum number of threads: 1024
  - Branching with penalty
  - Identifier: `threadIdx`
- **Grid** = Set of blocks
  - 1-, 2- or 3-dimensional index
  - Synchronization not possible
  - Branching without penalty
  - Identifier: `blockIdx`



CUDA Vector Copy



```
1  #include <stdio.h>
2  #define N 32
3
4  __global__ void copy(float *b, float *a) {
5    int i = threadIdx.x;
6    b[i] = a[i];
7  }
8
9  int main() {
10   int i;
11   float ha[N], hb[N];                    /* Arrays on host */
12   float *da, *db;                        /* Pointer to arrays on device */
13
14   for (i = 0; i < N; i++) ha[i] = i;
15
16   cudaMalloc((void **) &da, N * sizeof(float));
17   cudaMalloc((void **) &db, N * sizeof(float));
18
19   cudaMemcpy(da, ha, N*sizeof(float), cudaMemcpyHostToDevice);
20
21   dim3 Dg(1,1,1); dim3 Db(N,1,1);
22   copy<<<Dg, Db>>>(db, da);
23
24   cudaMemcpy(hb, db, N*sizeof(float), cudaMemcpyDeviceToHost);
25
26   return 0;
27 }
```

Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. **Device Utilization and Data Transfer**
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

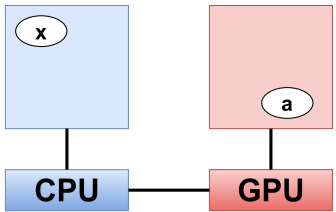
Overall Speed-up



- According to Amdahl's Law, the overall speed-up depends on
  - Speed-up factor S of the accelerated code parts ☞ Topic of the next section
  - Fraction f of the code that is accelerated
- Challenges for increasing f
  - A large fraction of the tasks run on the CPU only, while GPU is idle
  - Time spent on CPU↔GPU data transfer is large
- Performance engineering strategies
  - Port more tasks to the GPU
  - Overlap different tasks
  - Avoid unnecessary data transfer

Host versus Device Memory

- Device-accelerated nodes comprise of different memories
  - The address spaces are different, i.e. pointer can typically not be used for both memories
- Options for managing data
  1. Explicit data management by applications
  2. Implicit data management by software
  3. Implicit data management by hardware
- Implicit data management can be supported by hints



Overlapping Tasks



- **Consider the following sequence of tasks**:
  1. Start the application and initialise data structures on the CPU
  2. Transfer data from CPU to GPU memory
  3. Run a kernel on the GPU
  4. Transfer data from GPU to CPU memory
  5. Finish the application on the CPU
- **Latency in case of lacking overlap**:

$$\Delta t = \Delta t^{(1)}_{\text{CPU}} + \Delta t^{(2)}_{\text{CPU}\rightarrow\text{GPU}} + \Delta t^{(3)}_{\text{GPU}} + \Delta t^{(4)}_{\text{GPU}\rightarrow\text{CPU}} + \Delta t^{(5)}_{\text{CPU}}$$

- **Opportunities for overlapping tasks supported by the hardware**:
  - Tasks on CPU and GPU can execute independently
  - GPU DMA engines can transfer data independent of tasks running on CPU or GPU

Matrix-Vector Multiplication (1/3)



Example: $\vec{y} \leftarrow A \cdot \vec{x}$ (symmetric matrix of large size N)

Task	**Latency-bandwidth model**
Initialise A and x on CPU	$\Delta t^{(1)}_{\text{CPU}} = \lambda^{(1)} + (N^2 + N) \cdot w/\beta_{\text{mem,CPU}}$
Transfer A and x from CPU to GPU	$\Delta t^{(2)}_{\text{CPU}\to\text{GPU}} = \lambda^{(2)} + (N^2 + N) \cdot w/\beta_{\text{CPU}\leftrightarrow\text{GPU}}$
Compute $A \cdot \vec{x}$	$\Delta t^{(3)}_{\text{GPU}} = \lambda^{(3)} + (2 \cdot N^2 + N) \cdot w/\beta_{\text{mem,GPU}}$
Transfer y from GPU to CPU	$\Delta t^{(4)}_{\text{GPU}\to\text{CPU}} = \lambda^{(4)} + N \cdot w/\beta_{\text{CPU}\leftrightarrow\text{GPU}}$
Finish the application on the CPU	—

w ... size of the floating-point numbers

Matrix-Vector Multiplication (2/3)



Question: Is it a good idea to use the GPU in this case?

Matrix-Vector Multiplication (2/3)



Question: Is it a good idea to use the GPU in this case?

Most likely not!

- Typically, $\beta_{\text{mem,CPU}} \ll \beta_{\text{mem,GPU}}$
  $\Rightarrow$ Matrix-vector multiplication is expected to run faster on GPUs
- **But**: $\beta_{\text{CPU}\leftrightarrow\text{GPU}} \ll \beta_{\text{mem,GPU}}/2$
  $\Rightarrow$ Likely, most time will be spent on CPU$\leftrightarrow$GPU data transfer

Matrix-Vector Multiplication (3/3)



- Consider the **alternative example** where for $i = 0, 1, \ldots, M-1$ we need to compute

$$\vec{y}^{(i)} \leftarrow A \cdot \vec{x}^{(i)}$$

Question: Why is this example more suitable?

Matrix-Vector Multiplication (3/3)



- Consider the **alternative example** where for $i = 0, 1, \ldots, M-1$ we need to compute

$$\vec{y}^{(i)} \leftarrow A \cdot \vec{x}^{(i)}$$

Question: Why is this example more suitable?

- A should be copied only once
- For large M, the CPU↔GPU data transfer is dominated by transfer of $\vec{x}^{(i)}$ and $\vec{y}^{(i)}$
- Scaling according to the latency-bandwidth model

$$\begin{aligned} \Delta t^{(2,4)}_{\mathrm{CPU}\leftrightarrow\mathrm{GPU}} &\propto M \cdot 2 \cdot N \cdot w/\beta_{\mathrm{CPU}\leftrightarrow\mathrm{GPU}} \\ \Delta t^{(3)}_{\mathrm{GPU}} &\propto M \cdot (2 \cdot N^2 + N) \cdot w/\beta_{\mathrm{mem,GPU}} \end{aligned}$$

Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

Introduction



- Optimization targets considered in this section:
  - Instruction-throughput-limited applications: **maximise SM utilization**
    - Example: **Occupancy** = Ratio of active warps on an SM to the maximum number of active warps supported by the SM
  - Memory-performance-limited applications: **maximize memory interface utilization**

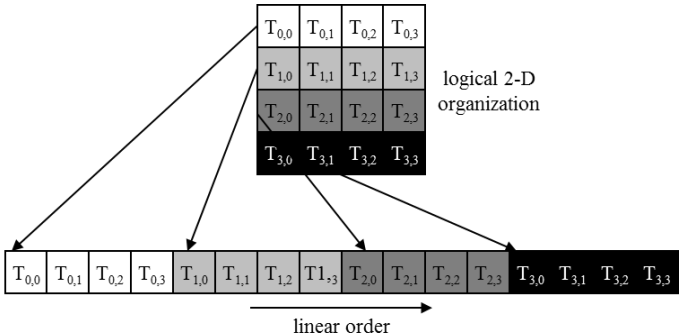
Warps as Scheduling Units



- Each block is divided into **thread warps** running multiple threads
  - An implementation technique (not part of the CUDA programming model)
  - Warps are scheduling units in SM
  - Threads in a warp execute in Single Instruction Multiple Data (SIMD) manner
  - The number of threads in a warp may vary in future generations

Warps in Multi-dimensional Thread Blocks

- The thread blocks are first linearized into 1D in row major order
  - In x-dimension first, y-dimension next, and z-dimension last



Blocks are Partitioned after Linearization



- Linearized thread blocks are partitioned
  - Thread indices within a warp are consecutive and increasing
  - Warp 0 starts with Thread 0
- Partitioning scheme is consistent across devices
  - Thus you can use this knowledge in control flow
  - However, the exact size of warps may change from generation to generation
- DO NOT rely on any ordering within or between warps
  - If there are any dependencies between threads, you must `__syncthreads()` to get correct results

SMs are SIMD Processors



- SM acts as control unit for instruction fetch, decode, and control for multiple shared processing units
- All threads in a warp must **execute the same instruction at any point in time**
- This works efficiently if all threads follow the same control flow path
  - All if-then-else statements make the same decision
  - All loops iterate the same number of times

Control Divergence



- Control divergence occurs when threads in a warp take different control flow paths by making different control decisions
  - Some take the then-path and others take the else-path of an if-statement
  - Some threads take different number of loop iterations than others
- The execution of threads taking different paths are serialized in current GPUs
  - The control paths taken by the threads in a warp are traversed one at a time until there is no more
  - During the execution of each path, all threads taking that path will be executed in parallel
  - The number of different paths can be large when considering nested control flow statements

Control Divergence Examples



- Divergence can arise when branch or loop condition is a function of thread indices
- Example kernel statement with divergence:

```
1 if (threadIdx.x > 2)
2 { ... }
```

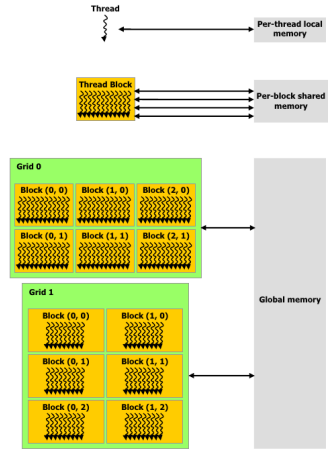
  - This creates two different control paths for threads in a block
  - Decision granularity $<$ warp size
  - Threads 0, 1 and 2 follow different path than the rest of the threads in the first warp
- Example without divergence:

```
1 if (blockIdx.x > 2)
2 { ... }
```

  - Decision granularity is a multiple of blocks size
  - All threads in any given warp follow the same path

CUDA Memory Hierarchy

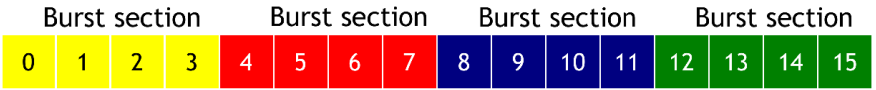
- Each thread can access registers and local memory
- Shared memory is visible to all threads of a block
- **Global memory space**
  - Resides in device memory
  - Accessible by all threads
- **Texture memory space**
  - Resides in device memory
  - Read via texture cache optimized for 2-d locality
- **Constant memory space**
  - Resides in device memory
  - Read-only, not dependent on thread ID



Memory Coalescing (1/2)

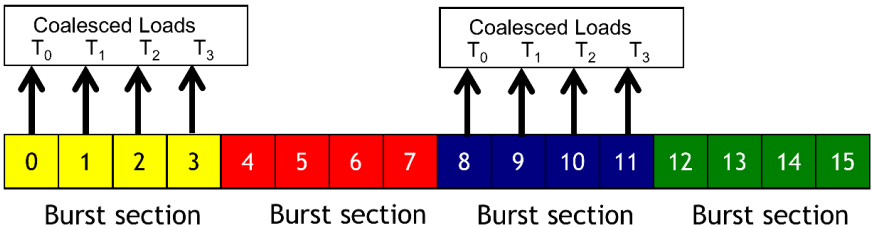


- Digression: DRAM organization
  - Each address space is partitioned into burst sections
  - Whenever a location is accessed, all other locations in the same section are also delivered to the processor



Memory Coalescing (2/2)

- Assume all threads of a warp execute a load instruction where all accessed locations fall into the same burst section ⇒ Only one DRAM request will be made, i.e. the access is fully **coalesced**



Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. **Multi-Device Communication**
7. Final

GPU-Aware MPI

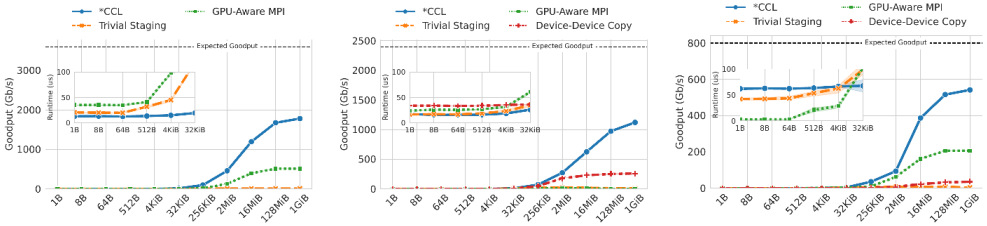


- **Challenge**: By default, MPI can only communicate data in the host memory
  - Before send (after receive) data needs to be copied from device to host (host to device)
- **Idea**: Allow network cards to transfer data from and to device
  - Benefit: Data is directly moved from device memory to network (and vice versa) without using host memory or host cores
  - Role of the host: Setup and guide the communication

GPU-to-GPU Communication

- **Challenge**: Collective communication operations involving all GPUs within a node
  - Should only use fast links connecting the GPUs
  - Possible difficulty: Complex topology
- **Solution**: Specialised libraries
  - NVIDIA Collective Communications Library (NCCL)
  - AMD ROCm Collective Communication Library (RCCL)

Example: Intra-node all-reduce performance [D. de Sensi et al.; 2024]



(a) Alps (b) Leonardo (c) LUMI

Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. **Final**

Summary Lecture #16



- I/O performance optimization techniques
  - Use container file formats
  - Improve I/O patterns
  - Use OS I/O buffers or I/O cache solutions
  - Do parallel I/O cooperatively
- Overview of different Hardware architectures and programming models
- Performance challenges
  - Device utilization
    - Host-device data transfer
  - Device kernel optimization
    - Optimizing concurrency: high occupancy, avoid control divergence
    - Efficient use of memory hierarchy: data reuse, memory coalesing
  - Efficient communication: GPU-aware MPI, GPU-GPU communication

Questions?



[Wiki Media]

These slides contain content and figures from

- Wen-mei Hwu et al., “GPU Teaching Kit”, 2020